The Code, Line by Line

Every line of the retrieval-augmented generation system over Jordanian legislation, printed with its line number and explained.

Covered	4,145 lines across 17 modules - the complete rag/ package
Format	Source with line numbers, in blocks, each block followed by what it does and why
Guarantee	The generator refuses to build unless the notes cover every line of every module: no gaps, no overlaps
Companion	RAG-system-explained.pdf covers the architecture and the data flow; this document covers the code itself

Generated by `docs/make_code_pdf.py` directly from the source files.

How to read this document

Each module begins with a one-paragraph summary of its job. The source then follows in blocks: the real code with its real line numbers, then a paragraph explaining what those lines do. Individual lines are called out in **bold** inside the explanation where they deserve their own sentence.

- **Nothing is skipped.** The blocks are checked to tile each file exactly, so imports, blank lines and closing brackets are all inside some block. Where a run of lines is self-evident, the block note covers them together rather than repeating the obvious.
- **The code is read from disk at build time.** Nothing here is retyped, so the listing cannot drift from what actually runs.
- **Explanations favour the reason over the mechanism.** That a line assigns a variable is visible; why the variable exists, and what breaks without it, is not.

The reading order follows the data: settings, then the ingestion path (reading documents, repairing Arabic, finding articles, chunking, embedding, storing), then the query path (retrieval, reranking, the model, the answer), then the interfaces that tie them together.

> **If a line looks strange, it usually has a story.** Several lines in this code exist because of a specific failure: a Windows file lock, a PDF that extracts backwards, an index that could silently misalign with its vectors. Those notes say so, because the alternative is a future reader removing the line.

Contents

`rag/config.py` 207 lines . . . 4

`rag/loaders.py` 578 lines . . . 9

`rag/arabic.py` 273 lines . . . 21

`rag/legal.py` 146 lines . . . 28

`rag/chunking.py` 356 lines . . . 32

`rag/embeddings.py` 170 lines . . . 40

`rag/bm25.py` 130 lines . . . 44

`rag/store.py` 367 lines . . . 48

`rag/retriever.py` 267 lines . . . 56

`rag/crossencoder.py` 100 lines . . . 62

`rag/llm.py` 396 lines . . . 65

`rag/generate.py` 164 lines . . . 73

`rag/rerank.py` 80 lines . . . 77

`rag/pipeline.py` 269 lines . . . 79

`rag/cli.py` 388 lines . . . 85

`rag/server.py` 216 lines . . . 93

`rag/__init__.py` 38 lines . . . 97

rag/config.py

207 lines. Every setting in the system, in one dataclass, plus the loader that reads a .env file. This module is imported before any other, because the defaults below are evaluated at import time from the environment.

```
"""Central configuration for the RAG system.

Settings come from a ``.env`` file next to the project (or in the working
directory), from environment variables, or from a ``Config`` passed in code --
in reverse order of precedence, so a real environment variable always wins over
the file. Defaults are chosen to work offline for everything except answer
generation, which needs a Gemini or Anthropic key.
"""
```

The module docstring states the precedence rule the rest of the file implements: a value passed in code beats a real environment variable, which beats the .env file, which beats the built-in default. Writing it at the top means the rule is stated once and the code below only has to obey it.

```

from __future__ import annotations

import os
from dataclasses import dataclass, field, asdict
from pathlib import Path
from typing import Any
```

Line 10 makes all type annotations lazy strings, which is what allows modern syntax such as `Path | None` on older interpreters and avoids import cycles. **12-15** are the only imports this module needs: `os` for the environment, dataclasses for the settings object, `Path` for filesystem paths, and `Any` for the dictionary return types.

```

ENV_FILENAME = ".env"


```

The filename is a named constant rather than the literal ".env" scattered through the code, because the CLI prints it back to the user when no key is found - and a message that names the wrong file is worse than no message.

```
def load_env_file(path: Path) -> dict[str, str]:
    """Read ``KEY=value`` lines into the environment, without overriding it.

    A deliberately small parser rather than a dependency: comments, blank
    lines, an optional ``export`` prefix and quoted values are all that a file
    holding a couple of API keys needs. An empty value is skipped, so a
    half-filled template cannot look like a configured key.
    """
```

`load_env_file` reads one file. The docstring justifies writing a parser instead of adding the python-dotenv dependency: for a file holding two API keys, the whole grammar is comments, blank lines, an optional export prefix and quotes. The last sentence describes a deliberate safety property implemented on line 43.

```
    loaded: dict[str, str] = {}
    try:
        text = path.read_text(encoding="utf-8")
    except (OSError, UnicodeDecodeError):
        return loaded
```

28 collects what this call actually set, which the CLI later shows as "from .env: GEMINI_API_KEY". **29-32** read the file, and a missing or unreadable file returns an empty dictionary rather than raising: configuration is optional, so its absence must not stop the program. Catching `UnicodeDecodeError` as well covers a file saved in another encoding.

```
33      for line in text.splitlines():
34          line = line.strip()
35          if not line or line.startswith("#") or "=" not in line:
36              continue
37          if line.startswith("export "):
38              line = line[len("export "):]
39          key, _, value = line.partition("=")
40          key, value = key.strip(), value.strip().strip("'\"").strip()
```

The parse loop. **34** strips surrounding whitespace. **35** skips blank lines, comments, and any line without an "=" - that last test is what lets prose sit in the file without breaking it. **37-38** drop a leading `export `, so a file copied from a shell script works. **39** splits on the first "=" only, so a value may itself contain "=" (base64 keys often do). **40** trims whitespace and one layer of single or double quotes.

```
41          # A real environment variable wins, so `set KEY=... && python -m rag`
42          # overrides the file for one run.
43          if key and value and key not in os.environ:
44              os.environ[key] = value
45              loaded[key] = value
46      return loaded
47
48
```

The precedence rule in one condition. **43** requires a non-empty key and a non-empty value and that the variable is not already set. The empty-value test is why the shipped template, which contains `GEMINI_API_KEY=` with nothing after it, does not look like a configured key. The `not in os.environ` test is what lets one command override the file for a single run.

```
49  def load_env_files() -> dict[str, str]:
50      """Load ``.env`` from the working directory and from the project root."""
51      project_root = Path(__file__).resolve().parent.parent
52      loaded: dict[str, str] = {}
53      for directory in dict.fromkeys([Path.cwd(), project_root]):
54          candidate = directory / ENV_FILENAME
55          if candidate.is_file():
56              loaded.update(load_env_file(candidate))
57      return loaded
```

`load_env_files` looks in two places: the directory the command was run from and the project root, so the key is found whether you run from inside the project or from somewhere else. **53** uses `dict.fromkeys` as an order-preserving way to drop the duplicate when those two directories are the same, which is the normal case.

```
58
59
60  # Must run before the Config defaults below are evaluated: they read
61  # os.environ at class-definition time.
62  LOADED_ENV = load_env_files()
```

This line runs at import time, and the comment explains why it has to: the dataclass fields below call `_env_str` and friends while the class body is being executed, so anything the .env file is going to contribute must already be in `os.environ` by then. Moving this line below the class would silently break .env support.

```
63
64
65  def _env_str(name: str, default: str) -> str:
66      return os.environ.get(name, default)
```

The four readers that follow all have the same shape: look the variable up, fall back to the default. The string version needs nothing else.

```
 67
 68
 69  def _env_int(name: str, default: int) -> int:
 70      raw = os.environ.get(name)
 71      if raw is None:
 72          return default
 73      try:
 74          return int(raw)
 75      except ValueError:
 76          return default
```

The integer version has to handle two different failures. **71-72**: the variable is absent, so use the default. **73-76**: the variable is present but not a number - a typo like `RAG_TOP_K=ten` - in which case the default is used rather than crashing the program at import time, when a traceback would point at config.py rather than at the typo.

```
 77
 78
 79  def _env_float(name: str, default: float) -> float:
 80      raw = os.environ.get(name)
 81      if raw is None:
 82          return default
 83      try:
 84          return float(raw)
 85      except ValueError:
 86          return default
```

The float version, identical in shape to the integer one.

```
 87
 88
 89  def _env_bool(name: str, default: bool) -> bool:
 90      raw = os.environ.get(name)
 91      if raw is None:
 92          return default
 93      return raw.strip().lower() in {"1", "true", "yes", "on"}
```

The boolean version accepts the spellings people actually type. **93** treats "1", "true", "yes" and "on" as true, case-insensitively, and everything else as false, so `RAG_RERANK=0` and `RAG_RERANK=false` both work.

```
 94
 95
 96  @dataclass
 97  class Config:
 98      # --- storage -----------------------------------------------------------
 99      index_dir: Path = field(
100          default_factory=lambda: Path(_env_str("RAG_INDEX_DIR", ".rag_index"))
101      )
```

The settings object itself. **96** makes it a dataclass, so the fields below get an `__init__`, a readable repr and equality for free. **99-101** need `field(default_factory=...)` rather than a plain default because the value is a mutable `Path` object; dataclasses refuse mutable defaults, since every instance would otherwise share one object.

```
102
103      # --- chunking ----------------------------------------------------------
104      # Sized so that `top_k` chunks comfortably fit in context alongside the
105      # prompt while still carrying enough surrounding text to be self-contained.
106      chunk_tokens: int = _env_int("RAG_CHUNK_TOKENS", 512)
107      chunk_overlap_tokens: int = _env_int("RAG_CHUNK_OVERLAP", 64)
108      min_chunk_chars: int = _env_int("RAG_MIN_CHUNK_CHARS", 80)
109      # Legislation: one chunk per article instead of fixed windows.
110      article_chunks: bool = _env_bool("RAG_ARTICLE_CHUNKS", True)
111      # Pack consecutive short articles up to this many tokens (0 = one per chunk).
112      # Measured on the legal test set, packing lowered retrieval quality at every
113      # context budget, so articles are indexed individually by default.
114      article_pack_tokens: int = _env_int("RAG_ARTICLE_PACK_TOKENS", 0)
115      # Prepend "<title> -- <article>" to every chunk before indexing it.
116      context_headers: bool = _env_bool("RAG_CONTEXT_HEADERS", True)
```

Chunking settings. **106-108** size an ordinary text window and the overlap between consecutive windows. **110** switches on the legal path - one chunk per article. **114** is a measured default: the comment records that packing short articles together was tried and lowered quality at every context size, so the value is 0 (never pack). **116** controls whether each chunk is indexed with its "law - article" label.

```
117
118     # --- embeddings ----------------------------------------------------------
119     # BAAI/bge-m3: 1024 dims, multilingual, strong on Arabic. Measured on the
120     # legal test set it beat multilingual-e5-small and -e5-large both before and
121     # after reranking (answer ranked first 80% vs 76% / 71%). The price is a
122     # 2.2 GB download and slower indexing on CPU; set RAG_EMBED_MODEL to
123     # intfloat/multilingual-e5-small for a fast, light alternative.
124     embed_model: str = _env_str("RAG_EMBED_MODEL", "BAAI/bge-m3")
125     embed_batch_size: int = _env_int("RAG_EMBED_BATCH", 16)
126     embed_max_tokens: int = _env_int("RAG_EMBED_MAX_TOKENS", 512)
127     embed_device: str = _env_str("RAG_EMBED_DEVICE", "auto")  # auto | cpu | cuda
128     embed_pooling: str = _env_str("RAG_EMBED_POOLING", "auto")  # auto | mean | cls
129     # e5 models are trained with asymmetric prefixes; harmless for other models
130     # only if disabled, so it is tied to the model name at runtime.
131     embed_query_prefix: str = _env_str("RAG_QUERY_PREFIX", "query: ")
132     embed_passage_prefix: str = _env_str("RAG_PASSAGE_PREFIX", "passage: ")
```

Embedding settings. The comment on **119-123** records the comparison that chose this model, including the alternative for a slower machine. **125** is the batch size for the GPU or CPU. **126** caps the input length the model sees. **127-128** are "auto" by default: the device is detected, and the pooling strategy is inferred from the model name, because choosing it wrongly degrades retrieval silently rather than failing. **131-132** hold the asymmetric prefixes that only the e5 family wants.

```
133
134     # --- retrieval -----------------------------------------------------------
135     # Most passages to consider for an answer; context_tokens then decides how
136     # many of them actually fit.
137     top_k: int = _env_int("RAG_TOP_K", 12)
138     context_tokens: int = _env_int("RAG_CONTEXT_TOKENS", 4000)
139     candidate_k: int = _env_int("RAG_CANDIDATE_K", 30)  # per retriever, pre-fusion
140     rrf_k: int = _env_int("RAG_RRF_K", 60)  # reciprocal-rank-fusion smoothing
141     # Relative trust in each retriever during fusion. Equal weights are the
142     # neutral prior; measure with eval/evaluate.py before moving them. Corpora
143     # of paraphrasable prose usually want dense above 1.0, corpora full of
144     # identifiers and error codes want lexical up.
145     dense_weight: float = _env_float("RAG_DENSE_WEIGHT", 1.0)
146     lexical_weight: float = _env_float("RAG_LEXICAL_WEIGHT", 1.0)
147     use_mmr: bool = _env_bool("RAG_USE_MMR", True)
148     mmr_lambda: float = _env_float("RAG_MMR_LAMBDA", 0.7)  # 1.0 = pure relevance
149     min_score: float = _env_float("RAG_MIN_SCORE", 0.0)
```

Retrieval settings. **137-138** are the pair that decides how much context an answer gets: at most 12 passages, and no more than 4,000 tokens of them. **139** is how deep each retriever searches before fusion, **140** the smoothing constant in the fusion formula. **145-146** let one retriever be trusted more than the other; both are 1.0 because the comment's advice is to measure before moving them. **147-148** control near-duplicate removal, where 1.0 would mean pure relevance and no diversity.

```
150     # Reorder the fused candidates with a model that reads question and
151     # passage together. "local" is an offline cross-encoder; "llm" spends one
152     # API call per query on the configured provider.
153     # On by default: on the legal test set it lifted "right passage ranked
154     # first" from 47% to 76%, for ~2 s per question on CPU.
155     rerank: bool = _env_bool("RAG_RERANK", True)
156     reranker: str = _env_str("RAG_RERANKER", "local")  # local | llm
157     reranker_model: str = _env_str("RAG_RERANKER_MODEL", "BAAI/bge-reranker-v2-m3")
158     rerank_candidates: int = _env_int("RAG_RERANK_CANDIDATES", 20)
159     reranker_batch_size: int = _env_int("RAG_RERANKER_BATCH", 8)
160     reranker_max_tokens: int = _env_int("RAG_RERANKER_MAX_TOKENS", 512)
161     reranker_quantize: bool = _env_bool("RAG_RERANKER_INT8", True)  # int8 on CPU
```

Reranking. **155** is on by default, and the comment carries the measurement that justifies the cost - 47% to 76% of questions with the answer ranked first, for about two seconds. **156** chooses the local cross-encoder over spending an API call. **158** is how many fused candidates get rescored; the reranker can only promote what it is shown. **161** turns on int8 weights, which made the large model usable on a CPU.

```
162
163      # --- generation ----------------------------------------------------------
164      # Which hosted model writes the answers. "auto" picks whichever provider has
165      # credentials, preferring Gemini (it has a free tier). Retrieval is local
166      # either way, so switching provider needs no re-indexing.
167      llm_provider: str = _env_str("RAG_LLM_PROVIDER", "auto")  # auto | gemini | claude
168      # Empty means the provider's default (see llm.py): claude-opus-5 / gemini-3.8-flash.
169      model: str = _env_str("RAG_MODEL", "")
170      max_tokens: int = _env_int("RAG_MAX_TOKENS", 16000)
171      effort: str = _env_str("RAG_EFFORT", "medium")  # low|medium|high|xhigh|max
172      stream: bool = _env_bool("RAG_STREAM", True)
173      cache_context: bool = _env_bool("RAG_CACHE_CONTEXT", True)
174      # Transient provider failures (503 overload, 429 rate limit) are retried
175      # with a doubling backoff; 4xx errors are not.
176      retry_attempts: int = _env_int("RAG_RETRY_ATTEMPTS", 2)
177      retry_initial_seconds: float = _env_float("RAG_RETRY_SECONDS", 1.5)
```

Generation. **167** defaults to "auto", which picks whichever provider has credentials. **169** is deliberately empty: an empty model name means "use this provider's default", which is how one setting serves two providers with different model names. **171** maps onto Gemini's thinking level and Claude's effort. **176-177** set the retry policy for transient provider failures.

```
178
179      def extract_cache_dir(self) -> Path | None:
180          """Where extracted text is cached (RAG_EXTRACT_CACHE; "off" disables)."""
181          override = os.environ.get("RAG_EXTRACT_CACHE")
182          if override and override.lower() in {"off", "0", "none", "false"}:
183              return None
184          return Path(override) if override else Path(self.index_dir) / "extracted"
```

Where extracted text is cached. **181-183** let the cache be switched off entirely with `RAG_EXTRACT_CACHE=off`, which the test suite uses so tests never share state on disk. **184** otherwise honours an explicit path, or puts the cache inside the index folder, so deleting the index deletes its cache too.

```
185
186      def dense_enabled(self) -> bool:
187          return _env_bool("RAG_DENSE", True)
```

A method rather than a field, because it is read at query time: setting `RAG_DENSE=0` in an already-running process takes effect on the next call, which is convenient when experimenting.

```
188
189      def to_dict(self) -> dict[str, Any]:
190          d = asdict(self)
191          d["index_dir"] = str(self.index_dir)
192          return d
```

`to_dict` serialises the settings - the HTTP API and the test suite both use it. **191** converts the Path to a string, because `Path` objects are not JSON-serialisable.

```
193
194      # Fingerprint of everything that, if changed, makes an existing index stale.
195      def index_fingerprint(self) -> dict[str, Any]:
196          return {
197              "embed_model": self.embed_model,
198              "chunk_tokens": self.chunk_tokens,
199              "chunk_overlap_tokens": self.chunk_overlap_tokens,
200              "embed_max_tokens": self.embed_max_tokens,
201              "article_chunks": self.article_chunks,
202              "article_pack_tokens": self.article_pack_tokens,
203              "context_headers": self.context_headers,
204          }
```

The fingerprint: the subset of settings that, if changed, makes an existing index invalid. It is written into the index and compared on load. Embedding vectors from two different models cannot be compared at all, and chunk boundaries decide what the vectors even describe - so a mismatch here is reported rather than silently producing wrong results. Settings not in this list, such as top_k, can be changed freely without re-indexing.

```
205
206
207  DEFAULT_CONFIG = Config()
```

One shared instance for callers that do not want to build their own. The pipeline copies it rather than mutating it, so that one component adopting an index's chunk settings cannot leak into another.

rag/loaders.py

578 lines. Turns a file on disk into plain text plus the page map that makes citations possible. Most of the length is PDF repair: the Arabic legal PDFs in this corpus arrive glued, reversed, or with their article numbers detached from their text.

```
"""Document loaders.

Each loader returns a :class:`Document` whose ``text`` is the full extracted
plain text plus ``page_spans`` -- (char_start, char_end, page_label) triples that
let a chunk report which page it came from, which is what makes PDF citations
useful instead of merely decorative.
"""
```

The docstring names the contract every loader below honours: return a `Document` whose text is the whole file, and a list of `page_spans` saying which character range came from which page. Without that second list a PDF citation could name the document but not the page, which is the difference between a usable citation and a decorative one.

```

from __future__ import annotations

import csv
import hashlib
import json
import logging
import re
from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Iterable, Iterator
```

Imports. `hashlib` provides both the document id and the cache key, `csv` and `json` serve their own loaders, and `logging` is here only to silence pypdf on line 51.

```

from .arabic import (
    average_word_length,
    fold_presentation_forms,
    merge_word_breaks,
    official_short_title,
    reverse_word_order,
    spacing_looks_broken,
    undouble_words,
    visual_to_logical,
    word_order_reversed,
)
```

Everything imported from `arabic` is a repair or a detector. Keeping them in their own module means this file decides when to repair text and that module decides how, which is what lets the repairs be tested on strings without ever opening a PDF.

```

TEXT_EXTENSIONS = {
    ".txt", ".md", ".markdown", ".rst", ".log",
    ".py", ".js", ".ts", ".tsx", ".jsx", ".java", ".go", ".rb", ".rs",
    ".c", ".h", ".cpp", ".hpp", ".sh", ".sql", ".yaml", ".yml", ".toml", ".ini",
}
SUPPORTED_EXTENSIONS = TEXT_EXTENSIONS | {
    ".pdf", ".docx", ".csv", ".tsv", ".json", ".jsonl", ".html", ".htm",
}
```

Two sets of extensions. **32-36** are files read as plain text, including source code, so the system can index a codebase as happily as a statute. **37-39** add the formats that need a real parser. Membership of this second set is what `discover` checks before offering a file for indexing.

```
40
41  # Directories that are never worth indexing even when file extensions match.
42  SKIP_DIR_NAMES = {
43      ".git", ".hg", ".svn", "node_modules", "__pycache__", ".venv", "venv",
44      ".mypy_cache", ".pytest_cache", ".rag_index", "catboost_info", ".idea",
45  }
46  MAX_FILE_BYTES = 32 * 1024 * 1024
```

42-45 are directories to walk straight past. Without this, pointing the indexer at a project directory would index its virtual environment and its own `.rag_index` folder - a system indexing its own index. **46** caps a single file at 32 MB, which keeps one pathological file from stalling an ingest.

```
47
48  # pypdf logs a warning for every malformed cross-reference entry it recovers
49  # from ("Ignoring wrong pointing object"); common in exported PDFs, harmless,
50  # and it buries the ingest output.
51  logging.getLogger("pypdf").setLevel(logging.ERROR)
```

A one-line fix for a noisy dependency. pypdf warns once per malformed cross-reference entry it recovers from, and exported PDFs have thousands; the warnings are harmless and they bury the actual ingest progress, so the library's logger is raised to ERROR.

```
52
53
54  @dataclass
55  class Document:
56      doc_id: str
57      source: str
58      title: str
59      text: str
60      metadata: dict = field(default_factory=dict)
61      # (char_start, char_end, page_label); empty for single-page formats.
62      page_spans: list[tuple[int, int, str]] = field(default_factory=list)
```

The `Document` dataclass: the single shape every loader returns, so the rest of the system never learns which format a document came from. **60** carries format-specific extras - page counts, which pages needed Arabic repair - as a free-form dictionary rather than as fields that would be empty for most formats. **62** is the page map, deliberately empty for formats that have no pages.

```
63
64      def page_for_offset(self, offset: int) -> str | None:
65          for start, end, label in self.page_spans:
66              if start <= offset < end:
67                  return label
68          return None
```

Given a character offset, which page was it on. A plain linear scan: a 300-page document has 300 spans, and this is called once per chunk, so the obvious implementation is also the right one.

```
69
70      def pages_for_range(self, start: int, end: int) -> list[str]:
71          hits = [
72              label
73              for p_start, p_end, label in self.page_spans
74              if p_start < end and start < p_end
75          ]
76          seen: set[str] = set()
77          return [p for p in hits if not (p in seen or seen.add(p))]
78
```

The range version, used when a chunk spans a page break. **73-75** is an interval overlap test, not containment, so a chunk that starts near the bottom of page 4 and ends on page 5 reports both. **76-77** remove duplicates while keeping page order, which matters because "pages 4-5" reads correctly and "pages 5, 4" does not.

```
79
80  def doc_id_for(path: Path) -> str:
81      return hashlib.sha1(str(path.resolve()).encode("utf-8")).hexdigest()[:16]
82
```

The document id is a hash of the resolved absolute path. Deriving it rather than counting means re-ingesting a file produces the same id, which is what lets the store replace that document's chunks instead of adding a second copy.

```
 83
 84  def _normalize(text: str) -> str:
 85      """Collapse the whitespace damage typical of PDF and HTML extraction."""
 86      text = text.replace("\r\n", "\n").replace("\r", "\n")
 87      text = text.replace("\u00A0", " ").replace("\uFEFF", "")
 88      # Rejoin words hyphenated across a line break: "inter-\nnational".
 89      text = re.sub(r"(\w)-\n(\w)", r"\1\2", text)
 90      text = re.sub(r"[ \t]+", " ", text)
 91      text = re.sub(r"\n{3,}", "\n\n", text)
 92      return text.strip()
 93
```

The whitespace repairs every format needs. **86-87** normalise line endings and delete the two invisible characters that most often survive extraction: a non-breaking space and a byte-order mark. **89** rejoins a word split across a line break by hyphenation, so "inter-\nnational" is searchable as one word. **90-91** collapse runs of spaces and limit blank lines to one, which preserves paragraph boundaries - the chunker's strongest split point - without letting a dozen blank lines count as a dozen boundaries.

```
 94
 95  # ---------------------------------------------------------------------------
 96  # Per-format loaders
 97  # ---------------------------------------------------------------------------
 98
```

A banner comment. The loaders below are each self-contained: one format, one function, returning the same Document.

```
 99  def load_text(path: Path) -> Document:
100      raw = path.read_text(encoding="utf-8", errors="replace")
101      return Document(
102          doc_id=doc_id_for(path),
103          source=str(path),
104          title=path.name,
105          text=_normalize(raw),
106          metadata={"format": path.suffix.lstrip(".") or "txt"},
107      )
108
```

Plain text, the simple case. **100** uses `errors="replace"` so a file saved in another encoding yields a few replacement characters rather than aborting the ingest. **106** records the extension as the format.

```
109
110  def load_pdf(path: Path) -> Document:
111      try:
112          from pypdf import PdfReader
113      except ImportError as exc:  # pragma: no cover - depends on environment
114          raise RuntimeError("Reading PDFs requires: pip install pypdf") from exc
115
116      reader = PdfReader(str(path))
117      pages: list[tuple[int, str]] = []
118      repaired_pages: list[int] = []
119      for page_no, page in enumerate(reader.pages, start=1):
120          try:
121              page_text = page.extract_text() or ""
122          except Exception:  # a single malformed page must not kill the document
123              page_text = ""
124          # Glyph codes -> real letters first: every check below measures words.
125          page_text = _strip_print_artifacts(fold_presentation_forms(page_text))
126          repaired = _repair_arabic_spacing(page, page_text)
127          if repaired is not None:
128              page_text = repaired
129              repaired_pages.append(page_no)
130          pages.append((page_no, page_text))
```

The PDF loader begins. **111-114** import pypdf lazily and translate the import error into an instruction, because "pip install pypdf" is more useful than a traceback. **120-123** wrap each page: one malformed page in a 200-page statute must not lose the other 199. **125** is ordering that matters - glyph codes are folded into real letters before anything measures the text, because every check below counts words. **126-129** attempt the Arabic spacing repair and record which pages needed it.

```
131
132      # Decided once per document: a file comes from one producer, and a single
133      # page can hold too few sentences to judge on its own.
134      reversed_words = word_order_reversed("\n".join(text for _, text in pages))
135
```

Word order is decided once for the whole document, not per page. A PDF comes from one producer, so either all its pages are stored in display order or none are; and a single page may hold too few sentences to judge reliably. Deciding once removes that coin-flip.

```
136      parts: list[str] = []
137      spans: list[tuple[int, int, str]] = []
138      cursor = 0
139      for page_no, page_text in pages:
140          if reversed_words:
141              plain_lines = [reverse_word_order(line) for line in page_text.splitlines()]
142              layout_lines = _layout_lines(reader.pages[page_no - 1])
143              page_text = "\n".join(
144                  _order_like_layout(layout_lines, plain_lines) if layout_lines else plain_lines
145              )
146          page_text = _normalize(undouble_words(page_text))
147          if not page_text:
148              continue
149          block = page_text + "\n\n"
150          parts.append(block)
151          spans.append((cursor, cursor + len(block), str(page_no)))
152          cursor += len(block)
153
```

Assembling the document and its page map. **140-145** run only for a reversed PDF: each line is flipped back into reading order, then `_order_like_layout` is given the same page extracted a second way. **146** normalises. **149-152** are the page map being built: each page contributes a block, and the span recorded is exactly the character range that block occupies in the final string - which is why the cursor is advanced by the block length including its trailing blank line.

```
154      info = getattr(reader, "metadata", None)
155      title = (getattr(info, "title", None) or "").strip() if info else ""
156      metadata: dict = {"format": "pdf", "pages": len(reader.pages)}
157      if repaired_pages:
158          metadata["arabic_spacing_repaired_pages"] = repaired_pages
159      if reversed_words:
160          metadata["arabic_word_order_repaired"] = True
161      return Document(
162          doc_id=doc_id_for(path),
163          source=str(path),
164          title=title or path.name,
165          text="".join(parts).strip(),
166          metadata=metadata,
167          page_spans=spans,
168      )
```

Metadata and the finished document. **155** reads the PDF's embedded title defensively, since the metadata object may be absent entirely. **157-160** record that a repair happened and on which pages - that is what lets the CLI print "repaired Arabic spacing on 12 pages" instead of silently changing the text. **164** prefers the embedded title and falls back to the filename.

```
169
170
171  def _layout_lines(page) -> list[str]:
172      """A page's lines in visual top-to-bottom order, each in reading order."""
173      try:
174          layout = page.extract_text(extraction_mode="layout") or ""
175      except Exception:
176          return []
177      # Reverse *before* folding: a lam-alef ligature is one glyph code here and
178      # must stay one character while the line is reversed.
179      lines = (fold_presentation_forms(visual_to_logical(l.strip())) for l in layout.splitlines())
180      return [l for l in _strip_print_artifacts("\n".join(lines)).splitlines() if l.strip()]
181
```

Layout-mode extraction, which preserves the visual position of text on the page. **177-179** carry a subtle ordering rule: the line is reversed before glyph codes are folded into letters, because a lam-alef ligature is a single glyph code at this stage and must stay one character while the line is reversed - fold first and it becomes two letters that the reversal puts in the wrong order.

```
182
183  _WORDS = re.compile(r"[\w\u0600-\u06FF]+")
184
```

The word pattern used for matching below: ASCII word characters plus the Arabic block, so a mixed Arabic and Latin line tokenises sensibly.

```
185
186  def _order_like_layout(layout_lines: list[str], plain_lines: list[str]) -> list[str]:
187      """Take line *order* from layout extraction and line *text* from plain.
188
189      In some legal-database PDFs, plain extraction emits every article heading
190      on a page first and every article body after them, so no body can be tied
191      to its article number. Layout extraction keeps each heading directly above
192      its own text, but decodes a few broken glyphs into longer junk than plain
193      extraction does. Each layout line is therefore swapped for its plain twin
194      (matched by shared words) wherever one exists.
195
196      Heading lines stay in their layout form, whose word order the article
197      splitter relies on.
198      """
199      pool = [(set(_WORDS.findall(line)), line) for line in plain_lines if line.strip()]
```

The function that fixes the hardest defect in this corpus. The docstring states it exactly: plain extraction of some legal-database PDFs emits every article heading on a page first and every article body afterwards, so no body can be attached to its number - the document is intact but unusable. Layout extraction keeps each heading above its own text but decodes some glyphs worse. So the order is taken from layout and the text from plain. **199** builds the pool of plain lines as word sets, ready to match.

```
200      used = [False] * len(pool)
201      out: list[str] = []
202      for line in layout_lines:
203          if _LAYOUT_HEADING.match(line):
204              out.append(line)
205              continue
206          words = set(_WORDS.findall(line))
207          best, best_score = -1, 0.0
208          for i, (plain_words, _) in enumerate(pool):
209              if used[i] or not words or not plain_words:
210                  continue
211              score = len(words & plain_words) / len(words | plain_words)
212              if score > best_score:
213                  best, best_score = i, score
214          if best >= 0 and best_score >= 0.6:
215              used[best] = True
216              out.append(pool[best][1])
217          else:
218              out.append(line)
219      return out
```

The matching loop. **203-205** leave heading lines in their layout form, because the article splitter depends on that word order. **208-213** score every unused plain line by Jaccard overlap - shared words over total words - and keep the best. **214-216** accept only at 0.6 or better and mark the line used, so two layout lines cannot both claim the same plain line. **217-218** keep the layout line when nothing matches well enough, which is the safe failure: slightly worse glyphs, never wrong text.

```
220
221
222  # "- al-madda N ..." as legal-database exports print an article heading.
223  _LAYOUT_HEADING = re.compile(r"^-\s*\u0627\u0644\u0645\u0627\u062F\u0629\s+\d+")
```

The heading pattern, written with explicit escapes: a dash, the Arabic word for "article", then a number, which is how these exports print a heading.

```
224
225
226  # Browser print footers ("https://site/print/...?a=1&b=2" plus a timestamp) and
227  # "page N of M" counters. They repeat on every page, so they add nothing a
228  # reader would search for while diluting every chunk they land in.
229  _URL_QUERY_NOISE = re.compile(r"https?|://|//:|www\.|\.com")
230  _PAGE_COUNTER = re.compile(
231      r"^.{0,12}?[\d\u0660-\u0669]{1,4}\s+(?:\u0645\u0646|of)\s+[\d\u0660-\u0669]{1,4}\s*$",
232      re.IGNORECASE,
233  )
234
```

Patterns for print noise. Browser print footers carry the source URL with its query string and a timestamp; page counters read "3 of 40". Both repeat on every single page, so they contribute nothing anyone would search for while diluting every chunk they land in. **231** accepts Arabic-Indic digits as well as ASCII ones.

```
235
236  def _strip_print_artifacts(text: str) -> str:
237      kept = []
238      for line in text.splitlines():
239          s = line.strip()
240          if _URL_QUERY_NOISE.search(s) and s.count("=") + s.count("&") >= 4:
241              continue  # print-footer URL with its query string
242          if len(s) <= 25 and _PAGE_COUNTER.match(s):
243              continue
244          kept.append(line)
245      return "\n".join(kept)
```

Applying those patterns. **240** deliberately requires both a URL-ish fragment and at least four query-string characters, so a line that merely mentions a website survives and only a machine-generated footer is dropped. **242** likewise bounds the page-counter test to short lines, so a sentence containing "3 of 40" is kept.

```
246
247
248  def _repair_arabic_spacing(page, plain_text: str) -> str | None:
249      """Re-extract a page whose Arabic words came out glued together.
250
251      Some PDFs store Arabic glyphs in display order. Plain extraction then drops
252      the spaces between words; layout extraction keeps them but emits every line
253      reversed. Reverse the layout lines back into reading order, then take the
254      *letters* from plain mode and only the *word breaks* from layout, which
255      also undoes the lam-alef ligatures the line reversal flipped.
256
257      Only used when plain extraction is measurably broken and the repair is a
258      clear improvement -- a well-formed page is never touched.
259      """
260      if not spacing_looks_broken(plain_text):
261          return None
```

The spacing repair. The docstring explains the defect and the remedy: with glyphs stored in display order, plain extraction loses the spaces between words and layout extraction keeps them but reverses every line. The fix takes letters from one and word breaks from the other. **260-261** are the guard - a page that extracts cleanly is returned untouched, so the repair can never damage a well-formed document.

```
262      try:
263          layout = fold_presentation_forms(page.extract_text(extraction_mode="layout") or "")
264      except Exception:
265          return None
266      candidate = "\n".join(visual_to_logical(line.strip()) for line in layout.splitlines())
267      if average_word_length(candidate) >= average_word_length(plain_text) * 0.75:
268          return None
269      # Prefer the exact merge; if the extractions disagree beyond ligature
270      # swaps, the reversed layout text is still far better than glued words.
271      return merge_word_breaks(plain_text, candidate) or candidate
272
```

263-265 re-extract in layout mode, giving up quietly if that fails. **266** reverses each line back into reading order. **267-268** are the second guard: unless the candidate's average word length is clearly shorter - meaning words really were glued together and are now separated - the repair is abandoned. **271** prefers the exact merge, falling back to the reversed layout text, which is still far better than glued words when the two extractions disagree beyond ligature swaps.

```
273
274 _W = "{http://schemas.openxmlformats.org/wordprocessingml/2006/main}"
275 _DC = "{http://purl.org/dc/elements/1.1/}"
276 # A paragraph this short with no sentence punctuation is treated as a heading
277 # ("Article 5", "Chapter Two") and kept on the same chunk as the text under it.
278 _HEADING_MAX_CHARS = 40
279 _SENTENCE_END = (".", ":", "\u061F", "?", "!", "\u060C", "\u061B")
280
```

Constants for Word documents. **274-275** are the XML namespaces; Word tags every element with them, so every tag comparison below has to include the prefix. **278-279** define what counts as a heading: short, and not ending in sentence punctuation - including the Arabic question mark, comma and semicolon.

```
281
282 def _docx_paragraph_text(p) -> str:
283     out: list[str] = []
284     for node in p.iter():
285         tag = node.tag
286         if tag == _W + "t":
287             out.append(node.text or "")
288         elif tag == _W + "tab":
289             out.append("\t")
290         elif tag in (_W + "br", _W + "cr"):
291             out.append("\n")
292     return "".join(out).strip()
```

Reading one Word paragraph. Word splits a sentence into runs whenever formatting changes, so the text of a paragraph is scattered across many nodes. **284** walks all of them in document order and **286-291** keep the three that carry content: text, tabs, and line breaks. Everything else - styling, revision marks, bookmarks - is skipped by omission.

```
293
294
295 def _docx_blocks(element):
296     """Yield paragraph and table text in document order.
297
298     Descends through content controls and similar wrappers, but treats a table
299     as one unit so its rows are not read as unrelated paragraphs.
300     """
301     for child in element:
302         tag = child.tag
303         if tag == _W + "p":
304             text = _docx_paragraph_text(child)
305             if text:
306                 yield "p", text
307         elif tag == _W + "tbl":
308             rows = []
309             for tr in child.iter(_W + "tr"):
310                 cells = [
311                     " ".join(filter(None, (_docx_paragraph_text(p) for p in tc.iter(_W + "p"))))
312                     for tc in tr.iter(_W + "tc")
313                 ]
314                 if any(cells):
315                     rows.append(" | ".join(cells))
316             if rows:
317                 yield "tbl", "\n".join(rows)
318         elif tag != _W + "sectPr":
319             yield from _docx_blocks(child)
320
```

Walking the document body. **301-306** yield paragraphs. **307-317** treat a table as one unit, joining cells with " | " and rows with newlines, so a table is not read as a pile of unrelated fragments. **318-319** recurse into anything else, which is what makes content controls and similar wrappers transparent, while skipping `sectPr` - section properties, which hold page setup, not text.

```
321
322  def load_docx(path: Path) -> Document:
323      """Word documents, read directly from the .docx XML -- no extra dependency.
324
325      Word stores text in reading order, so none of the PDF repairs are needed.
326      Footnotes and endnotes are appended, since in legal and academic documents
327      they often carry the amendment history or the source.
328      """
329      import xml.etree.ElementTree as ET
330      import zipfile
331
332      try:
333          archive = zipfile.ZipFile(path)
334      except zipfile.BadZipFile as exc:
335          raise RuntimeError(
336              "Not a valid .docx file. If it is an old binary .doc, open it in Word "
337              "and save it as .docx."
338          ) from exc
339
```

The Word loader. The docstring records the useful asymmetry: Word stores text in reading order, so none of the PDF repairs above are needed. **329-330** import inside the function because a .docx is just a zip of XML - no third-party dependency at all. **334-338** turn the "bad zip" error into the actual explanation, which is almost always an old binary .doc renamed.

```
340      with archive:
341          body = ET.fromstring(archive.read("word/document.xml")).find(_W + "body")
342          blocks = list(_docx_blocks(body)) if body is not None else []
343
344          notes: list[str] = []
345          for part in ("word/footnotes.xml", "word/endnotes.xml"):
346              if part in archive.namelist():
347                  root = ET.fromstring(archive.read(part))
348                  for note in root:
349                      if note.get(_W + "type") in ("separator", "continuationSeparator"):
350                          continue
351                      text = " ".join(filter(None, (_docx_paragraph_text(p) for p in note.iter(_W + "p"))))
352                      if text:
353                          notes.append(text)
354
355          title = ""
356          if "docProps/core.xml" in archive.namelist():
357              node = ET.fromstring(archive.read("docProps/core.xml")).find(_DC + "title")
358              title = (node.text or "").strip() if node is not None else ""
359
```

341-342 parse the body. **345-353** append footnotes and endnotes, skipping the separator entries Word stores alongside them; in legal documents these notes often carry the amendment history, so dropping them would lose exactly the sentence a user asks about. **356-358** read the embedded title if the file has one.

```
360      # Blank line between blocks (the chunker's strongest boundary), but a
361      # heading is glued to the block after it with a single newline.
362      parts: list[str] = []
363      for kind, text in blocks:
364          is_heading = (
365              kind == "p"
366              and len(text) <= _HEADING_MAX_CHARS
367              and not text.endswith(_SENTENCE_END)
368          )
369          parts.append(text + ("\n" if is_heading else "\n\n"))
370      text = "".join(parts)
371      if notes:
372          text += "\n\nNotes:\n\n" + "\n\n".join(notes)
373
```

Rebuilding the paragraph structure as text. **364-368** classify each block, and **369** encodes the result: an ordinary paragraph is followed by a blank line - the chunker's strongest boundary - but a heading gets only a newline, which glues it to the text it introduces so they cannot be split into different chunks.

```
    return Document(
        doc_id=doc_id_for(path),
        source=str(path),
        title=title or path.name,
        text=_normalize(text),
        metadata={"format": "docx", "paragraphs": len(blocks), "notes": len(notes)},
    )

```

The finished Word document, with paragraph and note counts in its metadata.

```

def load_csv(path: Path, max_rows: int = 2000) -> Document:
    """Render tabular data as one "col: value" record per block.

    Row-per-block keeps a record intact inside a single chunk, which matters far
    more for retrieval than preserving the visual table layout.
    """
    delimiter = "\t" if path.suffix.lower() == ".tsv" else ","
    blocks: list[str] = []
    total = 0
    with path.open("r", encoding="utf-8", errors="replace", newline="") as fh:
        reader = csv.DictReader(fh, delimiter=delimiter)
        for i, row in enumerate(reader):
            if i >= max_rows:
                break
            fields = [f"{k}: {v}" for k, v in row.items() if k and v not in (None, "")]
            if fields:
                blocks.append(f"[row {i + 1}] " + " | ".join(fields))
            total = i + 1
    header = f"Tabular file {path.name} ({total} rows indexed)"
```

Tabular files. The docstring states the choice: one record per block, rendered as "column: value" pairs. Preserving the visual table layout would look right and retrieve badly, because a row's values would be far from their column names. **389** picks the delimiter from the extension. **394-396** cap the rows read. **397** drops empty cells, so a sparse row stays short. **399** labels each block with its row number.

```
    return Document(
        doc_id=doc_id_for(path),
        source=str(path),
        title=path.name,
        text=header + "\n\n" + "\n\n".join(blocks),
        metadata={
            "format": "csv",
            "rows_indexed": total,
            "truncated": total >= max_rows,
        },
    )

```

The returned document, with **410** recording whether the row cap was hit - so a truncated ingest is visible rather than silent.

```

def load_json(path: Path) -> Document:
    raw = path.read_text(encoding="utf-8", errors="replace")
    if path.suffix.lower() == ".jsonl":
        blocks = []
        for i, line in enumerate(raw.splitlines()):
            line = line.strip()
            if not line:
                continue
            try:
                rendered = json.dumps(json.loads(line), ensure_ascii=False)
            except json.JSONDecodeError:
                rendered = line
            blocks.append(f"[record {i + 1}] {rendered}")
        text = "\n\n".join(blocks)
    else:
        try:
            text = json.dumps(json.loads(raw), ensure_ascii=False, indent=2)
        except json.JSONDecodeError:
            text = raw
    return Document(
        doc_id=doc_id_for(path),
        source=str(path),
        title=path.name,
        text=_normalize(text),
        metadata={"format": path.suffix.lstrip(".")},
    )

```

JSON and JSON Lines. **417-428** handle the line-delimited case, one record per block with its number, falling back to the raw line when a record does not parse - a corrupt line is still worth indexing as text. **430-433** handle ordinary JSON by re-dumping it indented, which gives the chunker line structure to split on. **424** and **431** both pass `ensure_ascii=False`, without which every Arabic character would be stored as a backslash escape and nothing would ever match it.

```

_TAG_RE = re.compile(r"<[^>]+>")
_SCRIPT_RE = re.compile(r"<(script|style)\b.*?</\1>", re.IGNORECASE | re.DOTALL)
_TITLE_RE = re.compile(r"<title[^>]*>(.*?)</title>", re.IGNORECASE | re.DOTALL)
_HTML_ENTITIES = {
    " ": " ", "&amp;": "&", "&lt;": "<", "&gt;": ">",
    "&quot;": chr(34), "&#39;": chr(39), "&apos;": chr(39),
}

```

A deliberately small HTML reader: strip tags, drop scripts and styles, find the title, decode the handful of entities that actually appear in prose. **448** uses `chr(34)` and `chr(39)` rather than quote characters, which keeps the dictionary readable.

```

def load_html(path: Path) -> Document:
    raw = path.read_text(encoding="utf-8", errors="replace")
    title_match = _TITLE_RE.search(raw)
    body = _SCRIPT_RE.sub(" ", raw)
    body = re.sub(r"</(p|div|h[1-6]|li|tr|section)>", "\n", body, flags=re.IGNORECASE)
    body = _TAG_RE.sub(" ", body)
    for entity, char in _HTML_ENTITIES.items():
        body = body.replace(entity, char)
    return Document(
        doc_id=doc_id_for(path),
        source=str(path),
        title=(title_match.group(1).strip() if title_match else path.name),
        text=_normalize(body),
        metadata={"format": "html"},
    )

```

455 removes script and style elements with their contents, which must happen before tag stripping or the JavaScript inside them would become body text. **456** turns block-level closing tags into newlines, preserving paragraph structure. **457** removes what is left of the markup.

```

LOADERS: dict[str, Callable[[Path], Document]] = {
    ".pdf": load_pdf,
    ".docx": load_docx,
    ".csv": load_csv,
    ".tsv": load_csv,
    ".json": load_json,
    ".jsonl": load_json,
    ".html": load_html,
    ".htm": load_html,
}

```

The dispatch table: extension to loader. Anything not listed falls back to `load_text`, so an unknown but readable file is still indexed rather than refused.

```

def _best_title(doc: Document, path: Path) -> str:
    """Pick the title that citations will show.

    1. A legal instrument's own naming clause, if present.
    2. Embedded metadata, but only if it says something: export tools often
       stamp every file with their own one-word name.
    3. The filename, which the user controls.
    """
    official = official_short_title(doc.text)
    if official:
        return official
    if doc.title and doc.title != path.name and len(doc.title.split()) >= 3:
        return doc.title
    return path.stem.replace("_", " ").strip()

```

Choosing the title a citation will display, in descending order of trustworthiness. **489-491** prefer the instrument's own naming clause - the sentence inside the law that says what the law is called - which is the most authoritative name available. **492** accepts embedded metadata only when it differs from the filename and runs to at least three words, because export tools routinely stamp every PDF they produce with their own one-word product name. **494** falls back to the filename, which the user controls.

```

# Bump whenever extraction output changes, so cached text from an older loader
# is never reused.
EXTRACTOR_VERSION = 4


```

The cache version. Every change to extraction output must bump this number, because the cache key is the file's bytes - which do not change when the code does. Without the version in the key, improving the Arabic repair would leave every previously ingested document stuck with its old, broken text.

```

def _extract(path: Path, cache_dir: Path | None) -> Document:
    """Run the format's loader, reusing a cached result for identical bytes.

    Layout-mode PDF extraction dominates ingest time, and its output depends
    only on the file's bytes -- so re-ingesting after a chunking change, or
    ingesting the same file into a second index, need not repeat it.
    """
    loader = LOADERS.get(path.suffix.lower(), load_text)
    if cache_dir is None:
        return loader(path)

    digest = hashlib.sha256(path.read_bytes()).hexdigest()[:24]
    cached = cache_dir / f"{digest}-v{EXTRACTOR_VERSION}.json"
    if cached.exists():
        try:
            data = json.loads(cached.read_text(encoding="utf-8"))
            data["page_spans"] = [tuple(span) for span in data.get("page_spans", [])]
            doc = Document(**data)
            # Identity comes from where the file is now, not where it was: the
            # same bytes may have been cached under another name.
            if doc.title == Path(doc.source).name:
                doc.title = path.name
            doc.doc_id, doc.source = doc_id_for(path), str(path)
            return doc
        except (OSError, ValueError, TypeError):
            pass  # unreadable cache entry: fall through and re-extract

```

Extraction with a content-addressed cache. The docstring gives the justification: layout-mode PDF extraction dominates ingest time and depends only on the file's bytes, so re-indexing after a chunking change should not repeat it. **513-514** key the cache on a hash of the content plus the extractor version. **518** restores the page spans as tuples, since JSON has no tuple type. **520-524** re-derive identity from where the file is now: the same bytes may have been cached under a different name, and the id, source and filename-derived title must follow the current path. **526-527** treat any unreadable cache entry as a miss.

```
529      doc = loader(path)
530      try:
531          cache_dir.mkdir(parents=True, exist_ok=True)
532          tmp = cached.with_suffix(".tmp")
533          tmp.write_text(json.dumps(doc.__dict__, ensure_ascii=False), encoding="utf-8")
534          tmp.replace(cached)
535      except OSError:
536          pass  # caching is an optimisation; never fail an ingest over it
537      return doc
538
```

The cache write. **532-534** write to a temporary file and rename it into place, so a process killed mid-write leaves no half-written entry that would later parse as garbage. **535-536** swallow filesystem errors entirely: caching is an optimisation, and a read-only disk should slow an ingest down, not fail it.

```
539
540  def load_document(path: str | Path, cache_dir: str | Path | None = None) -> Document:
541      path = Path(path)
542      doc = _extract(path, Path(cache_dir) if cache_dir else None)
543      if doc.metadata.get("format") in ("pdf", "docx"):
544          doc.title = _best_title(doc, path)
545      stat = path.stat()
546      doc.metadata.setdefault("bytes", stat.st_size)
547      doc.metadata.setdefault("mtime", stat.st_mtime)
548      doc.metadata["content_hash"] = hashlib.sha256(
549          doc.text.encode("utf-8")
550      ).hexdigest()[:16]
551      return doc
552
```

The public entry point. **543-544** apply the title rules only to PDF and Word files, the formats where the filename is often the worst available name. **546-547** record size and modification time. **548-550** hash the extracted text, not the file - that hash is what later lets the store notice two different files whose content is identical, and skip the duplicate.

```
553
554  def discover(paths: Iterable[str | Path], recursive: bool = True) -> Iterator[Path]:
555      """Yield indexable files from a mix of file and directory paths."""
556      for raw in paths:
557          p = Path(raw)
558          if p.is_file():
559              yield p
560              continue
561          if not p.is_dir():
562              continue
563          walker = p.rglob("*") if recursive else p.glob("*")
564          for child in walker:
565              if not child.is_file():
566                  continue
567              if any(part in SKIP_DIR_NAMES for part in child.parts):
568                  continue
569              if child.suffix.lower() not in SUPPORTED_EXTENSIONS:
570                  continue
571              if child.name.startswith("~$"):  # Word/Excel lock file, not a document
572                  continue
573              try:
574                  if child.stat().st_size > MAX_FILE_BYTES:
575                      continue
576              except OSError:
577                  continue
578              yield child
```

Walking paths into indexable files. **558-560** accept an explicitly named file without question - if a user names it, index it. **563** chooses recursive or flat traversal. **567** rejects anything inside a skipped directory, **569** anything whose extension is unsupported, and **571** Word and Excel lock files, which are real files with real extensions and no content. **573-577** check the size limit inside a try, because a broken symlink or a file deleted during the walk raises here and should simply be skipped.

rag/arabic.py

273 lines. Every Arabic-specific rule in the system, isolated so it can be tested on strings. Two separate jobs: folding spelling variants so search matches, and undoing the damage PDFs do to Arabic text.

```
"""Arabic text handling.

Two unrelated problems, both of which make retrieval fail silently rather than
loudly:

1. **Spelling variation.** The same word is written several valid ways --
   alef with or without hamza, ta marbuta or ha, with or without diacritics,
   Arabic-Indic or Western digits -- and the definite article and attached
   prepositions are part of the word ("al-qanun", "wa-l-qanun", "li-l-madda").
   Exact-token keyword search treats every variant as a different word.
   :func:`normalize_for_search` and :func:`light_stem` fold them together. They
   are applied identically to documents and queries, so they only need to be
   *consistent*, not linguistically perfect.

2. **Visual-order PDF text.** Some PDFs store Arabic glyphs in display order.
   Plain extraction then loses the spaces between words; layout extraction
   keeps the spaces but returns each line reversed. :func:`visual_to_logical`
   undoes the reversal. The loader only reaches for it when plain extraction is
   measurably broken, so well-formed PDFs are untouched.

Code points are written as escapes throughout: several are combining marks or
invisible, and a literal copy of them in source is impossible to review.
"""
```

The module docstring is the design rationale for the whole file. Both problems it names fail silently - search simply returns nothing useful, with no error anywhere - which is why they are stated explicitly rather than left to be rediscovered. The second sentence of problem 1 is the key insight: normalisation is applied identically to documents and to queries, so it only has to be consistent, never linguistically correct. **21-22** explain why every Arabic character in this file is written as a numeric escape: several are combining marks or invisible, and a literal copy of them in source code cannot be reviewed by eye.

```

from __future__ import annotations

import re
import unicodedata

# Arabic, Arabic Supplement, and the two presentation-form blocks.
_ARABIC_CHAR = re.compile(r"[\u0600-\u06FF\u0750-\u077F\uFB50-\uFDFF\uFE70-\uFEFF]")
_ARABIC_WORD = re.compile(r"[\u0600-\u06FF]+")
_PRESENTATION_FORM = re.compile(r"[\uFB50-\uFDFF\uFE70-\uFEFF]")
```

31-33 are the character-range tests. The first spans Arabic, Arabic Supplement and both presentation-form blocks, so it detects Arabic even in raw PDF glyph codes. The third matches only the presentation forms, which is how the code tells "text a person typed" from "glyph codes a PDF stored".

```

# Harakat (U+064B-065F), superscript alef (U+0670), honorific marks
# (U+0610-061A) and Quranic annotation marks (U+06D6-06ED). Stripped *before*
# tokenizing: they are combining marks, which a \w-based tokenizer treats as
# word breaks -- splitting a vowelled word into fragments.
_DIACRITICS = re.compile(
    r"[\u0610-\u061A\u064B-\u065F\u0670\u06D6-\u06DC\u06DF-\u06E8\u06EA-\u06ED]"
)
_TATWEEL = "\u0640"  # kashida: a stretching character with no meaning
```

The diacritics pattern, with the comment that explains why removing them is not cosmetic: they are combining marks, and a word-character tokenizer treats a combining mark as a word break - so a vowelled word silently tokenises into three fragments that match nothing. **42** is the tatweel, a pure stretching character with no meaning at all.

```
43
44  _ALEF = "\u0627"
45  _FOLD = str.maketrans(
46      {
47          "\u0623": _ALEF,  # alef with hamza above
48          "\u0625": _ALEF,  # alef with hamza below
49          "\u0622": _ALEF,  # alef with madda
50          "\u0671": _ALEF,  # alef wasla
51          "\u0649": "\u064A",  # alef maqsura -> ya
52          "\u0629": "\u0647",  # ta marbuta -> ha
53          **{chr(0x0660 + d): str(d) for d in range(10)},  # Arabic-Indic digits
54          **{chr(0x06F0 + d): str(d) for d in range(10)},  # Extended (Persian) digits
55      }
56  )
```

The folding table. **47-50** collapse the four written forms of alef into one, which is the single most valuable fold in Arabic search because the hamza is routinely omitted. **51-52** fold alef maqsura to ya and ta marbuta to ha, the two other endings people spell either way. **53-54** map both sets of Arabic-Indic digits to ASCII, so "article 61" matches whichever digits the document used.

```
57
58  # Definite article, alone or fused with a preposition or conjunction, longest
59  # first: wa-bi-al, wa-ka-al, wa-li-l, wa-al, fa-al, bi-al, ka-al, li-l, al.
60  # Applied after folding, so they only need to match folded text.
61  #
62  # A bare leading waw is deliberately *not* stripped: it is as often a root
63  # letter (wizara "ministry", wathiqa "document", wazir "minister") as the
64  # conjunction "and", and stripping it would break the match between the
65  # definite and indefinite forms of exactly those words.
66  _ARTICLE_PREFIXES = (
67      "\u0648\u0628\u0627\u0644",  # wbal
68      "\u0648\u0643\u0627\u0644",  # wkal
69      "\u0648\u0644\u0644",        # wll
70      "\u0648\u0627\u0644",        # wal
71      "\u0641\u0627\u0644",        # fal
72      "\u0628\u0627\u0644",        # bal
73      "\u0643\u0627\u0644",        # kal
74      "\u0644\u0644",              # ll
75      "\u0627\u0644",              # al
76  )
```

The prefix list, and the most interesting comment in the file. Arabic fuses the definite article with prepositions and conjunctions, so one noun appears as a dozen tokens. The list is longest-first because matching is first-wins. **62-65** record a deliberate omission: a bare leading waw is not stripped, because it is as often a root letter - in ministry, document, minister - as it is the word "and", and stripping it would break exactly the words a legal corpus is full of.

```
77  _MIN_STEM = 2  # keeps two-letter legal terms such as haqq ("right") intact
78
79
```

The minimum stem length. Without it, stripping a two-letter prefix from a four-letter word could leave a fragment that matches everything; the comment names the real case it protects, the two-letter legal term for "right".

```
80  def has_arabic(text: str) -> bool:
81      return bool(_ARABIC_CHAR.search(text))
82
83
```

A cheap test used as a guard throughout: if there is no Arabic, every function below returns its input untouched, which is what makes the module safe on English or mixed text.

```
 84  def normalize_for_search(text: str) -> str:
 85      """Fold spelling variants so equivalent Arabic words compare equal.
 86
 87      Returns non-Arabic text unchanged, so it is safe on mixed-language input.
 88      """
 89      if not has_arabic(text):
 90          return text
 91      if _PRESENTATION_FORM.search(text):
 92          # PDF glyph codes -> the base letters people actually type.
 93          text = unicodedata.normalize("NFKC", text)
 94      text = _DIACRITICS.sub("", text).replace(_TATWEEL, "")
 95      return text.translate(_FOLD)
 96
```

The normaliser, applied to documents and queries alike. **89-90** return non-Arabic text unchanged. **91-93** run NFKC only when presentation forms are present, converting PDF glyph codes into the letters a person would actually type. **94** removes diacritics and tatweel - note this happens before any tokenizing elsewhere in the system, which is the fix for the fragmenting described above. **95** applies the folding table.

```
 97
 98  def light_stem(token: str) -> str:
 99      """Strip the definite article and fused prefixes from one folded token."""
100      for prefix in _ARTICLE_PREFIXES:
101          if token.startswith(prefix) and len(token) - len(prefix) >= _MIN_STEM:
102              return token[len(prefix):]
103      return token
104
```

The stemmer, applied per token after folding. **101** carries both conditions: the token starts with the prefix and enough letters would remain. Returning on the first match is what makes the longest-first ordering of the prefix list significant.

```
105
106  # Runs that keep left-to-right order even inside a visual-order Arabic line:
107  # numbers in either digit set and Latin words, with internal separators.
108  _LTR_UNIT = r"[A-Za-z0-9\u0660-\u0669\u06F0-\u06F9]+"
109  _LTR_RUN = re.compile(_LTR_UNIT + r"(?:[.,:/%\-]" + _LTR_UNIT + r")*")
110
111
```

Patterns for runs that keep left-to-right order even inside a right-to-left line: numbers in any digit set and Latin words, including internal separators so a date or a decimal stays one unit.

```
112  def visual_to_logical(line: str) -> str:
113      """Reverse a visual-order Arabic line back into reading order.
114
115      Reversing the whole line restores the Arabic but also reverses embedded
116      numbers and Latin words, which were already in the right order -- so each
117      of those runs is flipped back afterwards ("21" stays "12"). Lines with no
118      Arabic are returned untouched.
119      """
120      if not has_arabic(line):
121          return line
122      reversed_line = line[::-1]
123      return _LTR_RUN.sub(lambda m: m.group(0)[::-1], reversed_line)
124
125
```

Reversing a visual-order line. The docstring states the complication honestly: reversing the whole line fixes the Arabic but also reverses the numbers and Latin words, which were already correct. **122** reverses everything; **123** then flips each left-to-right run back, so "12" does not become "21". **120-121** leave a line with no Arabic completely alone.

```
126  # ---------------------------------------------------------------------------
127  # Presentation forms and word-order reversal (e.g. PDFs exported by Qistas)
128  # ---------------------------------------------------------------------------
129
```

A section banner: below here, the repairs for PDF-specific damage.

```
130  # Initial-form ghain. An initial form cannot end a word, so at a word end it is
131  # a mis-mapped glyph: some fonts decode the tanween-fatha-on-alef ending
132  # ("wafqan", "ilzamiyyan") to this code point. Must be fixed *before* folding,
133  # which erases the positional information that makes it detectable.
134  _GHAIN_INITIAL_AT_WORD_END = re.compile("\uFECF(?=[\\s.,:\u060C\u061B\u061F)]|$)")
135
136
```

A single mis-mapped glyph, documented because it looks arbitrary. An initial-form letter cannot appear at the end of a word, so when it does the font has mis-mapped something - here, a common Arabic adverbial ending. **132-133** state the ordering constraint: this must be fixed before folding, because folding erases the positional information that makes the error detectable at all.

```
137  def fold_presentation_forms(text: str) -> str:
138      """Replace PDF glyph codes (U+FB50-FDFF, U+FE70-FEFF) with base letters.
139
140      Only those characters are touched; running NFKC over the whole text would
141      also rewrite unrelated characters such as fractions and superscripts.
142      """
143      if not _PRESENTATION_FORM.search(text):
144          return text
145      text = _GHAIN_INITIAL_AT_WORD_END.sub(_ALEF, text)
146      return _PRESENTATION_FORM.sub(lambda m: unicodedata.normalize("NFKC", m.group()), text)
147
148
```

Folding glyph codes to letters. **139-141** explain why the two presentation-form blocks are targeted individually instead of running NFKC over everything: NFKC would also rewrite unrelated characters such as fractions and superscripts, changing text that was never broken. **143-144** skip the work entirely when there is nothing to fold.

```
149  _SENTENCE_PUNCT = ".:\u060C\u061B"  # . : Arabic comma, Arabic semicolon
150
151
```

The four characters that end a sentence in Arabic, including the Arabic comma and semicolon, which are distinct code points from their Latin lookalikes.

```
152  def word_order_reversed(text: str, min_lines: int = 5) -> bool:
153      """Detect text whose *words* are in reverse order on each line.
154
155      In normal Arabic text, sentence punctuation ends lines; in word-reversed
156      extraction it starts them (".bil-hadith ... al-mutadarrir"). Whole lines
157      are used rather than individual tokens because reversed clause numbering
158      ("1. tathbut" -> "tathbut. 1") puts a period at a token end and would
159      otherwise masquerade as normal text.
160      """
161      starts = ends = 0
162      for line in text.splitlines():
163          s = line.strip()
164          if len(s) < 3 or not has_arabic(s):
165              continue
166          starts += s[0] in _SENTENCE_PUNCT
167          ends += s[-1] in _SENTENCE_PUNCT
168      return starts >= min_lines and starts > 2 * ends
169
170
```

The word-order detector, and a neat piece of reasoning. Normal text ends lines with punctuation; word-reversed text starts them with it. **161-167** count both and **168** requires a clear majority - at least five such lines, and more than twice as many as the normal pattern - so a document is only declared reversed on real evidence. **155-159** explain why whole lines are counted rather than tokens: reversed clause numbering puts a period at a token's end, which would masquerade as normal text.

```
# One unit that must survive reversal intact: a number or Latin run with its
# internal separators (2010-04-15, 3.5, 12/b) or a run of Arabic letters.
_UNIT = re.compile(
    _LTR_UNIT + r"(?:[.,:/%\-]" + _LTR_UNIT + r")*"
    r"|[\u0600-\u06FF]+"
    r"|\s+"
    r"|."
)
_MIRRORED = str.maketrans("()[]{}<>\u00AB\u00BB", ")(][}{><\u00BB\u00AB")
# After reversal, punctuation that belonged after a word can land before the
# next one ("1 .tathbut"); no Arabic typography puts it there, so move it back.
_DETACHED_PUNCT = re.compile("(?<=\\S) ([.:\u060C\u061B])(?=[^\\s\\d.:\u060C\u061B])")


```

173-178 define what survives reversal as one unit: a number or Latin run with its separators, a run of Arabic letters, whitespace, or any single character - the last alternative guaranteeing the pattern consumes everything. **179** mirrors brackets, which point the wrong way after reversal. **182** catches punctuation stranded before the next word instead of after the previous one; no Arabic typography puts it there, so its presence is unambiguous.

```
def reverse_word_order(line: str) -> str:
    """Put a word-reversed Arabic line back into reading order.

    Words keep their letters; only their sequence is reversed. Numbers and
    dates are treated as single units, brackets are mirrored, and punctuation
    stranded on the wrong side of a space is reattached.
    """
    if not has_arabic(line):
        return line
    units = _UNIT.findall(line.strip())
    rebuilt = "".join(reversed(units)).translate(_MIRRORED)
    return _DETACHED_PUNCT.sub(r"\1 ", rebuilt)


```

Reversing word order while keeping each word's letters. **194** splits into units, **195** reverses the sequence and mirrors brackets in one pass, and **196** reattaches the stranded punctuation.

```
# A bold word some PDFs draw twice comes out fused with itself ("al-aqar" x2
# with no space). Three-letter minimum halves: genuine Arabic reduplication
# (zalzala, waswasa) repeats only two letters, so it is never touched.
_DOUBLED_WORD = re.compile(r"(?<![\u0600-\u06FF])([\u0600-\u06FF]{3,})\1(?![\u0600-\u06FF])")


```

A fused-duplicate word, caused by PDFs that draw bold text twice. **202** requires at least three repeated letters and uses look-around to require word boundaries. The comment explains the three-letter floor precisely: genuine Arabic reduplication repeats only two letters, so real words are never damaged by this rule.

```
def undouble_words(text: str) -> str:
    """Collapse Arabic words fused with an identical copy of themselves."""
    return _DOUBLED_WORD.sub(r"\1", text)


```

Applying that pattern.

```
# Jordanian (and most Arab) legislation names itself in Article 1:
# "yusamma hadha al-qanun / al-nizam / al-ta'limat ( <short title> )".
_NAMING_CLAUSE = re.compile(
    "\u064A\u0633\u0645\u0649\\s+\u0647\u0630\u0627\\s+"
    "(?:\u0627\u0644\u0642\u0627\u0646\u0648\u0646|\u0627\u0644\u0646\u0638\u0627\u0645"
    "|\u0627\u0644\u062A\u0639\u0644\u064A\u0645\u0627\u062A)"
    "\\s*\\(\\s*([^()\\n]{4,90}?)\\s*\\)"
)


```

The naming clause. Jordanian legislation names itself in its own Article 1 - "this law shall be called (...)" - and **212-217** match that sentence for all three instrument types, capturing the title in the brackets. The 4-to-90 character bound keeps a stray pair of brackets from being mistaken for a title.

```
219
220  def official_short_title(text: str, search_chars: int = 20000) -> str | None:
221      """The instrument's own short title from its naming clause, if present.
222
223      Far more reliable than PDF metadata, which export tools often fill with
224      their own name -- every file from one legal database titled identically.
225      """
226      match = _NAMING_CLAUSE.search(text[:search_chars])
227      return " ".join(match.group(1).split()) if match else None
228
229
```

Extracting that title. **226** searches only the first 20,000 characters, since a naming clause appears at the top or not at all. The docstring states why this outranks PDF metadata: export tools stamp their own name into every file they produce, so an entire legal database can arrive with one identical title.

```
230  _LAM_ALEF = frozenset({chr(0x0644), chr(0x0627)})
231
232
```

The two letters of the lam-alef ligature, as a set so order does not matter in the comparison below.

```
233  def merge_word_breaks(letters_source: str, breaks_source: str) -> str | None:
234      """Combine one extraction's letters with another's word breaks.
235
236      For visual-order PDFs the two pypdf modes are each half right: plain mode
237      has every letter in reading order but no spaces; layout mode has the spaces
238      but, after :func:`visual_to_logical`, every lam-alef ligature comes out
239      swapped -- the ligature is one glyph that decodes to two letters already in
240      reading order, so reversing the line flips it.
241
242      Returns ``None`` unless the two agree letter-for-letter apart from those
243      swaps, so a mismatch can never produce shifted, misaligned text.
244      """
245      letters = [c for c in letters_source if not c.isspace()]
```

The merge that repairs glued Arabic. The docstring states exactly why each extraction mode is half right: plain mode has every letter in reading order but no spaces, and layout mode has the spaces but, once reversed, comes out with every lam-alef ligature swapped - because that ligature is a single glyph decoding to two letters already in reading order, which the reversal then flips.

```
246      layout_letters = [c for c in breaks_source if not c.isspace()]
247      if len(letters) != len(layout_letters):
248          return None
249      for a, b in zip(letters, layout_letters):
250          if a != b and {a, b} != _LAM_ALEF:
251              return None
252
```

The safety check that makes this function trustworthy. **247-248** require the same letter count and **249-251** require every letter to match, with the lam-alef swap as the single permitted difference. Any other disagreement returns `None`, so the caller falls back rather than producing text where letters and spaces have drifted out of alignment - which would be worse than the original defect, because it would look correct.

```
253      out: list[str] = []
254      it = iter(letters)
255      for ch in breaks_source:
256          out.append(ch if ch.isspace() else next(it))
257      return "".join(out)
258
259
```

The merge itself: walk the spaced version, emit its whitespace as-is, and draw every non-space character from the plain version's letters. The result has plain mode's letters in layout mode's word positions.

```
260  def average_word_length(text: str) -> float:
261      words = _ARABIC_WORD.findall(text)
262      return sum(map(len, words)) / len(words) if words else 0.0
263
264
```

Average length of the Arabic words in a string - the measurement both the detector below and the loader's repair guard are built on.

```
265  def spacing_looks_broken(text: str, min_words: int = 5, max_avg_len: float = 9.0) -> bool:
266      """Detect extraction that has glued Arabic words together.
267
268      Arabic words average roughly 4-6 letters. A page averaging well above that
269      is almost certainly run-together text rather than unusually long words.
270      """
271      if len(_ARABIC_WORD.findall(text)) < min_words:
272          return False
273      return average_word_length(text) > max_avg_len
```

The glued-text detector. Arabic words run four to six letters, so an average well above that means words have run together rather than that the document uses unusually long ones. **271-272** refuse to judge a page with too few words, where the average would be noise.

rag/legal.py

146 lines. Finds where each article of a law begins and ends. This is the module that lets a chunk be 'Labour Law, Article 61' instead of 'Labour Law, characters 41,000-41,500', which changes both what retrieval matches and what a citation can say.

```
"""Article structure of legislation.

Laws are cited and asked about by article ("what does Article 61 of the Labour
Law say?"), so the article -- not the page, and not an arbitrary 500-token
window -- is the natural unit for retrieval and for citations. This module
finds article boundaries in extracted text.

Three heading styles are recognised, all common in Arab legislation:

* ``- <al-madda> 61 <title> <effective-date label> 2019-05-16`` -- legal-database
  exports, where the heading line also carries a topic and an effective date.
* ``<al-madda> 7`` alone on a line -- Word and most official PDFs.
* ``<al-madda> 4- <text...>`` -- heading and first sentence on one line.

A line that merely *mentions* an article ("under Article 70 of this law") must
not start a new one. The guard is numbering: articles run in increasing order,
so a candidate is accepted only if it continues the sequence (a small forward
jump is allowed for repealed articles, and a restart at 1 for an annexed
regulation). Mid-text references almost never happen to continue it.

Code points are written as escapes (see arabic.py for why).
"""
```

The docstring argues the design before any code appears. People ask about laws by article number, so the article - not the page, not a fixed-size window - is the natural unit. **8-13** list the three heading styles actually found in this corpus, which is why there are three patterns below rather than one. **15-19** state the hard problem: a line that merely mentions an article must not start one, and the defence is numbering - real articles run in increasing order, and a passing reference almost never happens to continue the sequence.

```

from __future__ import annotations

import re
from dataclasses import dataclass

_MADDA = "\u0627\u0644\u0645\u0627\u062F\u0629"  # al-madda, "article"
_BIS = "\u0645\u0643\u0631\u0631[\u0629\u0647]?"  # mukarrar(a), "bis"
_EFFECTIVE = "\u062A\u0627\u0631\u064A\u062E \u0627\u0644\u0633\u0631\u064A\u0627\u0646"  # effective date
_FROM = "\u0645\u0646"  # "min", as in "Article 70 *of* this law"

```

The Arabic fragments the patterns are built from, each named and commented: the word "article", the suffix for an inserted article (the equivalent of "61 bis"), the phrase "effective date", and the preposition "of" that **102** uses to reject references.

```
# "- al-madda 61 <topic> <effective> 2019-05-16"
_DB_HEADING = re.compile(
    rf"^-\s*{_MADDA}\s+(\d+)\s*((?:{_BIS})?)\s*(.*?)\s*(?:{_EFFECTIVE}\s+(\d{{4}}-\d{{2}}-\d{{2}}))?\s*$"
)
# "al-madda 7" / "al-madda (7)" alone, optionally "bis" and trailing punctuation
_BARE_HEADING = re.compile(
    rf"^{_MADDA}\s*\(?\s*(\d+)\s*\)?\s*((?:{_BIS})?)\s*[-\u2013:.]?\s*$"
)
# "al-madda 4- first sentence..." (dash or colon right after the number)
_INLINE_HEADING = re.compile(
    rf"^{_MADDA}\s*\(?\s*(\d+)\s*\)?\s*((?:{_BIS})?)\s*[-\u2013:]\s*(\S.*)$"
)
```

The three heading patterns, assembled from those fragments. **35-37** is the legal-database export: a leading dash, the number, an optional bis, a topic, and an optional effective date - four capture groups, so a heading yields metadata and not just a boundary. **39-41** is a heading alone on its line, with the brackets, dashes and colons different publishers use. **43-45** is a heading whose first sentence runs on after the dash, which is why it captures the remaining text as group 3.

```
# Part / chapter / section lines that give an article its context.
_DIVISION = re.compile(
    "^(?:\u0627\u0644\u0643\u062A\u0627\u0628|\u0627\u0644\u0628\u0627\u0628"
    "|\u0627\u0644\u0641\u0635\u0644|\u0627\u0644\u0641\u0631\u0639|\u0627\u0644\u0642\u0633\u0645"
    "|\u0627\u0644\u062C\u0632\u0621)\\s+\\S"
)

```

The division pattern: the six words Arabic legislation uses for book, part, chapter, branch, section and division. A division line is not an article, but it tells you where an article sits, which is what makes a citation read "Chapter 3: Termination, Article 61".

```
MAX_FORWARD_JUMP = 50  # repealed or renumbered articles leave gaps
MIN_ARTICLES = 5       # fewer than this and the text is not treated as legislation


```

Two tuned constants. **53** allows the numbering to skip ahead by up to 50, because repealed and renumbered articles leave real gaps in real statutes. **54** is the floor below which the text is not treated as legislation at all - a contract that happens to contain one article-shaped line should fall back to ordinary chunking.

```
@dataclass
class Article:
    number: int
    label: str          # e.g. "61" or "42 bis" in the document's own wording
    start: int          # offset of the heading line
    body_start: int     # offset just after the heading (and its division line)
    end: int            # offset where the next article (or the text) begins
    topic: str = ""     # the database heading's topic, if any
    division: str = ""  # enclosing part / chapter, if known
    effective: str = ""  # effective date from the heading, if any

```

The `Article` record. The three offsets are the useful part: **61** is where the heading starts, **62** is where the body starts, and **63** is where the next article begins. Keeping heading and body boundaries separate is what lets a chunk be labelled with its heading while being searched on its body. **64-66** carry the metadata a database heading supplies, each defaulting to empty because the other two heading styles do not provide it.

```
    @property
    def citation(self) -> str:
        return f"{_MADDA} {self.label}"


```

The citation string, built from the document's own label, so "42 bis" cites as the document writes it rather than as a number the code invented.

```
def _accept(number: int, bis: bool, previous: int | None) -> bool:
    if previous is None or number == 1:
        return True
    if bis and number == previous:
        return True
    return previous < number <= previous + MAX_FORWARD_JUMP


```

The guard that separates real headings from references, in six lines. **74** accepts the first article seen, and accepts a restart at 1 - which is what an annexed regulation printed after the main law looks like. **76-77** accept a bis article repeating the previous number, because that is exactly what "61 bis" is. **78** is the main rule: strictly forward, by no more than the allowed jump. A sentence citing Article 70 inside Article 12 fails this test, which is the whole point.

```
 81  def find_articles(text: str) -> list[Article]:
 82      """Locate article headings; empty if the text does not look like a law."""
 83      articles: list[Article] = []
 84      lines = text.split("\n")
 85      offsets = []
 86      pos = 0
 87      for line in lines:
 88          offsets.append(pos)
 89          pos += len(line) + 1
 90
```

84-89 build a parallel list of character offsets, one per line, so the loop can report positions in the original string while iterating over lines. **89** adds one for the newline that `split` removed - the detail that keeps every offset in this module exact rather than approximately right.

```
 91      division = ""
 92      previous: int | None = None
 93      i = 0
 94      while i < len(lines):
 95          raw = lines[i]
 96          line = raw.strip()
 97          match_db = _DB_HEADING.match(line)
 98          match_bare = None if match_db else _BARE_HEADING.match(line)
 99          match_inline = None if (match_db or match_bare) else _INLINE_HEADING.match(line)
100          match = match_db or match_bare or match_inline
101
102          if match_inline and match_inline.group(3).startswith(_FROM + " "):
103              match = None  # "Article 70 of this law ..." -- a reference, not a heading
104
```

The scan begins. **94** is a while loop rather than a for loop because a database heading consumes its following division line and must be able to skip it. **97-100** try the three patterns in order of specificity, each only if the previous did not match. **102-103** are the reference guard at the sentence level: if an inline heading's text begins with "of", the line reads "Article 70 of this law..." and is a citation, not a heading.

```
105          if match:
106              number = int(match.group(1))
107              bis = bool(match.group(2))
108              if _accept(number, bis, previous):
109                  start = offsets[i] + (len(raw) - len(raw.lstrip()))
110                  body_start = offsets[i] + len(raw) + 1
111                  topic = match_db.group(3).strip() if match_db else ""
112                  effective = (match_db.group(4) or "") if match_db else ""
113                  if match_inline:
114                      # Body begins on the heading line itself, after the dash.
115                      body_start = offsets[i] + raw.index(match_inline.group(3))
116                  # A database heading is followed by its part name on one line.
117                  if match_db and i + 1 < len(lines) and _DIVISION.match(lines[i + 1].strip()):
118                      division = lines[i + 1].strip()
119                      body_start = offsets[i + 1] + len(lines[i + 1]) + 1
120                      i += 1
121                  label = str(number) + (" " + match.group(2) if bis else "")
```

Accepting a heading. **109** finds the heading's true start by adding back the indentation the strip removed. **110** assumes the body begins on the next line. **111-112** pull topic and effective date, available only from a database heading. **113-115** correct the body start for an inline heading, where the body begins on the same line just after the dash. **117-120** handle the database layout where a part name follows the heading on its own line: it is recorded as the division, the body start moves past it, and the loop index skips it. **121** builds the label in the document's own wording.

```
122                 articles.append(
123                     Article(
124                         number=number, label=label, start=start,
125                         body_start=min(body_start, len(text)), end=len(text),
126                         topic=topic, division=division, effective=effective,
127                     )
128                 )
129                 previous = number
130                 i += 1
131                 continue
132
```

The article is recorded. **125** clamps the body start to the length of the text, guarding the case where a heading is the very last line and the body would otherwise begin past the end; `end` is set to the end of the document and corrected later. **129** advances the numbering guard, which is what makes the sequence test work at all.

```
133         if _DIVISION.match(line) and len(line) <= 80:
134             # "Part Four" -- often followed by its title on the next line.
135             title = lines[i + 1].strip() if i + 1 < len(lines) else ""
136             if title and len(title) <= 60 and not _BARE_HEADING.match(title) and not _DB_HEADING.match(titl
                    e):
137                 division = f"{line}: {title}"
138             else:
139                 division = line
140         i += 1
141
```

Division lines that are not attached to an article. **133** bounds the line length, since a long line beginning with the word "chapter" is prose. **135-137** look ahead one line for the division's title and combine them, but only if that next line is not itself an article heading - otherwise the heading would be swallowed into the division name and lost.

```
142     for current, following in zip(articles, articles[1:]):
143         current.end = following.start
144     if len(articles) < MIN_ARTICLES:
145         return []
146     return articles
```

Two finishing steps. **142-143** set each article's end to the next one's start, which is how the last article keeps its end at the end of the document. **144-145** apply the minimum: if too few articles were found, return nothing at all, so the caller falls back to ordinary window chunking rather than carving the document at a handful of accidental matches.

rag/chunking.py

356 lines. Turns a document into the units that are actually searched. Two strategies live here: one article per chunk for legislation, and overlapping token windows for everything else. Every chunk keeps exact offsets back into its document.

```
"""Token-aware, structure-respecting chunking.

Text is first broken into *atoms* along the strongest available boundary
(blank line, then line, then sentence, then word). Atoms are packed greedily
into chunks up to ``chunk_tokens``, and each new chunk re-opens with the tail of
the previous one so a fact split across a boundary is still retrievable whole.

Every chunk keeps its ``(char_start, char_end)`` offsets into the parent
document, which is what lets a citation name a page and what lets the answer be
checked against the original file.
"""
```

The docstring states the two ideas. First, text is split along the strongest available boundary - blank line, then line, then sentence, then word - so a chunk breaks where a human would break it. Second, every chunk keeps its character offsets into the parent document, which is what lets a citation name a page and what lets any answer be checked against the original file.

```

from __future__ import annotations

import hashlib
import re
from dataclasses import dataclass, field, asdict
from functools import lru_cache
from typing import Any, Iterable

from .config import Config, DEFAULT_CONFIG
from .legal import find_articles
from .loaders import Document

```

Imports. `lru_cache` is here for the token encoder, which is expensive to construct and must be built once. The three local imports are this module's whole world: settings, the article finder, and the Document shape.

```
_MADDA = "\u0627\u0644\u0645\u0627\u062F\u0629"  # al-madda, "article"

_PARA_RE = re.compile(r"\n\s*\n")
_LINE_RE = re.compile(r"\n")
# Sentence end for Latin plus Arabic full stop / question mark.
_SENT_RE = re.compile(r"(?<=[.!?\u061F\u06D4])\s+")
_WORD_RE = re.compile(r"\s+")


```

27-31 are the four boundary patterns, in descending strength. **30** treats the Arabic question mark and the Arabic full stop as sentence ends alongside the Latin ones - without them, an Arabic paragraph has no sentence boundaries at all and the splitter would fall straight through to splitting on spaces.

```
@lru_cache(maxsize=1)
def _encoder():
    """Return a token encoder, or None if tiktoken is unavailable."""
    try:
        import tiktoken

        return tiktoken.get_encoding("cl100k_base")
    except Exception:  # pragma: no cover - optional dependency
        return None


```

The token encoder, cached so it is built once per process. **41-42** return `None` rather than raising, which makes tiktoken genuinely optional - the system still runs, just with approximate sizing.

```
45  def count_tokens(text: str) -> int:
46      """Token count, with a ~4-chars-per-token fallback when tiktoken is absent.
47
48      This is only used for sizing chunks, so an approximation is acceptable; the
49      Anthropic token count of the final prompt is a separate concern.
50      """
51      enc = _encoder()
52      if enc is None:
53          return max(1, len(text) // 4)
54      return len(enc.encode(text, disallowed_special=()))
55
56
```

Counting tokens. **52-53** are the fallback: roughly four characters per token. The docstring is careful to scope the approximation - it decides chunk sizes only, and the exact token count of the final prompt is the provider's business. **54** passes `disallowed_special=()`, without which a document containing a literal token-marker string would raise instead of being counted.

```
57  @dataclass
58  class Chunk:
59      chunk_id: str
60      doc_id: str
61      source: str
62      title: str
63      ordinal: int
64      text: str
65      char_start: int
66      char_end: int
67      token_count: int
68      pages: list[str] = field(default_factory=list)
69      metadata: dict[str, Any] = field(default_factory=dict)
```

The `Chunk` record: what it says, where it came from, and how big it is. **59** is a stable id built from the document id and the ordinal, so the same document re-ingested produces the same chunk ids. **65-66** are the offsets into the parent document and **68** the pages they fall on.

```
70      # Context prepended when the chunk is embedded, keyword-indexed and shown to
71      # the model -- e.g. "<law title> -- Article 61". A passage lifted out of a
72      # 500-page law rarely names the law itself; the header puts that back.
73      header: str = ""
74
```

The header field, with the comment that justifies it: a passage lifted out of a 500-page law almost never names the law it belongs to, so a question naming the law would not match the passage answering it. The header puts the missing context back.

```
75      @property
76      def indexed_text(self) -> str:
77          return f"{self.header}\n{self.text}" if self.header else self.text
78
```

What actually gets embedded and keyword-indexed: the header joined to the text. This one property is why the law's title and article number participate in matching rather than being decoration added at display time.

```
79      @property
80      def location(self) -> str:
81          """Human-readable provenance: ``Labour Law -- Article 61 (p.22)``,
82          ``report.pdf p.3`` or ``notes.md #2``."""
83          name = self.title or self.source
84          pages = ""
85          if self.pages:
86              pages = self.pages[0] if len(self.pages) == 1 else f"{self.pages[0]}-{self.pages[-1]}"
87          article = self.metadata.get("article")
88          if article:
89              where = f"{name} \u2014 {_MADDA} {article}"
90              return f"{where} (p.{pages})" if pages else where
91          if pages:
92              return f"{name} p.{pages}"
93          return f"{name} #{self.ordinal + 1}"
94
```

Building the human-readable location a citation shows. **86** renders a multi-page chunk as a range rather than a list. **87-90** prefer the article form, which is how a legal source should be cited. **91-92** fall back to a page, and **93** to a chunk number, so every chunk can always say where it came from, whatever the format.

```
 95      def to_dict(self) -> dict[str, Any]:
 96          return asdict(self)
 97
 98      @classmethod
 99      def from_dict(cls, data: dict[str, Any]) -> "Chunk":
100          return cls(**data)
101
102
```

Conversion to and from plain dictionaries - the index on disk is JSON, so every chunk has to survive a round trip through it.

```
103  def _split_offsets(text: str, start: int, pattern: re.Pattern[str]) -> list[tuple[int, int]]:
104      """Split ``text`` on ``pattern``, returning absolute (start, end) spans."""
105      spans: list[tuple[int, int]] = []
106      cursor = 0
107      for match in pattern.finditer(text):
108          if match.start() > cursor:
109              spans.append((start + cursor, start + match.start()))
110          cursor = match.end()
111      if cursor < len(text):
112          spans.append((start + cursor, start + len(text)))
113      return spans
114
115
```

Splitting while tracking absolute positions. The subtlety is **110**: the cursor jumps to the end of the separator match, so the separator itself belongs to neither span. That is what keeps blank lines out of chunk text while leaving the offsets exact. **111-112** emit the final span, which has no separator after it.

```
116  def _atomize(text: str, max_tokens: int) -> list[tuple[int, int]]:
117      """Break text into spans that each fit within ``max_tokens`` where possible."""
118      levels = [_PARA_RE, _LINE_RE, _SENT_RE, _WORD_RE]
119
120      def recurse(start: int, end: int, level: int) -> list[tuple[int, int]]:
121          segment = text[start:end]
122          if not segment.strip():
123              return []
124          if count_tokens(segment) <= max_tokens or level >= len(levels):
125              if count_tokens(segment) <= max_tokens:
126                  return [(start, end)]
127              # Nothing left to split on (e.g. one enormous token-dense blob):
```

The recursive splitter. **118** is the ladder of boundaries. **124** stops as soon as a segment fits, which is the point: a paragraph that fits is never split into sentences, so chunks follow the document's own structure wherever they can.

```
128              # cut on a character estimate so a chunk can never overflow.
129              approx_chars = max(1, max_tokens * 3)
130              return [
131                  (s, min(s + approx_chars, end))
132                  for s in range(start, end, approx_chars)
133              ]
134          pieces: list[tuple[int, int]] = []
135          for s, e in _split_offsets(segment, start, levels[level]):
136              pieces.extend(recurse(s, e, level + 1))
137          return pieces
138
139      return recurse(0, len(text), 0)
140
```

The escape hatch. When every boundary has been tried and a segment still does not fit - a single enormous paragraph with no spaces, a base64 blob, a table rendered as one line - **129-133** cut on a character estimate. It is crude by design: without it a chunk could exceed the model's limit, and a hard failure at generation time is worse than a slightly awkward split. **135-136** are the recursion itself, descending one level at a time.

```

def chunk_document(doc: Document, config: Config = DEFAULT_CONFIG) -> list[Chunk]:
    """Split a document into offset-tracked chunks.

    Legislation is split one article per chunk (long articles are windowed
    within their own bounds); anything else gets overlapping token windows.
    """
    if not doc.text.strip():
        return []
    articles = find_articles(doc.text) if config.article_chunks else []
    if articles:
        return _article_chunks(doc, articles, config)
    header = doc.title if config.context_headers else ""
    return _window_chunks(doc, config, 0, len(doc.text), header=header)

```

The entry point, and the fork in the road. **150** looks for articles only if article chunking is enabled. **151-152** take the legal path when articles were found - and recall that the finder returns nothing unless it is confident, so this is a genuine either/or rather than a guess. **153-154** are the ordinary path, with the document title as the header.

```

def _article_chunks(doc: Document, articles: list, config: Config) -> list[Chunk]:
    """Chunks aligned to article boundaries.

    Consecutive short articles are packed together up to
    ``article_pack_tokens``; an article is never split across chunks unless it
    alone exceeds ``chunk_tokens``. Packing matters because many articles are a
    single sentence ("The King is the head of state"), and a chunk that small
    carries too little context to be matched by meaning. Each article keeps its
    own heading inside the chunk text, so attribution survives packing.
    """
```

Article-aligned chunking. The docstring explains the packing option and why it exists: many constitutional articles are a single sentence, and a chunk that small carries too little context for meaning-based matching. It also records the safeguard - each article keeps its own heading inside the text, so attribution survives even when articles are packed together. In this corpus packing measured worse and is off by default.

```
    chunks: list[Chunk] = []
    fmt = doc.metadata.get("format", "txt")

    # Preamble before Article 1 (title page, enacting formula).
    if doc.text[: articles[0].start].strip():
        chunks.extend(
            _window_chunks(
                doc, config, 0, articles[0].start,
                header=doc.title if config.context_headers else "",
            )
        )

```

171-177 rescue the preamble: everything before Article 1 - the title page, the enacting formula, the publication details - is chunked the ordinary way rather than discarded. Without this, the text that names and dates the law would never be retrievable.

```
    def emit(group: list) -> None:
        first, last = group[0], group[-1]
        start, end = first.start, last.end
        body = doc.text[start:end]
        label = first.label if len(group) == 1 else f"{first.label}-{last.label}"
        topics = list(dict.fromkeys(a.topic for a in group if a.topic))[:3]
        parts = [doc.title, f"{_MADDA} {label}", " / ".join(topics)]
        header = " \u2014 ".join(p for p in parts if p) if config.context_headers else ""
        meta = {
            "format": fmt,
            "article": label,
            "article_number": first.number,
            "articles": [a.label for a in group],
            **({"topic": " / ".join(topics)} if topics else {}),
            **({"division": last.division} if last.division else {}),
            **({"effective": first.effective} if first.effective and len(group) == 1 else {}),
        }
```

Emitting one group of articles. **183** labels a group as a range. **184** collects up to three distinct topics, preserving order. **185-186** build the header: law title, article label, topics - which is the string that makes a chunk findable by a question naming any of the three. **187-194** record the structured metadata, each conditional entry included only when it has a value, so chunks do not carry empty keys.

```
196          tokens = count_tokens(body)
197          if tokens <= config.chunk_tokens:
198              ordinal = len(chunks)
199              chunks.append(
200                  Chunk(
201                      chunk_id=f"{doc.doc_id}:{ordinal:05d}",
202                      doc_id=doc.doc_id,
203                      source=doc.source,
204                      title=doc.title,
205                      ordinal=ordinal,
206                      text=body.strip(),
207                      char_start=start,
208                      char_end=end,
209                      token_count=tokens,
210                      pages=doc.pages_for_range(start, end),
211                      metadata=meta,
212                      header=header,
213                  )
214              )
```

The ordinary case: the group fits, so it becomes one chunk. **201** builds the zero-padded id, which keeps chunk ids sorting in document order. **210** resolves which pages this character range touched - the link back from an article to a printed page.

```
215          else:
216              # One long article: overlapping windows, all carrying its header.
217              chunks.extend(
218                  _window_chunks(
219                      doc, config, start, end, header=header,
220                      ordinal_offset=len(chunks), metadata=meta,
221                  )
222              )
223
```

The exception: one article longer than a whole chunk. It is windowed within its own bounds, and every window carries the same header and metadata - so a long article becomes several chunks that all still cite the same article number.

```
224      group: list = []
225      group_tokens = 0
226      for art in articles:
227          if not doc.text[art.start : art.end].strip():
228              continue
229          tokens = count_tokens(doc.text[art.start : art.end])
230          if group and group_tokens + tokens > config.article_pack_tokens:
231              emit(group)
232              group, group_tokens = [], 0
233          group.append(art)
234          group_tokens += tokens
235      if group:
236          emit(group)
237      return chunks
238
```

The packing loop. **227-228** skip articles whose body is empty. **230-232** flush the current group when adding the next article would exceed the packing budget. Since that budget is 0 by default, the condition is true on every iteration and each article becomes its own chunk - the measured best setting, reached without a special case in the code. **235-236** emit whatever is left.

```
239
240  def _window_chunks(
241      doc: Document,
242      config: Config,
243      span_start: int,
244      span_end: int,
245      header: str = "",
246      ordinal_offset: int = 0,
247      metadata: dict[str, Any] | None = None,
248  ) -> list[Chunk]:
249      """Overlapping token windows over ``doc.text[span_start:span_end]``."""
250      text = doc.text
251      atoms = [
252          (span_start + s, span_start + e)
253          for s, e in _atomize(text[span_start:span_end], config.chunk_tokens)
254      ]
255      if not atoms:
256          return []
257
```

Window chunking. **251-254** atomise the span and shift every offset back into document coordinates, which is what allows this function to be called for a sub-range - a single long article - as easily as for a whole file.

```
258      sized = [(s, e, count_tokens(text[s:e])) for s, e in atoms]
259      chunks: list[Chunk] = []
260      buffer: list[tuple[int, int, int]] = []
261      buffer_tokens = 0
262
```

258 costs each atom in tokens once, so the packing loop below never re-tokenises. **260-261** are the accumulator: a buffer of atoms and its running token count.

```
263      def overlap_tail() -> tuple[list[tuple[int, int, int]], int]:
264          """The tail of the current buffer to re-open the next chunk with."""
265          tail: list[tuple[int, int, int]] = []
266          tokens = 0
267          for atom in reversed(buffer):
268              if tokens + atom[2] > config.chunk_overlap_tokens:
269                  break
270              tail.insert(0, atom)
271              tokens += atom[2]
272          if tail:
273              return tail, tokens
274
```

Building the overlap. **267-271** walk backwards from the end of the chunk just emitted, taking whole atoms while they fit the overlap budget, and **270** inserts at the front so the tail stays in reading order.

```
275          # No whole atom fits the overlap budget -- which is the common case for
276          # dense prose, where one paragraph is larger than the overlap. Take a
277          # partial tail of the last atom instead, otherwise overlap silently
278          # becomes zero and facts on a chunk boundary get split in half.
279          start, end, tokens_in_atom = buffer[-1]
280          if tokens_in_atom <= 0:
281              return [], 0
282          chars_per_token = max(1.0, (end - start) / tokens_in_atom)
283          tail_chars = int(config.chunk_overlap_tokens * chars_per_token)
284          cut = max(start, end - tail_chars)
285          if cut <= start:  # would duplicate the whole atom: no progress
286              return [], 0
287          # Prefer a word boundary so the overlap does not start mid-token.
288          space = text.find(" ", cut, min(end, cut + 40))
289          if space != -1:
290              cut = space + 1
291          if cut >= end:
292              return [], 0
293          return [(cut, end, count_tokens(text[cut:end]))], count_tokens(text[cut:end])
294
```

The partial-tail path, which exists because of a real bug. For dense prose a single paragraph is larger than the whole overlap budget, so no whole atom ever fits - and the overlap silently became zero, exactly where it was needed most. **282-284** estimate how many characters the overlap is worth and cut there. **285-286** refuse a cut that would duplicate the entire atom, which would make no forward progress. **288-290** advance to the next space so the overlap does not begin in the middle of a word. The guards at **280**, **285** and **291** all return an empty tail rather than risking a loop that never advances.

```
295      def flush(final: bool = False) -> None:
296          nonlocal buffer, buffer_tokens
297          if not buffer:
298              return
299          start, end = buffer[0][0], buffer[-1][1]
300          body = text[start:end].strip()
301
```

Emitting a chunk. **299** takes its bounds from the first and last atoms in the buffer, so the offsets are the real text positions, not a running estimate.

```
302          # Too small to stand on its own: carry it into the next chunk rather
303          # than emitting a fragment or dropping the content.
304          if len(body) < config.min_chunk_chars and not final and chunks:
305              return
306
```

The small-fragment rule: a buffer below the minimum size is kept rather than emitted, so it joins the next chunk. The two extra conditions matter - not on the final flush, and not if this is the first chunk - because otherwise the tail of a document, or a short document entire, would be silently dropped.

```
307          if body:
308              ordinal = ordinal_offset + len(chunks)
309              chunks.append(
310                  Chunk(
311                      chunk_id=f"{doc.doc_id}:{ordinal:05d}",
312                      doc_id=doc.doc_id,
313                      source=doc.source,
314                      title=doc.title,
315                      ordinal=ordinal,
316                      text=body,
317                      char_start=start,
318                      char_end=end,
319                      token_count=buffer_tokens,
320                      pages=doc.pages_for_range(start, end),
321                      metadata=dict(metadata) if metadata else {"format": doc.metadata.get("format", "txt")},
322                      header=header,
323                  )
324              )
325          if final:
326              buffer, buffer_tokens = [], 0
327          else:
328              buffer, buffer_tokens = overlap_tail()
329
```

308 continues the ordinal across calls, which is how a windowed article keeps numbering in sequence with the chunks around it. **321** copies the supplied metadata rather than sharing it, so the article metadata handed in is not mutated by every chunk that receives it. **325-328** are the difference between finishing and continuing: a final flush empties the buffer, an ordinary one re-opens it with the overlap tail.

```
330      for atom in sized:
331          if buffer and buffer_tokens + atom[2] > config.chunk_tokens:
332              flush()
333          buffer.append(atom)
334          buffer_tokens += atom[2]
335
```

The main packing loop, in five lines: flush when the next atom would overflow, then append it. The order matters - flushing first means the overflowing atom starts the next chunk rather than being lost.

```
336      # Emit the remainder, unless the overlap tail is all that is left and the
337      # previous chunk already covers it.
338      if buffer:
339          tail_end = buffer[-1][1]
340          if not chunks or chunks[-1].char_end < tail_end:
341              flush(final=True)
342
343      return chunks
344
```

The remainder. **340** is the check that prevents a duplicate final chunk: if the buffer holds nothing but the overlap tail, and the previous chunk already extends that far, there is nothing new to emit.

```

def chunk_documents(
    docs: Iterable[Document], config: Config = DEFAULT_CONFIG
) -> list[Chunk]:
    out: list[Chunk] = []
    for doc in docs:
        out.extend(chunk_document(doc, config))
    return out

```

Chunking many documents is just chunking each one, concatenated.

```

def content_hash(chunk: Chunk) -> str:
    return hashlib.sha256(chunk.text.encode("utf-8")).hexdigest()[:16]
```

A hash of a chunk's text, used by the store to recognise identical content arriving from two different files.

rag/embeddings.py

170 lines. Turns text into vectors, which is what lets a question match a passage that shares no words with it. Two interchangeable backends behind one small interface, both returning normalised vectors so nothing downstream knows which was used.

```
"""Dense embedding backends.

Two backends are supported:

* **local** (default) -- a sentence-embedding transformer run through
  ``transformers`` + ``torch`` with mean pooling and L2 normalisation. No API
  key, no network at query time once the weights are cached.
* **voyage** -- Voyage AI, used when ``VOYAGE_API_KEY`` is set. Anthropic does
  not serve an embeddings endpoint, so Voyage is the hosted option that pairs
  with Claude; for Gemini, Google's own embedding models are the hosted option.

Both return L2-normalised ``float32`` vectors, so cosine similarity is a plain
dot product and the rest of the system does not care which backend produced
them.
"""
```

The docstring names both backends and the reason the hosted one is Voyage: Anthropic does not serve an embeddings endpoint, so Voyage is the option that pairs with Claude, while Google serves its own. **12-14** state the invariant that makes the two interchangeable - both return L2-normalised float32, so cosine similarity is a plain dot product and no code downstream has to normalise or even ask.

```

from __future__ import annotations

import os
from typing import Protocol, Sequence

import numpy as np

from .config import Config, DEFAULT_CONFIG

```

Imports: numpy for the vectors, and the settings object.

```

class Embedder(Protocol):
    name: str
    dim: int

    def encode_passages(self, texts: Sequence[str]) -> np.ndarray: ...
    def encode_queries(self, texts: Sequence[str]) -> np.ndarray: ...

```

The interface, written as a Protocol - structural typing, so a backend satisfies it by having these members, with no base class to inherit. The two encode methods are separate because some models encode a question differently from a passage, as **87-89** below makes use of.

```

def _l2_normalize(mat: np.ndarray) -> np.ndarray:
    norms = np.linalg.norm(mat, axis=1, keepdims=True)
    np.maximum(norms, 1e-12, out=norms)
    return (mat / norms).astype(np.float32)


```

Normalisation, with one defensive detail: **37** floors the norms at a tiny positive number, so a zero vector - which an empty or whitespace-only input produces - divides cleanly instead of yielding NaNs that would poison every later comparison.

```
41  class LocalEmbedder:
42      """Mean-pooled transformer encoder running on the local machine."""
43
44      def __init__(self, config: Config = DEFAULT_CONFIG):
45          try:
46              import torch
47              from transformers import AutoModel, AutoTokenizer
48          except ImportError as exc:  # pragma: no cover - environment dependent
49              raise RuntimeError(
50                  "Dense retrieval needs torch and transformers "
51                  "(pip install torch transformers), or run with RAG_DENSE=0 "
52                  "for keyword-only search."
53              ) from exc
54
```

The local backend. **45-53** import torch and transformers lazily and convert the import error into the two things a user can act on: the install command, and the flag that runs the system keyword-only without them.

```
55          # Model-loading chatter would otherwise interleave with CLI output.
56          try:
57              from transformers.utils import logging as hf_logging
58
59              hf_logging.set_verbosity_error()
60          except Exception:  # pragma: no cover - cosmetic only
61              pass
62
```

Silencing model-loading chatter, wrapped in its own try because this is cosmetic and must never be the reason an ingest fails.

```
63          self._torch = torch
64          self.config = config
65          self.name = config.embed_model
66
67          device = config.embed_device
68          if device == "auto":
69              device = "cuda" if torch.cuda.is_available() else "cpu"
70          self.device = device
71
72          self.tokenizer = AutoTokenizer.from_pretrained(config.embed_model)
73          self.model = AutoModel.from_pretrained(config.embed_model)
74          self.model.eval()
75          self.model.to(device)
76          self.dim = int(self.model.config.hidden_size)
77
```

Setting up the model. **67-70** resolve "auto" to the GPU if one exists. **74** puts the model in evaluation mode, which disables dropout - without it, encoding the same text twice would give slightly different vectors. **76** reads the output width from the model itself rather than hard-coding 1024, so changing models changes the dimension automatically.

```
78          # e5/gte-family models are trained with asymmetric query/passage
79          # prefixes; applying them to a model that was not is actively harmful.
80          lowered = config.embed_model.lower()
81          # BGE models are trained to be read from the [CLS] token; e5 and most
82          # sentence-transformers models from the mean of all tokens. Using the
83          # wrong one quietly degrades retrieval rather than failing.
84          self._pooling = config.embed_pooling
85          if self._pooling == "auto":
86              self._pooling = "cls" if "bge" in lowered else "mean"
87          self._use_prefix = "e5" in lowered
88          self._query_prefix = config.embed_query_prefix if self._use_prefix else ""
89          self._passage_prefix = config.embed_passage_prefix if self._use_prefix else ""
90
```

Three model-family decisions, all of which fail quietly if wrong. **81-86** choose the pooling strategy: BGE models are trained to be read from their first token, most others from the mean of all tokens, and the comment states the consequence of choosing wrong - retrieval degrades, nothing errors. **87-89** apply the asymmetric query and passage prefixes only to the e5 family, which is trained with them; adding them to a model that was not trained on them is actively harmful.

```
 91      def _encode(self, texts: Sequence[str], prefix: str) -> np.ndarray:
 92          if not texts:
 93              return np.zeros((0, self.dim), dtype=np.float32)
 94
 95          torch = self._torch
 96          # Batch texts of similar length: each batch is padded to its longest
 97          # member, and a corpus of legal articles mixes one-liners with pages.
 98          order = sorted(range(len(texts)), key=lambda i: len(texts[i] or ""))
 99          out: list[np.ndarray] = []
100          batch = max(1, self.config.embed_batch_size)
101          for i in range(0, len(order), batch):
102              window = [prefix + (texts[j] or " ") for j in order[i : i + batch]]
```

Encoding. **92-93** return a correctly-shaped empty array, so callers never special-case an empty batch. **96-98** are a genuine speed trick: every batch is padded to its longest member, and a legal corpus mixes one-line articles with multi-page ones, so sorting by length before batching means short texts are no longer padded out to the length of the longest article in the corpus. **102** substitutes a space for an empty text, since the tokenizer rejects an empty string.

```
103              encoded = self.tokenizer(
104                  window,
105                  padding=True,
106                  truncation=True,
107                  max_length=self.config.embed_max_tokens,
108                  return_tensors="pt",
109              ).to(self.device)
110              with torch.no_grad():
111                  hidden = self.model(**encoded).last_hidden_state
```

The tokenizer call. **105-107** pad to the batch, truncate at the configured limit, and **109** moves the tensors to the same device as the model. **110-111** disable gradient tracking - this is inference only, and without it torch would build a backward graph for every batch and consume memory for nothing.

```
112              if self._pooling == "cls":
113                  pooled = hidden[:, 0]
114              else:
115                  mask = encoded["attention_mask"].unsqueeze(-1).type_as(hidden)
116                  pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1e-9)
117              out.append(pooled.float().cpu().numpy())
118          stacked = np.vstack(out)
119          restored = np.empty_like(stacked)
120          restored[order] = stacked  # back to the caller's order
121          return _l2_normalize(restored)
122
```

Pooling and restoring order. **112-113** take the first token for CLS pooling. **115-116** are masked mean pooling: multiply by the attention mask so padding contributes nothing, sum, and divide by the real token count - the clamp preventing a division by zero. **119-120** undo the length sort, which is essential: the caller expects vector i to belong to text i, and a silent permutation here would mislabel every chunk in the index.

```
123      def encode_passages(self, texts: Sequence[str]) -> np.ndarray:
124          return self._encode(texts, self._passage_prefix)
125
126      def encode_queries(self, texts: Sequence[str]) -> np.ndarray:
127          return self._encode(texts, self._query_prefix)
128
129
```

The two public methods, differing only in which prefix they pass - empty for every model except e5.

```
130 class VoyageEmbedder:
131     """Hosted embeddings via Voyage AI (used when VOYAGE_API_KEY is set)."""
132
133     def __init__(self, config: Config = DEFAULT_CONFIG, model: str | None = None):
134         try:
135             import voyageai
136         except ImportError as exc:  # pragma: no cover - optional dependency
137             raise RuntimeError(
138                 "VOYAGE_API_KEY is set but the client is missing: pip install voyageai"
139             ) from exc
140         self._client = voyageai.Client()
141         self.name = model or os.environ.get("VOYAGE_MODEL", "voyage-3")
142         self.config = config
143         self.dim = 1024  # confirmed on first call
144
```

The hosted backend. **140** constructs the client, which reads the key from the environment. **143** sets a provisional dimension that **156** corrects from the first real response, so the code does not have to hard-code each model's width.

```
145     def _encode(self, texts: Sequence[str], input_type: str) -> np.ndarray:
146         if not texts:
147             return np.zeros((0, self.dim), dtype=np.float32)
148         vectors: list[list[float]] = []
149         batch = max(1, min(self.config.embed_batch_size * 4, 128))
150         for i in range(0, len(texts), batch):
151             result = self._client.embed(
152                 list(texts[i : i + batch]), model=self.name, input_type=input_type
153             )
154             vectors.extend(result.embeddings)
155         mat = np.asarray(vectors, dtype=np.float32)
156         self.dim = mat.shape[1]
157         return _l2_normalize(mat)
158
```

Hosted encoding. **149** uses larger batches than the local path, because here the cost is network round trips rather than memory. **152** passes the input type, which is how this API expresses the same query/passage asymmetry the local path handles with prefixes.

```
159     def encode_passages(self, texts: Sequence[str]) -> np.ndarray:
160         return self._encode(texts, "document")
161
162     def encode_queries(self, texts: Sequence[str]) -> np.ndarray:
163         return self._encode(texts, "query")
164
165
```

The same two methods, with this API's own labels for the two input types.

```
166 def get_embedder(config: Config = DEFAULT_CONFIG) -> Embedder:
167     """Pick a backend: Voyage when a key is present, otherwise local weights."""
168     if os.environ.get("VOYAGE_API_KEY"):
169         return VoyageEmbedder(config)
170     return LocalEmbedder(config)
```

Backend selection in three lines: if a Voyage key is present use the hosted service, otherwise run locally. The local path is the default precisely because it needs no key and no network once the weights are cached.

rag/bm25.py

130 lines. Keyword search, written out in full rather than imported. This is the half of retrieval that finds an exact article number or a rare term the embedding model never saw.

```
"""BM25 (Okapi) lexical retrieval.

Implemented directly rather than pulled in as a dependency: the whole algorithm
is about sixty lines, and owning it means the tokenizer can be shared with the
rest of the pipeline and tuned for the corpus.

BM25 earns its place next to dense retrieval because embeddings are weak exactly
where keyword search is strong -- rare proper nouns, identifiers, error codes,
version numbers, and any term the embedding model never saw in training.
"""
```

Two justifications. **3-5**: the algorithm is sixty lines, and owning it means the tokenizer can be shared with the rest of the pipeline - which matters here, because the Arabic folding has to be applied identically to documents and queries, and a third-party BM25 would tokenise its own way. **7-9**: why keyword search still earns its place next to embeddings - it is strong exactly where they are weak, on rare proper nouns, identifiers, numbers and anything outside the embedding model's vocabulary.

```

from __future__ import annotations

import math
import re
from collections import Counter
from typing import Iterable, Sequence

from .arabic import has_arabic, light_stem, normalize_for_search

```

Imports. The three functions from `arabic` are what make this index work on Arabic at all.

```
# Unicode-aware: keeps Arabic, Cyrillic, CJK and accented Latin word characters.
# Arabic text is normalized *before* this runs -- see tokenize().
_TOKEN_RE = re.compile(r"[^\W_]+", re.UNICODE)

```

The token pattern: everything that is not a non-word character, with underscore excluded. Written this way rather than as a letter range, it keeps Arabic, Cyrillic, CJK and accented Latin without naming any of them. **22** records the ordering dependency - Arabic must be normalised before this pattern is applied.

```
# Deliberately small. Aggressive stopword removal hurts phrase-like queries, and
# BM25's IDF term already discounts words that appear everywhere. Negations
# (la, lam, lan) are kept on purpose: in legal text they carry the meaning.
_ENGLISH_STOPWORDS = {
    "a", "an", "and", "are", "as", "at", "be", "but", "by", "for", "from",
    "how", "in", "is", "it", "of", "on", "or", "that", "the", "this", "to",
    "was", "were", "what", "when", "where", "which", "who", "why", "with",
}
```

The stopword list, and an argument for keeping it short. Aggressive stopword removal hurts phrase-like queries, and BM25's own IDF term already discounts words that appear everywhere, so the list only holds words with no discriminating power at all. The comment notes one deliberate inclusion-by-omission: Arabic negations are kept, because in legal text the difference between "shall" and "shall not" is the entire meaning.

```
# Arabic function words as escapes (see arabic.py for why), folded with the same
# normalizer as the text so their spelling variants are caught too.
_ARABIC_STOPWORDS = (
    "\u0641\u064A", "\u0645\u0646", "\u0639\u0644\u0649", "\u0625\u0644\u0649",  # fi, min, ala, ila
    "\u0639\u0646", "\u0647\u0630\u0627", "\u0647\u0630\u0647", "\u0630\u0644\u0643",  # an, hadha, hadhihi
        , dhalika
    "\u062A\u0644\u0643", "\u0647\u0648", "\u0647\u064A", "\u0623\u0648",  # tilka, huwa, hiya, aw
    "\u0623\u0646", "\u0625\u0646", "\u0627\u0644\u062A\u064A", "\u0627\u0644\u0630\u064A",  # an, inna, al
        lati, alladhi
    "\u0645\u0639", "\u0628\u064A\u0646", "\u0642\u062F", "\u0639\u0646\u062F",  # ma, bayna, qad, inda
)
STOPWORDS = _ENGLISH_STOPWORDS | {normalize_for_search(w) for w in _ARABIC_STOPWORDS}

```

Arabic function words, as escapes with transliterations in the comments so the list can be reviewed. **42** is the important line: the Arabic words are passed through the same normaliser as the text before being added to the set, so a stopword written with a hamza in the document still matches the folded entry here.

```

def tokenize(text: str, drop_stopwords: bool = True) -> list[str]:
    """Lower-case word tokens, with Arabic spelling variants folded together.

    Normalization must come first: Arabic diacritics are combining marks, and
    the word regex would otherwise split a vowelled word at every one of them.
    """
    text = normalize_for_search(text)
    tokens = [t.lower() for t in _TOKEN_RE.findall(text)]
    if drop_stopwords:
        tokens = [t for t in tokens if t not in STOPWORDS]
    if has_arabic(text):  # English-only text skips the per-token pass
        tokens = [light_stem(t) for t in tokens]
    return tokens

```

The tokenizer, shared by indexing and querying. **51** normalises first, and the docstring says why it cannot be done later: Arabic diacritics are combining marks, so the word pattern on **23** would treat each one as a word break and shatter a vowelled word into fragments. **55-56** run the stemmer only when the text contains Arabic, so an English corpus does not pay for a per-token function call that can never change anything.

```

class BM25Index:
    """Okapi BM25 over an in-memory corpus.

    ``k1`` controls term-frequency saturation and ``b`` the length
    normalisation; the defaults are the standard ones and are a reasonable
    starting point for mixed prose and code.
    """

```

The index class. The docstring names the two tuning constants: `k1` controls how quickly repeated terms stop adding score, and `b` how strongly long documents are penalised.

```
    def __init__(self, corpus: Sequence[str] | None = None, k1: float = 1.5, b: float = 0.75):
        self.k1 = k1
        self.b = b
        self.doc_count = 0
        self.avg_len = 0.0
        self.doc_lens: list[int] = []
        self.postings: dict[str, list[tuple[int, int]]] = {}
        self.idf: dict[str, float] = {}
        if corpus is not None:
            self.build(corpus)

```

The data structures. **74** is the inverted index - term to a list of (document, count) pairs - which is what makes search proportional to the number of documents containing the query's terms rather than to the size of the corpus. **75** caches each term's inverse document frequency, computed once at build time.

```
79      def build(self, corpus: Sequence[str]) -> "BM25Index":
80          self.doc_count = len(corpus)
81          self.doc_lens = []
82          self.postings = {}
83          if self.doc_count == 0:
84              self.avg_len = 0.0
85              self.idf = {}
86              return self
87
```

Building. **83-86** handle the empty corpus explicitly, which is what keeps an empty index from dividing by zero when the average document length is computed below.

```
88          doc_freq: Counter[str] = Counter()
89          for idx, text in enumerate(corpus):
90              tokens = tokenize(text)
91              self.doc_lens.append(len(tokens))
92              counts = Counter(tokens)
93              for term, tf in counts.items():
94                  self.postings.setdefault(term, []).append((idx, tf))
95              doc_freq.update(counts.keys())
96
```

The indexing pass. **90-92** tokenise each document and record its length - the length is needed at search time for the normalisation term. **93-94** append to each term's posting list. **95** counts distinct terms per document, which is the definition of document frequency; counting raw occurrences here would make common words look rarer than they are.

```
97          self.avg_len = sum(self.doc_lens) / self.doc_count or 1.0
98          # Robertson/Sparck-Jones IDF with the +1 smoothing that keeps very
99          # common terms at a small positive weight instead of a negative one.
100         self.idf = {
101             term: math.log(1 + (self.doc_count - df + 0.5) / (df + 0.5))
102             for term, df in doc_freq.items()
103         }
104         return self
105
```

97 computes the average length, with the `or 1.0` guarding a corpus of empty strings. **100-103** are the IDF formula, and the comment explains the +1: the classical form goes negative for a term appearing in more than half the documents, which would mean a document is penalised for containing a query term. The smoothing keeps such terms at a small positive weight instead.

```
106     def search(self, query: str, top_k: int = 10) -> list[tuple[int, float]]:
107         """Return ``(doc_index, score)`` for the best ``top_k`` matches."""
108         if self.doc_count == 0:
109             return []
110         query_terms = tokenize(query)
111         if not query_terms:
112             return []
113
```

Searching. **108-112** return early for an empty index and for a query that tokenises to nothing - which happens when a query is entirely stopwords.

```
114         scores: dict[int, float] = {}
115         for term in set(query_terms):
116             postings = self.postings.get(term)
117             if not postings:
118                 continue
119             idf = self.idf.get(term, 0.0)
120             for doc_idx, tf in postings:
121                 norm = 1 - self.b + self.b * (self.doc_lens[doc_idx] / self.avg_len)
122                 scores[doc_idx] = scores.get(doc_idx, 0.0) + idf * (
123                     tf * (self.k1 + 1) / (tf + self.k1 * norm)
124                 )
125
```

The scoring loop, which is BM25 itself. **115** iterates over the set of query terms, so a word repeated in the question does not count twice. **121** is the length normalisation - a long document needs more occurrences to score the same. **122-124** are the saturation term: the second occurrence of a word adds much less than the first, which is what stops a document that repeats one word from beating a document that actually covers the topic.

```
126          ranked = sorted(scores.items(), key=lambda kv: kv[1], reverse=True)
127          return ranked[:top_k]
128
129      def __len__(self) -> int:
130          return self.doc_count
```

Rank and return. The whole search is a dictionary scan and a sort - on this corpus it runs in a fraction of a millisecond, which is why the fused pipeline can afford to run it on every query alongside the vector search.

rag/store.py

367 lines. The index itself: what is on disk, how it is loaded and saved, and how documents are added without creating duplicates or leaving the vectors out of step with the chunks.

```
"""Persistent hybrid index: dense vectors + BM25 + chunk metadata.

Layout on disk (``.rag_index/`` by default)::

    chunks.jsonl     one chunk per line, with offsets and provenance
    embeddings.npy   float32 [n_chunks, dim], L2-normalised, row i <-> line i
    meta.json        embedder identity, chunking fingerprint, per-source state

Deliberately file-based. A dedicated vector database earns its operational cost
somewhere north of a million chunks; below that, an in-memory matrix and an
exact dot product are faster than an approximate index, exactly accurate, and
trivial to back up or diff. The retrieval interface is narrow enough that
swapping in FAISS or pgvector later means reimplementing
:meth:`VectorStore.search_dense` and nothing else.

The matrix is read into RAM rather than memory-mapped. Every query multiplies
the whole matrix, so a mapping saves no I/O -- it only adds a file handle that
Windows then refuses to let the next atomic save replace.
"""
```

The docstring is three decisions. **3-7** is the on-disk layout, and the alignment rule that everything depends on: row i of the matrix is line i of the chunk file. **9-14** defend being file-based - a vector database earns its operational cost somewhere north of a million chunks, and below that an exact dot product is faster than an approximate index, as well as exactly accurate and trivial to back up. **16-18** record a bug and its fix: memory-mapping the matrix saved nothing, because every query multiplies all of it, and left a file handle open that Windows then refused to let the next save replace.

```

from __future__ import annotations

import json
import logging
import shutil
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable, Sequence

import numpy as np

from .bm25 import BM25Index
from .chunking import Chunk, chunk_document
from .config import Config, DEFAULT_CONFIG
from .loaders import Document

log = logging.getLogger("rag.store")

CHUNKS_FILE = "chunks.jsonl"
EMBEDDINGS_FILE = "embeddings.npy"
META_FILE = "meta.json"

```

Imports and the three filenames, named once so no string literal has to be kept in sync across the load and save paths.

```

@dataclass
class IngestReport:
    added_docs: int = 0
    skipped_docs: int = 0
    updated_docs: int = 0
    added_chunks: int = 0
    removed_chunks: int = 0
    failures: list[tuple[str, str]] = None  # type: ignore[assignment]
    duplicates: list[tuple[str, str]] = None  # type: ignore[assignment]  (skipped, kept)
    pruned: list[str] = None  # type: ignore[assignment]  sources whose file is gone

```

The ingest report: what happened, in numbers the CLI can print. **51-53** are typed as lists but default to `None`, because a dataclass cannot take a mutable default - every report would share one list.

```
55      def __post_init__(self) -> None:
56          if self.pruned is None:
57              self.pruned = []
58          if self.failures is None:
59              self.failures = []
60          if self.duplicates is None:
61              self.duplicates = []
62
```

Which is why the lists are created here, once per report.

```
63      def summary(self) -> str:
64          parts = [
65              f"{self.added_docs} new",
66              f"{self.updated_docs} updated",
67              f"{self.skipped_docs} unchanged",
68              f"+{self.added_chunks} chunks",
69          ]
70          if self.removed_chunks:
71              parts.append(f"-{self.removed_chunks} stale chunks")
72          if self.pruned:
73              parts.append(f"{len(self.pruned)} deleted file(s) removed")
74          if self.duplicates:
75              parts.append(f"{len(self.duplicates)} duplicate(s) skipped")
76          if self.failures:
77              parts.append(f"{len(self.failures)} failed")
78          return ", ".join(parts)
79
```

The one-line summary. **70-77** add each clause only when it is non-zero, so an ordinary ingest prints a short line and an interesting one prints what made it interesting - rather than four zeros the reader has to scan past every time.

```
80
81  class VectorStore:
82      def __init__(self, config: Config = DEFAULT_CONFIG):
83          self.config = config
84          self.dir = Path(config.index_dir)
85          self.chunks: list[Chunk] = []
86          self.embeddings: np.ndarray | None = None
87          # source path -> {content_hash, mtime, title, chunks}
88          self.sources: dict[str, dict[str, Any]] = {}
89          self.embedder_name: str | None = None
90          self.fingerprint: dict[str, Any] = {}
91          self._bm25: BM25Index | None = None
92          self._vectors_dirty = False
93
```

The store's state. **88** is the per-source bookkeeping that makes incremental ingest possible: content hash, modification time and chunk count, keyed by file path. **91** and **92** are caches - the keyword index, built on demand, and a flag recording whether the vectors have actually changed.

```
94      # -- lifecycle ---------------------------------------------------------
95
96      @property
97      def size(self) -> int:
98          return len(self.chunks)
99
```

A section marker, and the size of the index in chunks.

```
100     @property
101     def bm25(self) -> BM25Index:
102         if self._bm25 is None:
103             # Rebuilt from chunk text rather than persisted: it costs well under
104             # a second for tens of thousands of chunks and can never go stale
105             # against the chunks actually in the index.
106             self._bm25 = BM25Index([c.indexed_text for c in self.chunks])
107         return self._bm25
108
```

The keyword index, built the first time it is asked for. The comment gives the reasoning for not persisting it: building it costs well under a second for tens of thousands of chunks, and a rebuilt index can never be stale against the chunks it was built from - whereas a persisted one could silently disagree with them. Note **106** indexes `indexed_text`, so the law title and article number are keyword-searchable too.

```
109      def _invalidate(self) -> None:
110          self._bm25 = None
111
```

Dropping the cache, called after every mutation. One line, called from four places, which is how the cache cannot be forgotten.

```
112      @classmethod
113      def load(cls, config: Config = DEFAULT_CONFIG) -> "VectorStore":
114          store = cls(config)
115          meta_path = store.dir / META_FILE
116          if not meta_path.exists():
117              return store
118
119          meta = json.loads(meta_path.read_text(encoding="utf-8"))
120          store.sources = meta.get("sources", {})
121          store.embedder_name = meta.get("embedder")
122          store.fingerprint = meta.get("fingerprint", {})
123
```

Loading. **116-117** return an empty store when there is no index yet, so every caller can load unconditionally and ask about the size afterwards. **119-122** restore the metadata, including the fingerprint that records which settings built this index.

```
124          chunks_path = store.dir / CHUNKS_FILE
125          if chunks_path.exists():
126              with chunks_path.open("r", encoding="utf-8") as fh:
127                  store.chunks = [Chunk.from_dict(json.loads(line)) for line in fh if line.strip()]
128
```

Chunks are stored one JSON object per line, so they stream in and a single corrupt line could be identified rather than destroying the file's structure.

```
129          emb_path = store.dir / EMBEDDINGS_FILE
130          if emb_path.exists():
131              # Read fully into RAM: see the module docstring on why not mmap.
132              embeddings = np.load(emb_path)
133              if embeddings.shape[0] != len(store.chunks):
134                  # A crash between the two writes; keyword search still works,
135                  # and the next ingest rebuilds the vectors.
136                  embeddings = None
137              store.embeddings = embeddings
138          return store
139
```

Loading the vectors, with a consistency check that matters. **133-136** handle a crash between the two writes: if the matrix has a different number of rows than there are chunks, every row is potentially mapped to the wrong chunk, so the vectors are dropped entirely. Keyword search keeps working and the next ingest rebuilds them - a degraded index that is correct, instead of a complete one that is silently wrong.

```
140      def save(self) -> None:
141          self.dir.mkdir(parents=True, exist_ok=True)
142          tmp_chunks = self.dir / (CHUNKS_FILE + ".tmp")
143          with tmp_chunks.open("w", encoding="utf-8") as fh:
144              for chunk in self.chunks:
145                  fh.write(json.dumps(chunk.to_dict(), ensure_ascii=False) + "\n")
146          tmp_chunks.replace(self.dir / CHUNKS_FILE)
147
```

Saving, atomically. The chunks are written to a temporary file and renamed into place, so an interrupted save leaves the previous index intact rather than a half-written one.

```
148          emb_path = self.dir / EMBEDDINGS_FILE
149          if self.embeddings is not None and len(self.chunks):
150              # Rewrite only when the vectors actually changed: re-running ingest
151              # on an unchanged corpus should not rewrite hundreds of megabytes.
152              if self._vectors_dirty or not emb_path.exists():
153                  # Must keep the .npy suffix: np.save appends one otherwise.
154                  tmp_emb = self.dir / "embeddings.tmp.npy"
155                  np.save(tmp_emb, np.asarray(self.embeddings, dtype=np.float32))
156                  tmp_emb.replace(emb_path)
157          elif emb_path.exists():
158              emb_path.unlink()  # dense vectors were dropped; do not leave a stale file
159          self._vectors_dirty = False
160
```

The vectors. **152** is the optimisation that makes re-running ingest on an unchanged corpus cheap - without it, every run would rewrite hundreds of megabytes to produce an identical file. **153-154** carry a real trap in a comment: `np.save` appends ".npy" to whatever name it is given, so a temporary file named without that suffix is written somewhere other than where the rename then looks. **157-158** delete the file when the vectors have been dropped, so a stale matrix cannot be loaded next time and matched against the wrong chunks.

```
161          meta = {
162              "embedder": self.embedder_name,
163              "fingerprint": self.fingerprint or self.config.index_fingerprint(),
164              "chunk_count": len(self.chunks),
165              "dim": int(self.embeddings.shape[1]) if self.embeddings is not None else None,
166              "sources": self.sources,
167          }
168          tmp_meta = self.dir / (META_FILE + ".tmp")
169          tmp_meta.write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
170          tmp_meta.replace(self.dir / META_FILE)
171
```

The metadata file, written last and also atomically. It records who embedded the index and with what settings, which is what `check_fingerprint` compares against later.

```
172      def clear(self) -> None:
173          self.embeddings = None
174          if self.dir.exists():
175              shutil.rmtree(self.dir)
176          self.chunks = []
177          self.sources = {}
178          self._vectors_dirty = False
179          self._invalidate()
180
```

Deleting the index, in memory and on disk. **173** releases the matrix before **175** removes the directory, which on Windows is the difference between a clean delete and a permission error.

```
181      # -- mutation ---------------------------------------------------------
182
183      def remove_source(self, source: str) -> int:
184          """Drop every chunk belonging to ``source``. Returns chunks removed."""
185          keep = [i for i, c in enumerate(self.chunks) if c.source != source]
186          removed = len(self.chunks) - len(keep)
187          if removed:
188              self.chunks = [self.chunks[i] for i in keep]
189              if self.embeddings is not None and len(self.embeddings):
190                  self.embeddings = np.asarray(self.embeddings)[keep]
191                  self._vectors_dirty = True
192              self._invalidate()
193          self.sources.pop(source, None)
194          return removed
195
```

Removing one source. **185-188** rebuild the chunk list without it. **189-190** is the line that keeps the index coherent: the same index list is applied to the matrix, so rows and chunks stay aligned. Deleting from one and not the other would leave every chunk after the deletion point holding its neighbour's vector.

```
196      def needs_reindex(self, doc: Document) -> bool:
197          known = self.sources.get(doc.source)
198          if known is None:
199              return True
200          return known.get("content_hash") != doc.metadata.get("content_hash")
201
```

Whether a document needs re-indexing: unknown source, or changed content hash. Hashing the extracted text rather than the file means a PDF re-saved with new metadata but identical content is correctly recognised as unchanged.

```
202      def add_documents(
203          self,
204          docs: Iterable[Document],
205          embedder: Any | None = None,
206          force: bool = False,
207          progress: Any | None = None,
208      ) -> IngestReport:
209          """Chunk, embed and index documents, skipping unchanged sources."""
210          report = IngestReport()
211          pending: list[Chunk] = []
212
```

The main ingest entry point, with its report and the list of chunks to embed.

```
213          # Identical content under another filename (a file downloaded twice) is
214          # skipped: indexing it would return every passage twice, spending half
215          # of top_k on repeats and citing two "sources" that are one document.
216          seen_content = {
217              meta["content_hash"]: source
218              for source, meta in self.sources.items()
219              if meta.get("content_hash")
220          }
221
```

Duplicate detection, and the comment explains what it is worth: the same document downloaded twice under two names would return every passage twice, spending half the context budget on repeats and presenting two citations that are really one source.

```
222          for doc in docs:
223              content = doc.metadata.get("content_hash")
224              twin = seen_content.get(content) if content else None
225              if twin and twin != doc.source:
226                  report.duplicates.append((doc.source, twin))
227                  if progress:
228                      progress(doc.source, f"identical to {Path(twin).name} - skipped")
229                  continue
230
```

224-229 skip a document whose content hash matches a different file already indexed, recording both names so the CLI can say which was kept.

```
231              if not force and not self.needs_reindex(doc):
232                  report.skipped_docs += 1
233                  if progress:
234                      progress(doc.source, "unchanged")
235                  continue
236
```

Skipping unchanged documents, unless the user forced a rebuild.

```
237              is_update = doc.source in self.sources
238              if is_update:
239                  report.removed_chunks += self.remove_source(doc.source)
240                  report.updated_docs += 1
241              else:
242                  report.added_docs += 1
243
```

Update versus insert. **239** removes the old chunks first, which is what makes re-ingesting an edited file replace its content instead of adding a second copy alongside it.

```
244             doc_chunks = chunk_document(doc, self.config)
245             pending.extend(doc_chunks)
246             if content:
247                 seen_content[content] = doc.source
248             self.sources[doc.source] = {
249                 "doc_id": doc.doc_id,
250                 "title": doc.title,
251                 "content_hash": doc.metadata.get("content_hash"),
252                 "mtime": doc.metadata.get("mtime"),
253                 "format": doc.metadata.get("format"),
254                 "chunks": len(doc_chunks),
255             }
```

Chunking, and recording the per-source state that the next run's skip check will read.

```
256             repairs = []
257             spacing = doc.metadata.get("arabic_spacing_repaired_pages")
258             if spacing:
259                 self.sources[doc.source]["arabic_spacing_repaired_pages"] = spacing
260                 repairs.append(f"word spacing on {len(spacing)} page(s)")
261             if doc.metadata.get("arabic_word_order_repaired"):
262                 self.sources[doc.source]["arabic_word_order_repaired"] = True
263                 repairs.append("reversed word order")
264             if progress:
265                 note = f" [repaired Arabic {', '.join(repairs)}]" if repairs else ""
266                 title = f" — {doc.title}" if doc.title and doc.title != Path(doc.source).stem else ""
267                 progress(doc.source, f"{len(doc_chunks)} chunks{title}{note}")
268
```

Reporting Arabic repairs. **256-263** record what was repaired both in the index metadata and in a human-readable list. **264-267** print it, so a user ingesting a corpus sees that pages were repaired at the moment it happens - rather than discovering silently altered text later. **266** adds the resolved title when it differs from the filename, which is how the naming-clause extraction becomes visible.

```
269         if not pending:
270             return report
271
272         new_vectors: np.ndarray | None = None
273         if embedder is not None:
274             new_vectors = embedder.encode_passages([c.indexed_text for c in pending])
275             self.embedder_name = getattr(embedder, "name", None)
276             self.fingerprint = self.config.index_fingerprint()
```

Embedding the new chunks. **274** encodes `indexed_text` - header included - so what is embedded is exactly what is keyword-indexed. **276** stamps the current settings onto the index.

```
277         elif self.embeddings is not None:
278             # Adding chunks with no embedder would leave the matrix covering
279             # only part of the corpus. Rather than carry a half-dense index --
280             # which silently hides the new content from dense search and
281             # misaligns row indices on the next delete -- drop to BM25-only and
282             # say so. Re-running ingest with dense enabled rebuilds it.
283             log.warning(
284                 "Indexing without an embedder: discarding %d existing dense "
285                 "vectors so the index stays consistent. Re-run ingest with "
286                 "dense retrieval enabled to rebuild them.",
287                 len(np.asarray(self.embeddings)),
288             )
289             self.embeddings = None
290             self.embedder_name = None
291             self._vectors_dirty = True
292
```

The half-dense case, prevented deliberately. Adding chunks with no embedder would leave a matrix covering only part of the corpus, which hides the new chunks from vector search and misaligns every row index on the next delete. Rather than carry that, **289-291** drop the vectors entirely and **283-288** say so in a warning that names the remedy. A smaller, correct index beats a larger one that is quietly broken.

```
293         self.chunks.extend(pending)
294         report.added_chunks = len(pending)
295
```

The new chunks join the list, and the count goes into the report.

```
296          if new_vectors is not None:
297              if self.embeddings is None or len(self.embeddings) == 0:
298                  self.embeddings = new_vectors
299              else:
300                  existing = np.asarray(self.embeddings)
301                  if existing.shape[1] != new_vectors.shape[1]:
302                      raise ValueError(
303                          f"Embedding dimension changed ({existing.shape[1]} -> "
304                          f"{new_vectors.shape[1]}). Re-index from scratch: rag reset"
305                      )
306                  self.embeddings = np.vstack([existing, new_vectors])
307              self._vectors_dirty = True
308
309          self._invalidate()
310          return report
311
```

Appending the new vectors. **301-305** refuse to stack matrices of different widths, which is what happens when the embedding model is changed without re-indexing - and the error names the command that fixes it. Without this check numpy would raise something far less helpful, or worse, broadcast.

```
312      # -- retrieval ---------------------------------------------------------
313
314      def search_dense(self, query_vector: np.ndarray, top_k: int) -> list[tuple[int, float]]:
315          """Exact cosine search (vectors are L2-normalised, so: a dot product)."""
316          if self.embeddings is None or len(self.chunks) == 0:
317              return []
318          matrix = np.asarray(self.embeddings)
319          scores = matrix @ query_vector.astype(np.float32).ravel()
320          k = min(top_k, scores.shape[0])
321          if k <= 0:
322              return []
323          # argpartition keeps this O(n) instead of sorting the whole corpus.
324          top = np.argpartition(-scores, k - 1)[:k]
325          top = top[np.argsort(-scores[top])]
326          return [(int(i), float(scores[i])) for i in top]
327
```

Dense search. The whole thing is **319**: one matrix-vector product, which is cosine similarity because every vector is L2-normalised. **324-325** use `argpartition` to find the top k in linear time and sort only those k, rather than sorting the entire corpus to look at twenty rows.

```
328      def search_lexical(self, query: str, top_k: int) -> list[tuple[int, float]]:
329          return self.bm25.search(query, top_k)
330
```

Keyword search, delegated to the BM25 index.

```
331      def stats(self) -> dict[str, Any]:
332          by_format: dict[str, int] = {}
333          for chunk in self.chunks:
334              fmt = chunk.metadata.get("format", "txt")
335              by_format[fmt] = by_format.get(fmt, 0) + 1
336          tokens = sum(c.token_count for c in self.chunks)
337          return {
338              "index_dir": str(self.dir),
339              "sources": len(self.sources),
340              "chunks": len(self.chunks),
341              "tokens": tokens,
342              "avg_chunk_tokens": (tokens // len(self.chunks)) if self.chunks else 0,
343              "dense": self.embeddings is not None,
344              "embedder": self.embedder_name,
345              "dim": int(np.asarray(self.embeddings).shape[1]) if self.embeddings is not None else None,
346              "by_format": by_format,
347          }
348
```

Index statistics, which is what `rag stats` prints. **343** reports whether dense search is available at all, which is the first thing to check when results look purely lexical.

```
349      def check_fingerprint(self, keys: Sequence[str] | None = None) -> str | None:
350          """Return a warning if the index was built with different settings.
351
352          Pass ``keys`` to limit the check. At query time only the embedding
353          model matters -- chunk size is fixed at ingest and cannot go stale for
354          a search -- so querying checks ``("embed_model",)`` alone.
355          """
356          if not self.fingerprint:
```

The settings-drift check. The docstring explains the `keys` parameter: at query time only the embedding model can invalidate anything, because chunk size was fixed at ingest and cannot go stale for a search - so a query checks one key and an ingest checks them all.

```
357              return None
358          current = self.config.index_fingerprint()
359          drifted = {
360              k: (v, current.get(k))
361              for k, v in self.fingerprint.items()
362              if current.get(k) != v and (keys is None or k in keys)
363          }
364          if not drifted:
365              return None
366          details = "; ".join(f"{k}: indexed with {old!r}, now {new!r}" for k, (old, new) in drifted.items())
367          return f"Index settings changed ({details}). Re-run ingest with --force."
```

359-363 collect every setting whose current value differs from the indexed one. **366-367** report them with both values and the command to fix it, because "index settings changed" without saying which, from what, to what, is not an actionable message.

rag/retriever.py

267 lines. Where a question becomes a ranked list of passages. Four stages: search both indexes, fuse their rankings, drop near-duplicates, and optionally rerank what survives.

```
"""Hybrid retrieval: dense + BM25, fused, then diversified.

The pipeline is:

1. **Recall** -- take ``candidate_k`` from each retriever independently.
2. **Fuse** -- combine with Reciprocal Rank Fusion. RRF uses *ranks*, not raw
   scores, which is what makes it safe: a cosine similarity and a BM25 score
   live on incomparable scales, and any attempt to weight them directly needs
   per-corpus calibration that goes stale.
3. **Diversify** -- Maximal Marginal Relevance drops near-duplicate chunks, a
   real problem with overlapping windows over repetitive documents.
4. **Rerank** (optional) -- a cross-encoder (local, default) or the configured
   LLM provider rescores the survivors for relevance.
"""
```

The docstring is the algorithm. The important argument is **6-9**: fusion uses ranks, not scores. A cosine similarity and a BM25 score live on scales that cannot be compared - one is bounded, the other unbounded and corpus-dependent - so any attempt to weight the raw numbers needs calibration that goes stale the moment the corpus changes. Ranks have no such problem, which is why the fusion step is three lines long and never needs tuning.

```

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Callable, Sequence

import numpy as np

from .chunking import Chunk
from .config import Config, DEFAULT_CONFIG
from .store import VectorStore


```

Imports: the chunk type, the settings, and the store that holds both indexes.

```
@dataclass
class RetrievedChunk:
    chunk: Chunk
    score: float
    dense_score: float | None = None
    lexical_score: float | None = None
    dense_rank: int | None = None
    lexical_rank: int | None = None
    rerank_score: float | None = None
    index: int = -1
    rank: int = 0  # final position after fusion, MMR and any reranking

```

The result record. Each hit carries not just its final score but where each retriever put it - **32-35** - which is what makes the system debuggable: when an answer is wrong you can see whether the passage was found by meaning, by keywords, by both, or promoted by the reranker.

```
    @property
    def text(self) -> str:
        return self.chunk.text

    @property
    def location(self) -> str:
        return self.chunk.location

```

Convenience accessors that spare callers from reaching through to the chunk.

```
48      def why(self) -> str:
49          """Short explanation of how this chunk was found -- useful when tuning."""
50          bits = []
51          if self.dense_rank is not None:
52              bits.append(f"dense #{self.dense_rank + 1} ({self.dense_score:.3f})")
53          if self.lexical_rank is not None:
54              bits.append(f"bm25 #{self.lexical_rank + 1} ({self.lexical_score:.2f})")
55          if self.rerank_score is not None:
56              bits.append(f"rerank {self.rerank_score:.2f}")
57          return ", ".join(bits) or "n/a"
58
```

The explanation string behind `--why`: "dense #3 (0.812), bm25 #1 (14.2), rerank 9.5". Reading a few of these answers questions about retrieval quality that no amount of staring at the final ranking will.

```
59      def to_dict(self) -> dict[str, Any]:
60          return {
61              "chunk_id": self.chunk.chunk_id,
62              "source": self.chunk.source,
63              "location": self.location,
64              "rank": self.rank,
65              "score": round(self.score, 6),
66              "dense_score": self.dense_score,
67              "lexical_score": self.lexical_score,
68              "rerank_score": self.rerank_score,
69              "text": self.chunk.text,
70          }
71
```

Serialisation for JSON output, so the retrieval step can be scripted and evaluated independently of the answer step - which is exactly how the evaluation harness uses it.

```
72
73  ChunkFilter = Callable[[Chunk], bool]
74
75
76  def make_filter(source: str | None = None, fmt: str | None = None) -> ChunkFilter | None:
77      """Build a chunk predicate from the common CLI-level filters."""
78      if not source and not fmt:
79          return None
80
81      def predicate(chunk: Chunk) -> bool:
82          if source and source.lower() not in chunk.source.lower():
83              return False
84          if fmt and chunk.metadata.get("format") != fmt:
85              return False
86          return True
87
88      return predicate
89
```

Filters by source path and format. **82** does a case-insensitive substring match, so `--source labour` finds the Labour Law without the full path. **78-79** return `None` rather than a predicate that always passes, which lets the caller skip the filtering work entirely.

```
90
91  def reciprocal_rank_fusion(
92      ranked_lists: Sequence[Sequence[tuple[int, float]]],
93      k: int = 60,
94      weights: Sequence[float] | None = None,
95  ) -> dict[int, float]:
96      """RRF: score(d) = sum over lists of weight / (k + rank(d)).
97
98      ``k`` damps the contribution of top ranks so that a single retriever cannot
99      dominate the fusion on the strength of one confident hit. ``weights`` lets a
100     corpus that measurably favours one retriever say so -- equal weights are the
101     right default, not the right answer for every corpus.
102     """
```

Reciprocal Rank Fusion. The docstring explains the constant: **k** damps the contribution of top ranks, so one retriever's single confident hit cannot dominate a document that both retrievers rank well. The last sentence is a caution against treating the equal default weights as a finding.

```
    fused: dict[int, float] = {}
    for list_idx, ranked in enumerate(ranked_lists):
        weight = weights[list_idx] if weights and list_idx < len(weights) else 1.0
        for rank, (doc_idx, _score) in enumerate(ranked):
            fused[doc_idx] = fused.get(doc_idx, 0.0) + weight / (k + rank + 1)
    return fused

```

The whole fusion, in five lines: each document's score is the sum over lists of its weight divided by its rank plus the constant. **107** adds one because ranks here are zero-based and the formula is defined on one-based ranks.

```

def _minmax(values: np.ndarray) -> np.ndarray:
    """Scale to [0, 1] so relevance and redundancy are comparable."""
    lo, hi = float(values.min()), float(values.max())
    if hi - lo < 1e-12:
        return np.ones_like(values)
    return (values - lo) / (hi - lo)


```

Min-max scaling to [0, 1]. **114-115** handle the degenerate case where every value is the same - returning ones rather than dividing by zero, which keeps a tie from producing NaNs that would propagate through the selection below.

```
def mmr_select(
    vectors: np.ndarray,
    candidate_ids: Sequence[int],
    relevance_scores: Sequence[float],
    top_k: int,
    lambda_mult: float = 0.7,
) -> list[int]:
    """Maximal Marginal Relevance over candidate row indices.

    Relevance comes from the *fused* ranking rather than raw cosine, so the
    first pick is always the top hybrid hit and MMR only breaks up redundancy
    below it. Both terms are normalised to [0, 1], which is what makes
    ``lambda_mult`` mean what it looks like it means.
    """
```

Maximal Marginal Relevance. Two design notes in the docstring, both about correctness rather than performance. Relevance comes from the fused ranking, not raw cosine, so the first pick is always the top hybrid hit and diversification only operates below it. And both terms are normalised, which is what makes the lambda parameter mean what its name suggests instead of being an arbitrary dial.

```
    if len(candidate_ids) <= top_k:
        return list(candidate_ids)

    cand = np.asarray(candidate_ids)
    mat = np.asarray(vectors)[cand]
    relevance = _minmax(np.asarray(relevance_scores, dtype=np.float32))

```

133-134 short-circuit when there is nothing to choose between - with fewer candidates than the target, every one is selected. **137** gathers just the candidates' vectors, so the similarity work below is over a few dozen rows rather than the whole corpus.

```
    selected: list[int] = []
    remaining = list(range(len(cand)))
    while remaining and len(selected) < top_k:
        if not selected:
            best = int(np.argmax(relevance[remaining]))
            selected.append(remaining.pop(best))
            continue
```

The first pick is pure relevance: with nothing selected yet there is no redundancy to measure, so the top fused hit is always taken first.

```
147          chosen = mat[selected]
148          # Redundancy = similarity to the most similar already-selected chunk.
149          redundancy = np.clip((mat[remaining] @ chosen.T).max(axis=1), 0.0, 1.0)
150          mmr = lambda_mult * relevance[remaining] - (1 - lambda_mult) * redundancy
151          best = int(np.argmax(mmr))
152          selected.append(remaining.pop(best))
153
154      return [int(cand[i]) for i in selected]
155
```

The MMR step. **149** defines redundancy as similarity to the most similar already-selected chunk - not the average, which would let a candidate that duplicates one selection but differs from the rest slip through. **150** trades relevance against that redundancy. The practical effect on this corpus is that three overlapping windows of the same article no longer occupy three of the twelve context slots.

```
156
157  class Retriever:
158      def __init__(
159          self,
160          store: VectorStore,
161          embedder: Any | None = None,
162          config: Config = DEFAULT_CONFIG,
163      ):
164          self.store = store
165          self.embedder = embedder
166          self.config = config
167
```

The retriever holds the store, the embedder and the settings. Nothing else.

```
168      def retrieve(
169          self,
170          query: str,
171          top_k: int | None = None,
172          chunk_filter: ChunkFilter | None = None,
173          rerank: bool | None = None,
174      ) -> list[RetrievedChunk]:
175          cfg = self.config
176          top_k = top_k or cfg.top_k
177          if self.store.size == 0:
178              return []
179
```

The query path begins. **177-178** return immediately for an empty index - which also avoids loading a 2 GB embedding model to search nothing, a short-circuit that took the test suite from seventeen seconds to one.

```
180          candidate_k = max(cfg.candidate_k, top_k * 4)
181
182          # A filter must be applied before truncation, or it silently shrinks the
183          # result set: widen recall, then filter, then cut to top_k.
184          widen = candidate_k * 3 if chunk_filter else candidate_k
185
```

Two widening steps. **180** searches at least four times deeper than the number of results wanted, because fusion and diversification both need candidates to work with. **182-184** widen further when a filter is active, and the comment states the bug this prevents: filtering after truncation silently shrinks the result set, so a filtered search would return three results where it should return twelve.

```
186          dense_hits: list[tuple[int, float]] = []
187          query_vector: np.ndarray | None = None
188          if self.embedder is not None and self.store.embeddings is not None:
189              query_vector = self.embedder.encode_queries([query])[0]
190              dense_hits = self.store.search_dense(query_vector, widen)
191
192          lexical_hits = self.store.search_lexical(query, widen)
193
```

Both searches run. **188** requires both an embedder and an actual matrix, so a keyword-only index works with no special-casing anywhere else. **192** always runs - BM25 costs a fraction of a millisecond, so there is no reason to make it conditional.

```
194         if chunk_filter is not None:
195             dense_hits = [h for h in dense_hits if chunk_filter(self.store.chunks[h[0]])]
196             lexical_hits = [h for h in lexical_hits if chunk_filter(self.store.chunks[h[0]])]
197 
198         dense_hits = dense_hits[:candidate_k]
199         lexical_hits = lexical_hits[:candidate_k]
200         if not dense_hits and not lexical_hits:
201             return []
202 
```

Filtering, then truncation to the real candidate depth - in that order, which is the point of the widening above. **200-201** return early when nothing survived.

```
203         dense_rank = {idx: r for r, (idx, _) in enumerate(dense_hits)}
204         dense_score = {idx: s for idx, s in dense_hits}
205         lexical_rank = {idx: r for r, (idx, _) in enumerate(lexical_hits)}
206         lexical_score = {idx: s for idx, s in lexical_hits}
207 
```

Four lookup tables recording each retriever's rank and score per chunk. These feed both the fusion and the explanation string, which is why the original scores are preserved rather than discarded once ranks are known.

```
208         lists: list[Sequence[tuple[int, float]]] = []
209         weights: list[float] = []
210         if dense_hits:
211             lists.append(dense_hits)
212             weights.append(cfg.dense_weight)
213         if lexical_hits:
214             lists.append(lexical_hits)
215             weights.append(cfg.lexical_weight)
216 
```

Assembling the lists to fuse. Each is added only if it has hits, so an empty dense result does not contribute an empty list that would shift the weights.

```
217         fused = reciprocal_rank_fusion(lists, k=cfg.rrf_k, weights=weights)
218         ordered = sorted(fused.items(), key=lambda kv: kv[1], reverse=True)
219         candidate_ids = [idx for idx, _ in ordered]
220 
```

Fuse, sort, and the ranked candidate list is ready.

```
221         if cfg.use_mmr and self.store.embeddings is not None and len(candidate_ids) > top_k:
222             pool = candidate_ids[: max(top_k * 4, top_k)]
223             selected = mmr_select(
224                 self.store.embeddings,
225                 pool,
226                 [fused[i] for i in pool],
227                 top_k,
228                 cfg.mmr_lambda,
229             )
230             # Keep anything MMR dropped as the tail, so top_k is always filled.
231             tail = [i for i in candidate_ids if i not in set(selected)]
232             candidate_ids = selected + tail
233 
```

Diversification. **221** runs it only when there are vectors to measure similarity with and more candidates than needed. **222** limits MMR to a pool rather than the whole candidate list, since diversifying the hundredth-best hit is wasted work. **230-232** are a small but important detail: whatever MMR dropped is kept as a tail, so the result list is always filled to the requested size rather than coming back short because everything looked redundant.

```
234         should_rerank = cfg.rerank if rerank is None else rerank
235         # A reranker can only promote what it is shown, so give it a deeper pool.
236         pool_size = max(top_k * 3, cfg.rerank_candidates if should_rerank else 0)
237 
```

The reranking pool. The comment states the constraint exactly: a reranker can only promote what it is shown, so the pool is deliberately deeper than the number of results - promoting the twentieth candidate to first is the entire value of this step.

```
238         results = [
239             RetrievedChunk(
240                 chunk=self.store.chunks[idx],
241                 score=fused[idx],
242                 dense_score=dense_score.get(idx),
243                 lexical_score=lexical_score.get(idx),
244                 dense_rank=dense_rank.get(idx),
245                 lexical_rank=lexical_rank.get(idx),
246                 index=idx,
247                 rank=rank,
248             )
249             for rank, idx in enumerate(candidate_ids[: pool_size])
250         ]
251
```

Building the result objects, carrying every score and rank collected along the way.

```
252         if should_rerank and results:
253             if cfg.reranker in ("llm", "claude"):  # "claude" kept as an alias
254                 from .rerank import llm_rerank
255
256                 results = llm_rerank(query, results, config=cfg)
257             else:
258                 from .crossencoder import local_rerank
259
260                 results = local_rerank(query, results, config=cfg)
261
```

Choosing a reranker. Both imports are inside the branch, so neither torch nor the provider SDK is loaded by a query that does not rerank. **253** keeps an old configuration value working as an alias, which costs one condition and avoids breaking anyone's existing settings.

```
262         results = results[:top_k]
263         if cfg.min_score > 0:
264             results = [r for r in results if r.score >= cfg.min_score]
265         for rank, hit in enumerate(results):
266             hit.rank = rank
267         return results
```

Final trimming. **263-264** apply a minimum score only if one was configured - it is off by default, because a fused RRF score has no natural threshold. **265-266** renumber the ranks so they describe the list actually returned.

rag/crossencoder.py

100 lines. The reranker that produced the largest single quality gain in this system: a model that reads the question and a passage together and scores how well one answers the other.

```
"""Local cross-encoder reranking.

Retrieval scores the question and each passage *separately* -- an embedding per
side, or keyword overlap -- which is fast enough to search thousands of chunks
but cannot weigh how well a passage actually answers the question. A
cross-encoder reads the pair *together* and scores relevance directly. It is too
slow to run over the whole corpus, so it only reorders the few dozen candidates
that fusion has already found.

Runs locally: no API key, no per-query cost, and it works offline once the
weights are downloaded. The default model is multilingual and handles Arabic.
"""
```

The docstring explains why a second model is worth its cost. Retrieval scores the question and the passage separately - one embedding each, or counted keyword overlap - which is what makes searching thousands of chunks fast, and also what makes it unable to judge whether a passage actually answers the question. A cross-encoder reads the pair together, which is far more accurate and far too slow to run over a corpus, so it reorders only the few dozen candidates fusion already found. On this corpus that moved the correct article into first place for 47% to 76% of the evaluation questions.

```

from __future__ import annotations

from functools import lru_cache
from typing import Sequence

from .config import Config, DEFAULT_CONFIG


```

Imports; everything heavy is imported inside the loader below.

```
@lru_cache(maxsize=4)
def _load(model_name: str, device: str, quantize: bool):
    import torch
    from transformers import AutoModelForSequenceClassification, AutoTokenizer

    try:
        from transformers.utils import logging as hf_logging

        hf_logging.set_verbosity_error()
    except Exception:  # pragma: no cover - cosmetic only
        pass

```

Loading the model, cached on its arguments so a repeated query reuses the loaded weights instead of paying several seconds again. **27-32** silence the loading chatter, in its own try because it is cosmetic.

```
    if device == "auto":
        device = "cuda" if torch.cuda.is_available() else "cpu"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(model_name)
    model.eval()
    if quantize and device == "cpu":
        # int8 weights for the linear layers: several times faster on CPU,
        # where a cross-encoder is otherwise the slowest step in a query.
        model = torch.quantization.quantize_dynamic(model, {torch.nn.Linear}, dtype=torch.qint8)
    model.to(device)
    return tokenizer, model, device

```

34-35 resolve the device. **38** switches to evaluation mode. **39-42** are the line that made this practical on a CPU: int8 quantisation of the linear layers, which is several times faster at a negligible cost in ranking quality. Note **39** applies it only on CPU - on a GPU the full-precision model is already fast and quantising would just lose accuracy.

```

class CrossEncoderReranker:
    def __init__(self, config: Config = DEFAULT_CONFIG):
        self.config = config
        self.name = config.reranker_model
        self.tokenizer, self.model, self.device = _load(
            self.name, config.embed_device, config.reranker_quantize
        )

```

The reranker object, which is only a thin wrapper around the cached load - so constructing one per query is free after the first.

```
    def score(self, query: str, passages: Sequence[str]) -> list[float]:
        """Relevance of each passage to ``query``; higher is better."""
        if not passages:
            return []
        import torch

        # Batch passages of similar length together: a batch is padded to its
        # longest member, and legal articles range from one line to a page.
        order = sorted(range(len(passages)), key=lambda i: len(passages[i]))
        scores = [0.0] * len(passages)
        batch = max(1, self.config.reranker_batch_size)
```

Scoring. **61-63** sort by length before batching, for the same reason the embedder does: every batch is padded to its longest member, and these articles range from one line to a full page, so unsorted batches spend most of their compute on padding.

```
        for start in range(0, len(order), batch):
            ids = order[start : start + batch]
            window = [passages[i] for i in ids]
            encoded = self.tokenizer(
                [query] * len(window),
                window,
                padding=True,
                truncation="only_second",  # never truncate the question
                max_length=self.config.reranker_max_tokens,
                return_tensors="pt",
            ).to(self.device)
```

The pair encoding. **70-71** pass the question and the passage as two separate sequences, which is what a cross-encoder expects and what lets it attend across them. **73** is a small but decisive detail: truncation is restricted to the second sequence, so a long passage is cut and the question never is. Truncating the question would be scoring passages against half a question.

```
            with torch.no_grad():
                logits = self.model(**encoded).logits
            # One relevance logit per pair; some models emit [irrelevant, relevant].
            column = logits[:, -1] if logits.shape[-1] > 1 else logits.view(-1)
            for i, value in zip(ids, column.float().cpu().tolist()):
                scores[i] = value
        return scores
```

77-78 run inference without gradient tracking. **79-80** handle both model shapes: some emit a single relevance logit, others a pair of class logits, in which case the last column is the relevant one. **81-82** write each score back to its original position, undoing the length sort.

```


def local_rerank(query: str, candidates: Sequence, config: Config = DEFAULT_CONFIG) -> list:
    """Reorder retrieved chunks by cross-encoder relevance.

    Only the first ``rerank_candidates`` are scored; any beyond that keep their
    fused order after the reranked ones.
    """
```

The public function. The docstring records the policy: only the configured number of candidates is scored, and anything beyond that keeps its fused order after the reranked ones - so the pool is improved and the tail is never lost.

```
 92      if len(candidates) <= 1:
 93          return list(candidates)
 94      pool = list(candidates[: config.rerank_candidates])
 95      reranker = CrossEncoderReranker(config)
 96      scores = reranker.score(query, [hit.chunk.indexed_text for hit in pool])
 97      for hit, score in zip(pool, scores):
 98          hit.rerank_score = score
 99      pool.sort(key=lambda hit: hit.rerank_score, reverse=True)
100      return pool + list(candidates[len(pool):])
```

96 scores `indexed_text`, header included, so the reranker sees "Labour Law - Article 61" above the passage and can use it. **99** sorts by the new score, and **100** appends the unscored tail, which is what keeps the result count unchanged regardless of how deep the reranker reached.

rag/llm.py

396 lines. The only part of the system that talks to a hosted model, and the only part that knows which provider is in use. Two backends - Claude and Gemini - behind one interface, so the provider can be swapped without re-indexing anything.

```
"""Answer-generation backends: Claude or Gemini.

Only the *generation* step talks to a hosted model. Loading, chunking,
embedding, keyword search and reranking all run locally, so switching provider
changes nothing about the index and needs no re-ingest.

Both backends expose the same two calls -- :meth:`complete` and
:meth:`stream` -- over a provider-neutral request: a list of system parts and a
list of ``{"role": ..., "content": ...}`` messages. Everything provider-shaped
(Anthropic cache breakpoints, Gemini's ``model`` role and thinking levels) stays
inside the backend, so the prompt, citation parsing and the rest of the
pipeline never branch on which provider is in use.

Provider is chosen by ``RAG_LLM_PROVIDER``, or automatically from whichever
credentials are present.
"""
```

The docstring states the boundary this module exists to draw. **3-5**: only generation is hosted - loading, chunking, embedding, keyword search and reranking all run locally, so switching provider changes nothing about the index. **7-12** define the neutral request shape and name what is kept inside each backend: cache breakpoints, role names, thinking levels. Because those stay here, the prompt builder, the citation parser and the CLI never branch on which provider is configured.

```

from __future__ import annotations

import os
from dataclasses import dataclass, field
from typing import Any, Iterator, Protocol, Sequence

from .config import Config

```

Imports. Neither provider SDK is imported here - each is loaded inside its own backend.

```
CLAUDE_DEFAULT_MODEL = "claude-opus-5"
# Gemini 3.x; the 2.5 models are restricted to accounts that already used them.
GEMINI_DEFAULT_MODEL = "gemini-3.8-flash"

GEMINI_KEY_VARS = ("GEMINI_API_KEY", "GOOGLE_API_KEY")
ANTHROPIC_KEY_VARS = ("ANTHROPIC_API_KEY", "ANTHROPIC_AUTH_TOKEN")

```

Default models and the environment variables each provider's credentials can arrive in. **27** records something learned by trying: the 2.5 Gemini models are restricted to accounts that already used them, so a new key gets a 404 from them and the 3.x default avoids that.

```
CREDENTIAL_HINT = (
    "No credentials for answer generation found. Set GEMINI_API_KEY (free key from "
    "https://aistudio.google.com/apikey) or ANTHROPIC_API_KEY, then retry. "
    "Retrieval (`rag search`) needs no key."
)


class MissingCredentials(RuntimeError):
    pass

```

One message shown whenever credentials are missing, wherever that is discovered. It names both options, links the free one, and ends with the fact that saves a confused user: retrieval needs no key at all.

```
43
44  @dataclass
45  class Completion:
46      text: str
47      model: str
48      usage: dict[str, Any] = field(default_factory=dict)
49      stop_reason: str | None = None
50      refused: bool = False
51      refusal_detail: str = ""
52
```

The provider-neutral result. **50-51** carry refusal as data rather than as an exception, because a refusal is a legitimate outcome that the caller should display, not an error to be caught.

```
53
54  class Backend(Protocol):
55      name: str
56      model: str
57
58      def complete(
59          self,
60          system_parts: Sequence[str],
61          messages: Sequence[dict[str, Any]],
62          max_tokens: int | None = ...,
63          effort: str | None = ...,
64      ) -> Completion: ...
65
66      def stream(
67          self,
68          system_parts: Sequence[str],
69          messages: Sequence[dict[str, Any]],
70          max_tokens: int | None = ...,
71          effort: str | None = ...,
72      ) -> Iterator[str]: ...
73
74      def describe_error(self, exc: Exception) -> str: ...
75
```

The interface both backends implement: complete, stream, and a method that turns one of the SDK's exceptions into a sentence a user can act on. Written as a Protocol, so neither backend inherits anything.

```
76
77  # ---------------------------------------------------------------------------
78  # Claude
79  # ---------------------------------------------------------------------------
80
81  class ClaudeBackend:
82      name = "claude"
83
```

The Claude backend begins.

```
84      def __init__(self, config: Config):
85          self.config = config
86          self.model = config.model or CLAUDE_DEFAULT_MODEL
87          try:
88              import anthropic
89          except ImportError as exc:  # pragma: no cover - environment dependent
90              raise MissingCredentials("Claude answers need: pip install anthropic") from exc
91          self._anthropic = anthropic
92          # The SDK resolves ANTHROPIC_API_KEY, then ANTHROPIC_AUTH_TOKEN, then an
93          # `ant auth login` profile, so an unset API key is not proof of no
94          # credentials. Let it decide, and translate its failure.
95          try:
96              self._client = anthropic.Anthropic()
97          except Exception as exc:
98              raise MissingCredentials(CREDENTIAL_HINT) from exc
99
```

Construction. **92-94** record a subtlety worth knowing: the Anthropic SDK resolves a key from the API key variable, then an auth token, then a logged-in profile - so an unset environment variable is not proof that there are no credentials. The code lets the SDK decide and translates its failure, rather than guessing from the environment.

```
100      def _params(self, system_parts, messages, max_tokens, effort) -> dict[str, Any]:
101          system: list[dict[str, Any]] = [{"type": "text", "text": part} for part in system_parts if part]
102          if system and self.config.cache_context:
103              # Cache breakpoint after the retrieved context, so follow-up turns
104              # over the same passages reuse it.
105              system[-1]["cache_control"] = {"type": "ephemeral"}
106          return {
107              "model": self.model,
108              "max_tokens": max_tokens or self.config.max_tokens,
109              "thinking": {"type": "adaptive"},
110              "output_config": {"effort": effort or self.config.effort},
111              "system": system,
112              "messages": list(messages),
113          }
114
```

Building the request. **102-105** place a cache breakpoint after the retrieved context, so a follow-up question over the same passages is billed at the cached rate. **109** uses adaptive thinking, letting the model decide how much reasoning a question needs. **110** maps the configured effort onto the output configuration.

```
115      def complete(self, system_parts, messages, max_tokens=None, effort=None) -> Completion:
116          params = self._params(system_parts, messages, max_tokens, effort)
117          try:
118              if self.config.stream:
119                  # Streaming keeps a long answer inside the request timeout.
120                  with self._client.messages.stream(**params) as stream:
121                      message = stream.get_final_message()
122              else:
123                  message = self._client.messages.create(**params)
124          except Exception as exc:
125              raise RuntimeError(self.describe_error(exc)) from exc
126
```

The non-streaming call, which still streams internally. **118-121** carry the reason in a comment: a long answer requested in one shot can exceed the request timeout, while a streamed one keeps the connection alive and is assembled at the end. **124-125** convert any SDK exception into a readable message.

```
127          refused = message.stop_reason == "refusal"
128          details = getattr(message, "stop_details", None)
129          return Completion(
130              text="".join(
131                  b.text for b in message.content if getattr(b, "type", None) == "text"
132              ).strip(),
133              model=message.model,
134              usage=_claude_usage(message.usage),
135              stop_reason=message.stop_reason,
136              refused=refused,
137              refusal_detail=str(getattr(details, "category", "") or "") if refused else "",
138          )
139
```

Reading the response. **130-132** concatenate only the text blocks, since the response can also contain thinking blocks that are not part of the answer. **127** and **137** capture a refusal and its category, which the CLI shows instead of pretending an empty answer was produced.

```
140      def stream(self, system_parts, messages, max_tokens=None, effort=None) -> Iterator[str]:
141          params = self._params(system_parts, messages, max_tokens, effort)
142          try:
143              with self._client.messages.stream(**params) as stream:
144                  yield from stream.text_stream
145          except Exception as exc:
146              raise RuntimeError(self.describe_error(exc)) from exc
147
```

Streaming, which is the same request yielding text as it arrives.

```
148      def describe_error(self, exc: Exception) -> str:
149          anthropic = self._anthropic
150          if isinstance(exc, (anthropic.AuthenticationError, MissingCredentials)):
151              return CREDENTIAL_HINT
152          if "could not resolve authentication" in str(exc).lower():
153              return CREDENTIAL_HINT
154          if isinstance(exc, anthropic.PermissionDeniedError):
155              return "The credential is valid but lacks permission for this model."
156          if isinstance(exc, anthropic.NotFoundError):
157              return f"Unknown model id {self.model!r}. Check RAG_MODEL."
158          if isinstance(exc, anthropic.RateLimitError):
159              headers = getattr(getattr(exc, "response", None), "headers", {}) or {}
160              return f"Rate limited. Retry after {headers.get('retry-after', '60')}s."
161          if isinstance(exc, anthropic.APIConnectionError):
162              return "Could not reach the Anthropic API. Check network connectivity."
163          if isinstance(exc, anthropic.APIStatusError):
164              if exc.status_code >= 500:
165                  return f"Anthropic server error ({exc.status_code}). Retry shortly."
166              return f"API error {exc.status_code}: {getattr(exc, 'message', exc)}"
167          return str(exc)
168
```

Error translation, ordered from most specific to least. Each branch answers the only question a user has at that moment: **150-153** no usable credentials, **154-155** valid credentials without access to this model, **156-157** a model id that does not exist and the setting that holds it, **158-160** a rate limit with the retry delay read from the response headers, **161-162** no network. **167** falls back to the raw error rather than hiding an unanticipated failure behind a generic message.

```
169
170  def _claude_usage(usage: Any) -> dict[str, Any]:
171      if usage is None:
172          return {}
173      out = {
174          "input_tokens": getattr(usage, "input_tokens", None),
175          "output_tokens": getattr(usage, "output_tokens", None),
176          "cached_tokens": getattr(usage, "cache_read_input_tokens", None),
177          "cache_written_tokens": getattr(usage, "cache_creation_input_tokens", None),
178      }
179      return {k: v for k, v in out.items() if v is not None}
180
```

Usage figures, read defensively with `getattr` so an SDK that stops reporting a field degrades the statistics instead of raising. **179** drops the missing ones, so the output shows what is actually known.

```
181
182  # ---------------------------------------------------------------------------
183  # Gemini
184  # ---------------------------------------------------------------------------
185
186  # Claude's effort scale mapped onto Gemini's thinking levels. Gemini 3.x thinks
187  # by default and cannot be switched off, only turned down.
188  _THINKING_LEVEL = {
189      "low": "low",
190      "medium": "medium",
191      "high": "high",
192      "xhigh": "high",
193      "max": "high",
194  }
195  # Finish reasons that mean the model declined rather than answered.
196  _GEMINI_REFUSALS = {"SAFETY", "PROHIBITED_CONTENT", "BLOCKLIST", "SPII", "RECITATION"}
197
```

The Gemini section opens with the two mappings that differ from Claude. **186-194** map the effort scale onto thinking levels, collapsing the top three to "high" because that is the ceiling here - and the comment records that Gemini 3.x always thinks and can only be turned down, not off. **196** lists the finish reasons that mean the model declined rather than answered.

```
198
199 class GeminiBackend:
200     name = "gemini"
201
202     def __init__(self, config: Config):
203         self.config = config
204         self.model = config.model or GEMINI_DEFAULT_MODEL
205         try:
206             from google import genai
207             from google.genai import errors, types
208         except ImportError as exc:
209             raise MissingCredentials("Gemini answers need: pip install google-genai") from exc
210         self._genai, self._types, self._errors = genai, types, errors
211
```

Construction, with its own lazy import so a user of one provider never needs the other's SDK installed.

```
212         vertex = os.environ.get("GOOGLE_GENAI_USE_VERTEXAI", "").lower() in {"1", "true", "yes"}
213         if not vertex and not any(os.environ.get(v) for v in GEMINI_KEY_VARS):
214             raise MissingCredentials(CREDENTIAL_HINT)
215         try:
216             # Reads GEMINI_API_KEY / GOOGLE_API_KEY, or Vertex AI settings.
217             self._client = genai.Client()
218         except Exception as exc:
219             raise MissingCredentials(CREDENTIAL_HINT) from exc
220
```

Credential checks. **212-214** accept either an API key or Vertex AI configuration, failing early with the shared message if neither is present - which turns a confusing SDK error into one that names the variable to set.

```
221     def _request(self, system_parts, messages, max_tokens, effort) -> dict[str, Any]:
222         types = self._types
223         contents = [
224             types.Content(
225                 # Gemini calls the assistant turn "model".
226                 role="model" if m.get("role") == "assistant" else "user",
227                 parts=[types.Part(text=str(m.get("content", "")))],
228             )
229             for m in messages
230         ]
231         level = _THINKING_LEVEL.get((effort or self.config.effort).lower(), "medium")
232         config = types.GenerateContentConfig(
233             system_instruction="\n\n".join(part for part in system_parts if part),
234             max_output_tokens=max_tokens or self.config.max_tokens,
235             thinking_config=types.ThinkingConfig(thinking_level=level),
236             # No tools are declared, and leaving automatic function calling on
237             # makes the SDK warn on every call.
238             automatic_function_calling=types.AutomaticFunctionCallingConfig(disable=True),
239         )
240         return {"model": self.model, "contents": contents, "config": config}
241
```

Translating the neutral request into this provider's shape. **225-226** rename the assistant role to "model", which is this API's name for the same thing. **233** joins the system parts, since this API takes one instruction rather than a list. **236-238** disable automatic function calling: no tools are declared, and leaving it enabled makes the SDK warn on every single call.

```
242     def _retryable(self, exc: Exception) -> bool:
243         """Overload (503) and rate limits (429) clear on their own; 4xx do not."""
244         errors = self._errors
245         if isinstance(exc, errors.ServerError):
246             return True
247         return isinstance(exc, errors.ClientError) and getattr(exc, "code", None) == 429
248
```

Which failures are worth retrying: server errors, which clear on their own, and rate limits. Ordinary client errors are not retried, because a malformed request or a bad key will fail identically every time.

```
249      def _with_retries(self, call):
250          """Run ``call``, retrying transient failures with a short backoff.
251
252          Gemini returns 503 when its capacity is busy, often enough to be worth
253          handling: observed once in a handful of real requests here.
254          """
255          import time
256
257          delay = self.config.retry_initial_seconds
258          for attempt in range(self.config.retry_attempts + 1):
259              try:
260                  return call()
261              except Exception as exc:
262                  if attempt >= self.config.retry_attempts or not self._retryable(exc):
263                      raise RuntimeError(self.describe_error(exc)) from exc
264                  time.sleep(delay)
265                  delay *= 2
266
```

The retry loop. The docstring records why it was added rather than theorised: this provider returns 503 when busy, and it happened during real use here. **264-265** double the delay after each attempt, so two retries wait 1.5 and 3 seconds rather than hammering a service that is already overloaded. **262-263** stop immediately for anything not retryable.

```
267      def complete(self, system_parts, messages, max_tokens=None, effort=None) -> Completion:
268          request = self._request(system_parts, messages, max_tokens, effort)
269          response = self._with_retries(lambda: self._client.models.generate_content(**request))
270
271          blocked = getattr(getattr(response, "prompt_feedback", None), "block_reason", None)
272          candidate = (response.candidates or [None])[0]
273          finish = str(getattr(candidate, "finish_reason", "") or "").rsplit(".", 1)[-1]
274          refused = bool(blocked) or finish in _GEMINI_REFUSALS
275          return Completion(
276              text=(response.text or "").strip() if not refused else "",
277              model=self.model,
278              usage=_gemini_usage(getattr(response, "usage_metadata", None)),
279              stop_reason="refusal" if refused else ("max_tokens" if finish == "MAX_TOKENS" else "end_turn"),
280              refused=refused,
281              refusal_detail=str(blocked or finish or ""),
282          )
283
```

The call and its response. **271-274** check for refusal in both places this API can report it - a blocked prompt and a refusing finish reason. **273** takes the text after the last dot, because the finish reason may arrive as an enum whose string form is qualified. **279** maps the finish reason onto the neutral vocabulary, so a caller can tell a truncated answer from a complete one without knowing whose API produced it.

```
284      def stream(self, system_parts, messages, max_tokens=None, effort=None) -> Iterator[str]:
285          request = self._request(system_parts, messages, max_tokens, effort)
286          # Only the *opening* of the stream is retried: once text has reached the
287          # caller, restarting would repeat it.
288          started = False
289          try:
290              for chunk in self._with_retries(
291                  lambda: self._client.models.generate_content_stream(**request)
292              ):
293                  text = getattr(chunk, "text", None)
294                  if text:
295                      started = True
296                      yield text
297          except RuntimeError:
298              raise
299          except Exception as exc:
300              message = self.describe_error(exc)
301              if started:
302                  message = f"The answer was cut short: {message}"
303              raise RuntimeError(message) from exc
304
```

Streaming, with a distinction that matters. **286-288**: only the opening of the stream is retried, because once text has reached the caller a restart would repeat it. **299-303** handle a failure mid-stream - which happened in real use - by prefixing the message with "the answer was cut short", so a user who already has half an answer on screen understands what they are looking at rather than assuming it is complete.

```
305      def describe_error(self, exc: Exception) -> str:
306          errors = self._errors
307          if isinstance(exc, MissingCredentials):
308              return CREDENTIAL_HINT
309          if isinstance(exc, errors.ClientError):
310              code = getattr(exc, "code", None)
311              message = getattr(exc, "message", str(exc))
312              # An invalid key comes back as a 400, not a 401.
313              if "api key not valid" in str(message).lower() or "api_key_invalid" in str(message).lower():
314                  return (
315                      "Gemini rejected the API key, so a key was found but is wrong or "
316                      "expired. Check GEMINI_API_KEY in your .env file (or the "
317                      "environment, which overrides it). Free key: "
318                      "https://aistudio.google.com/apikey"
319                  )
```

Error translation for this provider. **312-319** exist because of a real diagnostic trap: an invalid key is reported as a 400, not a 401, so without this branch the message would be a generic bad-request error. Instead it says a key was found and is wrong, names the file to check, notes that the environment overrides that file, and links where to get a new one.

```
320              if code == 401 or code == 403:
321                  return f"Gemini rejected the credentials ({code}). Check GEMINI_API_KEY."
322              if code == 404:
323                  return (
324                      f"Gemini does not offer model {self.model!r} to this key. "
325                      "Set RAG_MODEL to a current model, e.g. gemini-3.8-flash."
326                  )
327              if code == 429:
328                  return "Gemini rate limit or quota reached. Retry later, or use a different key."
329              return f"Gemini rejected the request ({code}): {message}"
330          if isinstance(exc, errors.ServerError):
331              return f"Gemini server error ({getattr(exc, 'code', '5xx')}). Retry shortly."
332          if isinstance(exc, errors.APIError):
333              return f"Gemini API error: {getattr(exc, 'message', exc)}"
334          return str(exc)
335
```

The remaining cases: rejected credentials, a model this key cannot use - with a current model named, since that was the actual cause when it occurred - a quota limit, and server errors.

```
336
337  def _gemini_usage(usage: Any) -> dict[str, Any]:
338      if usage is None:
339          return {}
340      out = {
341          "input_tokens": getattr(usage, "prompt_token_count", None),
342          "output_tokens": getattr(usage, "candidates_token_count", None),
343          "cached_tokens": getattr(usage, "cached_content_token_count", None),
344          "thinking_tokens": getattr(usage, "thoughts_token_count", None),
345      }
346      return {k: v for k, v in out.items() if v is not None}
347
348
```

Usage figures again, under this provider's field names. **344** reports thinking tokens, which have no Claude equivalent and are worth seeing because they are billed.

```
349  # ---------------------------------------------------------------------------
350  # selection
351  # ---------------------------------------------------------------------------
352
353  BACKENDS = {"claude": ClaudeBackend, "gemini": GeminiBackend}
354  _CACHE: dict[tuple[str, str], Backend] = {}
355
356
```

The registry and a backend cache keyed by provider and model, so repeated queries in one process do not reconstruct a client each time.

```
357  def credentials_present() -> dict[str, bool]:
358      """Which providers have credentials in the environment."""
359      anthropic_profile = any(
360          os.path.isdir(os.path.expanduser(p)) for p in ("~/.config/anthropic", "~/.anthropic")
361      )
362      return {
363          "gemini": any(bool(os.environ.get(v)) for v in GEMINI_KEY_VARS)
364          or os.environ.get("GOOGLE_GENAI_USE_VERTEXAI", "").lower() in {"1", "true", "yes"},
365          "claude": any(bool(os.environ.get(v)) for v in ANTHROPIC_KEY_VARS) or anthropic_profile,
366      }
367
368
```

Detecting credentials without constructing anything. **359-361** also look for a logged-in Anthropic profile directory, because of the resolution order noted at **92-94** - checking only environment variables would report "no credentials" for a machine that is in fact logged in.

```
369  def resolve_provider(config: Config) -> str:
370      """The provider to use: the configured one, or whichever has credentials."""
371      choice = (config.llm_provider or "auto").lower()
372      if choice in BACKENDS:
373          return choice
374      if choice not in ("auto", ""):
375          raise ValueError(
376              f"Unknown RAG_LLM_PROVIDER {choice!r}. Use one of: {', '.join(BACKENDS)}, auto."
377          )
378      present = credentials_present()
379      # Gemini first: it is the one with a free tier, so a key for it is the more
380      # likely deliberate choice when both are present.
381      for provider in ("gemini", "claude"):
382          if present[provider]:
383              return provider
384      raise MissingCredentials(CREDENTIAL_HINT)
385
```

Provider resolution. **372-377** accept an explicit choice and reject an unknown one by listing the valid values. **378-383** are the automatic path, and **379-380** explain the order: Gemini is tried first because it is the one with a free tier, so a key for it is the more likely deliberate choice when both exist.

```
386
387  def get_backend(config: Config) -> Backend:
388      provider = resolve_provider(config)
389      key = (provider, config.model or "")
390      if key not in _CACHE:
391          _CACHE[key] = BACKENDS[provider](config)
392      return _CACHE[key]
393
394
395  def default_model(provider: str) -> str:
396      return GEMINI_DEFAULT_MODEL if provider == "gemini" else CLAUDE_DEFAULT_MODEL
```

Getting a backend, and the default model for a provider - used by the CLI to print what will actually be called before anything is called.

rag/generate.py

164 lines. The prompt, and what happens to the answer afterwards. This is the module that decides the system says 'not in these documents' instead of inventing something plausible.

```
"""Grounded answer generation with inline citations.

Design decisions worth knowing:

* Retrieved context goes in the **system** part, not the user turn. On Claude it
  carries a cache breakpoint, so follow-up turns over the same passages reuse it;
  Gemini caches repeated prefixes on its own. Nothing here is provider-specific:
  the request is handed to a backend in ``llm.py``, which is what makes the same
  prompt, citations and refusal handling work on Claude or Gemini.
* The model is told to answer *only* from the context and to say so when the
  context does not contain the answer. An honest "not in these documents" is the
  single most valuable behaviour in a RAG system -- a fluent guess is worse than
  no answer because it is indistinguishable from a real one.
* Citations are ``[n]`` markers keyed to numbered sources. They are parsed back
  out afterwards so the caller gets a structured list of what was actually used,
  not just prose that happens to contain brackets.
"""
```

Three decisions, stated before any code. **5-9**: retrieved context goes in the system part rather than the user turn, which is what lets Claude cache it across follow-up questions. **10-13** is the one that matters most - an honest "not in these documents" is the single most valuable behaviour in a retrieval system, because a fluent guess is worse than no answer: it is indistinguishable from a real one. **14-16**: citations are parsed back out of the answer, so the caller gets a structured list of what was actually used rather than prose that happens to contain brackets.

```

from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Any, Iterator, Sequence

from .config import Config, DEFAULT_CONFIG
from .llm import Backend, get_backend
from .retriever import RetrievedChunk

```

Imports: the settings, the provider abstraction, and the retrieved-chunk type.

```
SYSTEM_PROMPT = """You answer questions using only the excerpts provided in <context>.

Rules:
- Ground every factual claim in the excerpts. Do not use outside knowledge, and do not fill gaps with plaus
    ible-sounding detail.
- Cite with bracketed numbers matching the excerpt ids, e.g. [2] or [1][4]. Place a citation immediately af
    ter the claim it supports.
- When an excerpt's location names a law and an article, also name them in the answer (e.g. "under Article
    61 of the Labour Law [2]"), since that is how legal provisions are referred to. Never cite an article n
    umber that does not appear in an excerpt's location or text.
- If the excerpts do not contain the answer, say so plainly and state what is missing. Do not guess.
- If the excerpts disagree, surface the disagreement rather than silently picking one.
- Answer in the language the question was asked in.
- Be direct. No preamble, no restating the question."""

```

The system prompt - the most consequential twelve lines in the project. **32** forbids outside knowledge and, specifically, filling gaps with plausible detail. **33** fixes the citation format and requires the marker next to the claim, not collected at the end, so a reader can check one sentence against one excerpt. **34** is corpus-specific: name the law and article, because that is how legal provisions are referred to - and never cite an article number that does not appear in an excerpt, which is the instruction that stops the most convincing kind of fabrication. **35** requires an explicit admission of a gap. **36** surfaces disagreement between excerpts rather than silently choosing - with amended legislation, two excerpts disagreeing is a fact about the corpus. **37** keeps an Arabic question answered in Arabic.

```
CITATION_RE = re.compile(r"\[(\d{1,3})\]")


```

The citation pattern. Bounded to three digits, so a year in brackets is not mistaken for a citation.

```
@dataclass
class Answer:
    question: str
    text: str
    sources: list[RetrievedChunk] = field(default_factory=list)
    cited_indices: list[int] = field(default_factory=list)
    usage: dict[str, Any] = field(default_factory=dict)
    stop_reason: str | None = None
    model: str | None = None

```

The answer record: the text, the sources it was given, which ones it actually cited, and the usage and stop reason from the provider.

```
    @property
    def cited_sources(self) -> list[RetrievedChunk]:
        return [self.sources[i - 1] for i in self.cited_indices if 1 <= i <= len(self.sources)]

    @property
    def grounded(self) -> bool:
        """Whether the answer actually pointed at its evidence."""
        return bool(self.cited_indices)

```

54-55 resolve citation numbers back to the chunks, ignoring any out of range. **57-60** is a quality signal in one line: an answer with no citations did not point at its evidence, which the caller can warn about.

```
    def to_dict(self) -> dict[str, Any]:
        return {
            "question": self.question,
            "answer": self.text,
            "cited": [
                {"n": i, "location": self.sources[i - 1].location,
                 "source": self.sources[i - 1].chunk.source}
                for i in self.cited_indices
                if 1 <= i <= len(self.sources)
            ],
            "sources": [s.to_dict() for s in self.sources],
            "usage": self.usage,
            "model": self.model,
            "stop_reason": self.stop_reason,
        }

```

Serialisation, used by the HTTP API, the JSON output mode and the evaluation harness. The cited list carries each source's location, so a consumer gets "Labour Law - Article 61 (p.22)" without having to look anything up.

```

def format_context(chunks: Sequence[RetrievedChunk]) -> str:
    """Render retrieved chunks as numbered, attributed excerpts."""
    blocks = []
    for i, hit in enumerate(chunks, start=1):
        blocks.append(
            f'<excerpt id="{i}" source="{hit.chunk.source}" location="{hit.location}">\n'
            f"{hit.text}\n"
            f"</excerpt>"
        )
    return "<context>\n" + "\n\n".join(blocks) + "\n</context>"

```

Rendering the context. Each excerpt is numbered and carries its source path and its human-readable location as attributes - **84** - which is what makes the citation instruction on **34** satisfiable: the model can only name the article because the location attribute tells it which one this is.

```

def parse_citations(text: str, n_sources: int) -> list[int]:
    """Extract cited excerpt numbers, in order of first appearance."""
    seen: list[int] = []
    for match in CITATION_RE.finditer(text):
        n = int(match.group(1))
        if 1 <= n <= n_sources and n not in seen:
            seen.append(n)
    return seen

```

Parsing citations back out. **96** keeps only numbers that refer to an excerpt actually provided and drops repeats, so the result is the set of sources used, in the order they were first cited.

```
100
101 def _build_request(
102     question: str,
103     chunks: Sequence[RetrievedChunk],
104     history: Sequence[dict[str, Any]] | None = None,
105 ) -> tuple[list[str], list[dict[str, Any]]]:
106     """System parts and messages, in the provider-neutral shape."""
107     system_parts = [SYSTEM_PROMPT, format_context(chunks)]
108     messages = list(history or []) + [{"role": "user", "content": question}]
109     return system_parts, messages
110
```

Assembling the request in the provider-neutral shape: two system parts - rules, then context - and the conversation with the new question appended. **108** is what makes follow-up questions work, since prior turns are carried while the context is rebuilt from the new retrieval.

```
111
112 def generate_answer(
113     question: str,
114     chunks: Sequence[RetrievedChunk],
115     config: Config = DEFAULT_CONFIG,
116     history: Sequence[dict[str, Any]] | None = None,
117 ) -> Answer:
118     """Answer the question from the retrieved context, on either provider."""
119     if not chunks:
120         return Answer(
121             question=question,
122             text="Nothing in the index matched that question, so there is no "
123                  "grounded answer to give. Try rephrasing, or index more documents.",
124             sources=[],
125         )
126
```

Generating an answer. **119-125** handle the empty-retrieval case without calling the model at all: there is nothing to ground an answer in, so the honest response is the one written here, and it suggests the two things that actually help.

```
127     backend: Backend = get_backend(config)
128     system_parts, messages = _build_request(question, chunks, history)
129     completion = backend.complete(system_parts, messages)
130
131     if completion.refused:
132         return Answer(
133             question=question,
134             text=f"The model declined to answer this request ({completion.refusal_detail or 'unspecified'})
                    .",
135             sources=list(chunks),
136             stop_reason="refusal",
137             model=completion.model,
138         )
139
```

The call, and refusal handling. **131-138** turn a refusal into a plain statement that the model declined, carrying the category - rather than an empty answer, which would look like a bug, or an exception, which would lose the retrieved sources.

```
140     return Answer(
141         question=question,
142         text=completion.text,
143         sources=list(chunks),
144         cited_indices=parse_citations(completion.text, len(chunks)),
145         usage=completion.usage,
146         stop_reason=completion.stop_reason,
147         model=completion.model,
148     )
149
```

The successful answer, with citations parsed and usage recorded. Note **143** keeps all the retrieved chunks, not just the cited ones, so a reader can see what the model was given and judge what it chose to ignore.

```

def stream_answer(
    question: str,
    chunks: Sequence[RetrievedChunk],
    config: Config = DEFAULT_CONFIG,
    history: Sequence[dict[str, Any]] | None = None,
) -> Iterator[str]:
    """Yield answer text incrementally. Terminal UIs and SSE both use this."""
    if not chunks:
        yield "Nothing in the index matched that question."
        return

    backend: Backend = get_backend(config)
    system_parts, messages = _build_request(question, chunks, history)
    yield from backend.stream(system_parts, messages)
```

The streaming variant, used by both the terminal and the web API. It is deliberately thinner than the non-streaming path: citations cannot be parsed until the text is complete, so the caller does that once the stream ends.

rag/rerank.py

80 lines. The alternative reranker: ask the language model to score the candidates instead of running a local cross-encoder. Off by default, and written so that it can never make retrieval worse than not using it.

```
"""Optional LLM reranking.

Hybrid fusion is good at *recall* -- getting the right chunk into the top 30 --
and only approximately right about *order*. A reranker reads the query and each
candidate together and scores actual relevance, which is what the fused ranking
can only approximate.

It costs one API call per query on whichever provider is configured, so the
local cross-encoder in ``crossencoder.py`` is the default; this one is for when
you would rather spend an API call than CPU time.
"""
```

The docstring states the division of labour. Hybrid fusion is good at recall - getting the right passage into the top thirty - and only approximately right about order. **8-10** give the honest trade: this costs an API call per query, so the local cross-encoder is the default and this exists for when spending a call is preferable to spending CPU time.

```

from __future__ import annotations

import re
from typing import Sequence

from .config import Config, DEFAULT_CONFIG
from .llm import get_backend
from .retriever import RetrievedChunk

RERANK_SYSTEM = """You score how well each excerpt answers a question.

Output one line per excerpt, exactly: ID=SCORE
SCORE is 0-10, where 10 means the excerpt directly answers the question, 5 means
it is related background, and 0 means it is irrelevant. Judge only whether the
excerpt helps answer *this* question. Output nothing else."""

_SCORE_RE = re.compile(r"^\s*(\d{1,3})\s*=\s*(\d{1,2}(?:\.\d+)?)\s*$", re.MULTILINE)
_SNIPPET_CHARS = 1200


def llm_rerank(
    query: str,
    candidates: Sequence[RetrievedChunk],
    config: Config = DEFAULT_CONFIG,
    top_n: int | None = None,
) -> list[RetrievedChunk]:
    """Re-order candidates by model-judged relevance.

    On any failure the original fused order is returned unchanged -- a reranker
    that can break retrieval is worse than no reranker.
    """
```

The function, and its governing rule, stated in the docstring: on any failure the original order is returned unchanged. A reranker that can break retrieval is worse than no reranker, so every path below either improves the order or leaves it alone.

```
44      if len(candidates) <= 1:
45          return list(candidates)
46
47      pool = list(candidates[: top_n or len(candidates)])
48      excerpts = "\n\n".join(
49          f'<excerpt id="{i}">\n{hit.chunk.indexed_text[:_SNIPPET_CHARS]}\n</excerpt>'
50          for i, hit in enumerate(pool, start=1)
51      )
52      prompt = f"<question>{query}</question>\n\n{excerpts}"
```

Building the prompt. **48-51** wrap each excerpt in a tag carrying its number, which is both how the model is told to refer to them and what makes the response parseable. **49** sends `indexed_text`, so the model sees the law and article label as well as the passage.

```
53
54      # Relevance judgement is a cheap discrimination task, not a reasoning one.
55      completion = get_backend(config).complete(
56          [RERANK_SYSTEM],
57          [{"role": "user", "content": prompt}],
58          max_tokens=1024,
59          effort="low",
60      )
61      text = completion.text
```

The call. **59** asks for low effort deliberately - the comment explains why: judging whether a passage is relevant is a quick discrimination task, not a reasoning one, and paying for extended thinking here would buy nothing.

```
62      scores: dict[int, float] = {}
63      for match in _SCORE_RE.finditer(text):
64          idx, score = int(match.group(1)), float(match.group(2))
65          if 1 <= idx <= len(pool):
66              scores[idx] = score
67
68      if not scores:  # unparseable response -- keep the fused order
69          return list(candidates)
70
```

Parsing. **65** ignores any id outside the range actually sent, so a hallucinated excerpt number is discarded rather than indexing into the wrong candidate. **68-69** are the main safety net: if nothing parsed at all, the fused order is returned untouched.

```
71      for i, hit in enumerate(pool, start=1):
72          hit.rerank_score = scores.get(i)
73
74      # Unscored candidates sink below scored ones but keep their relative order.
75      reranked = sorted(
76          pool,
77          key=lambda h: (h.rerank_score if h.rerank_score is not None else -1.0, h.score),
78          reverse=True,
79      )
80      return reranked + list(candidates[len(pool):])
```

Reordering. **74-79** sort by the model's score with the fused score as a tiebreak, and candidates the model did not score sink below those it did while keeping their relative order - so a partial response still helps rather than scrambling the list. **80** re-attaches anything beyond the reranked pool.

rag/pipeline.py

269 lines. The object everything else drives: the CLI, the web server and the evaluation harness all call these methods. It wires the store, the retriever and the model together and owns the decisions that span them.

```
"""The facade: ingest documents, search them, ask questions about them.

    from rag import RAGPipeline

    rag = RAGPipeline()
    rag.ingest(["./docs"])
    answer = rag.ask("What is the deployment procedure?")
    print(answer.text)
    for source in answer.cited_sources:
        print(" -", source.location)
"""
```

The docstring is a complete session: construct, ingest, ask, then print the answer and the sources it cited. **9-10** deliberately show the citation loop, because an answer without its sources is only half of what this system produces.

```

from __future__ import annotations

import logging
import os
from dataclasses import replace
from pathlib import Path
from typing import Any, Callable, Iterator, Sequence

from .chunking import Chunk
from .config import Config, DEFAULT_CONFIG
from .generate import Answer, generate_answer, stream_answer
from .loaders import discover, load_document
from .retriever import ChunkFilter, RetrievedChunk, Retriever
from .store import IngestReport, VectorStore

log = logging.getLogger("rag")

```

Imports, which read as a map of the package: chunking, config, generation, loaders, retrieval, storage. The pipeline is the only module that touches all of them.

```

class RAGPipeline:
    def __init__(
        self,
        config: Config | None = None,
        load: bool = True,
        adopt_index_settings: bool = True,
    ):
        # Copy the shared default so adopting index settings never leaks into
        # other pipelines in the same process.
        self.config = config if config is not None else replace(DEFAULT_CONFIG)
        self.store = VectorStore.load(self.config) if load else VectorStore(self.config)
        self._embedder: Any | None = None
        self._embedder_failed = False
        if load and adopt_index_settings:
            self._adopt_index_chunking()

```

Construction. **38-40** copy the shared default settings rather than using the object itself - the comment gives the reason: this pipeline may adopt the index's chunk settings below, and mutating the shared default would leak those into every other pipeline in the same process. **41** loads the index unless told not to. **42-43** leave the embedder unloaded; it is built on first use.

```
47      def _adopt_index_chunking(self) -> None:
48          """Keep chunking consistent with how the existing index was built.
49
50          Without this, an index built with RAG_CHUNK_TOKENS=1200 would get 512-
51          token chunks for every file added later unless the variable was set
52          again each time. A value the caller chose explicitly is left alone.
53          """
54          stored = self.store.fingerprint
55          for field_name, env_var in (
56              ("chunk_tokens", "RAG_CHUNK_TOKENS"),
57              ("chunk_overlap_tokens", "RAG_CHUNK_OVERLAP"),
58          ):
59              if field_name not in stored or env_var in os.environ:
60                  continue
61              default = Config.__dataclass_fields__[field_name].default
62              if getattr(self.config, field_name) == default:
63                  setattr(self.config, field_name, stored[field_name])
64
```

Adopting the index's own chunking settings, which exists because of a real failure mode spelled out in the docstring: an index built with 1,200-token chunks would silently get 512-token chunks for every file added later, unless the environment variable happened to be set again. **59** leaves an explicit environment setting alone, and **62** only overrides a value still at its default - so a caller who chose a size keeps it, and a caller who did not gets consistency with the index.

```
65      # -- components ---------------------------------------------------------
66
67      @property
68      def embedder(self) -> Any | None:
69          """Load embedding weights on first use; ``None`` means keyword-only.
70
71          Dense retrieval is a quality upgrade, not a hard requirement, so a
72          missing model or absent torch degrades to BM25 with a warning instead of
73          taking the whole system down.
74          """
```

The embedder, loaded lazily. The docstring states the policy: dense retrieval is a quality upgrade, not a hard requirement, so a missing model or an absent torch must degrade the system rather than take it down.

```
75          if self._embedder is None and not self._embedder_failed:
76              if not self.config.dense_enabled():
77                  self._embedder_failed = True
78                  return None
79              try:
80                  from .embeddings import get_embedder
81
82                  self._embedder = get_embedder(self.config)
83              except Exception as exc:
84                  self._embedder_failed = True
85                  log.warning("Dense retrieval unavailable (%s); using BM25 only.", exc)
86          return self._embedder
87
```

75 and the failure flag together mean the load is attempted exactly once - without the flag, a machine with no torch would retry the import on every single query. **83-85** catch everything and warn, which is the difference between "search still works, keyword-only" and a traceback. **76-78** handle dense retrieval being switched off deliberately, which takes the same path without the warning.

```
88      @property
89      def retriever(self) -> Retriever:
90          return Retriever(self.store, self.embedder, self.config)
91
92      # -- ingestion ----------------------------------------------------------
93
```

The retriever is constructed per access rather than cached, because it holds no state - it is three references - and building it fresh means a configuration change takes effect immediately.

```
 94      def ingest(
 95          self,
 96          paths: Sequence[str | Path],
 97          recursive: bool = True,
 98          force: bool = False,
 99          progress: Callable[[str, str], None] | None = None,
100      ) -> IngestReport:
101          """Index files and directories. Unchanged files are skipped.
102
103          Files that were indexed from one of these directories but no longer
104          exist there (deleted or renamed) are removed from the index first --
105          otherwise their passages keep appearing in answers, and a renamed file
106          would be mistaken for a duplicate of its own former name.
107          """
```

Ingestion, and a subtle piece of housekeeping explained in the docstring: files that were indexed from these directories but no longer exist are removed first. Without that, a deleted document's passages keep appearing in answers, and a renamed file is mistaken for a duplicate of its own former name - which would silently skip it.

```
108          report = IngestReport()
109          report.pruned = self._prune_missing(paths)
110          documents = []
111          for path in discover(paths, recursive=recursive):
112              try:
113                  documents.append(load_document(path, cache_dir=self.config.extract_cache_dir()))
114              except Exception as exc:
115                  report.failures.append((str(path), str(exc)))
116                  log.warning("Could not read %s: %s", path, exc)
117
118          sub_report = self.store.add_documents(
119              documents, embedder=self.embedder, force=force, progress=progress
120          )
121          sub_report.failures.extend(report.failures)
122          sub_report.pruned = report.pruned
123          sub_report.removed_chunks += sum(self._pruned_chunk_counts.values())
124          self.store.save()
125          return sub_report
126
```

The ingest body. **111-116** load each discovered file, recording a failure and continuing rather than aborting - one unreadable PDF in a folder of two hundred must not stop the other 199. **118-120** hand everything to the store. **121-123** merge the pruning results into the store's report so the caller sees one summary. **124** saves once at the end rather than per document.

```
127      def _prune_missing(self, paths: Sequence[str | Path]) -> list[str]:
128          """Drop indexed files under ``paths`` that no longer exist on disk.
129
130          Scoped to the directories being ingested, so ingesting one folder never
131          touches sources from another, and non-file sources (``ingest_text``)
132          are never pruned.
133          """
134          self._pruned_chunk_counts: dict[str, int] = {}
135          roots = [Path(p).resolve() for p in paths if Path(p).is_dir()]
136          if not roots:
```

Pruning, scoped deliberately. The docstring names both limits: only sources under the directories actually being ingested are considered, so ingesting one folder never deletes another folder's documents, and text ingested directly as a string is never pruned because it has no file to be missing.

```
137              return []
138          for source in list(self.store.sources):
139              candidate = Path(source)
140              if "://" in source or candidate.exists():
141                  continue
142              resolved = candidate.resolve()
143              if any(resolved.is_relative_to(root) for root in roots):
144                  self._pruned_chunk_counts[source] = self.store.remove_source(source)
145          return list(self._pruned_chunk_counts)
146
```

138-141 skip anything that still exists, and anything whose source is a URL rather than a path. **143** is the scoping test: only sources genuinely inside one of the ingested roots are removed. **144** records how many chunks each pruned source contributed, which is what lets the summary say "-312 stale chunks".

```
147     def ingest_text(self, text: str, source: str, title: str | None = None) -> IngestReport:
148         """Index a string directly -- for content that never lands on disk."""
149         import hashlib
150 
151         from .loaders import Document
152 
153         doc = Document(
154             doc_id=hashlib.sha1(source.encode("utf-8")).hexdigest()[:16],
155             source=source,
156             title=title or source,
157             text=text,
158             metadata={
159                 "format": "text",
160                 "content_hash": hashlib.sha256(text.encode("utf-8")).hexdigest()[:16],
161             },
162         )
163         report = self.store.add_documents([doc], embedder=self.embedder, force=True)
164         self.store.save()
165         return report
166 
```

Indexing a string directly, for content that never lands on disk. It builds a Document by hand with the same identity and content hashes the file loader would produce, so the store cannot tell the difference. **163** forces the add, since there is no file whose modification time could be compared.

```
167     def remove(self, source: str) -> int:
168         removed = self.store.remove_source(source)
169         self.store.save()
170         return removed
171 
172     def reset(self) -> None:
173         self.store.clear()
174 
175     # -- query ---------------------------------------------------------
176 
```

Removing one source, and clearing the index entirely. Both save immediately.

```
177     def search(
178         self,
179         query: str,
180         top_k: int | None = None,
181         chunk_filter: ChunkFilter | None = None,
182         rerank: bool | None = None,
183     ) -> list[RetrievedChunk]:
184         """Retrieve without generating -- no API key needed."""
185         if self.store.size == 0:
186             # Nothing to search: return before loading embedding weights, which
187             # takes seconds and would be pure waste on an empty index.
188             return []
189         return self.retriever.retrieve(query, top_k=top_k, chunk_filter=chunk_filter, rerank=rerank)
190 
```

Search without generation - which needs no API key at all, and is the half of the system that can be evaluated offline. **185-188** short-circuit an empty index, and the comment says what that saves: loading embedding weights takes seconds and would be pure waste with nothing to search.

```
191     def context_for(
192         self,
193         question: str,
194         top_k: int | None = None,
195         chunk_filter: ChunkFilter | None = None,
196         rerank: bool | None = None,
197     ) -> list[RetrievedChunk]:
198         """The passages to put in front of the model, filled to a token budget.
199 
200         A budget rather than a fixed count, because chunk sizes vary tenfold --
201         one article can be a sentence, a page window a thousand tokens. Measured
202         on the legal test set, filling ~4,000 tokens with the best-ranked chunks
203         puts the answering passage in front of the model far more often than a
204         fixed six. The top hit is always included, whatever its size.
205         """
```

Context selection, and the reasoning that sets this system apart from a fixed top-k. Chunk sizes vary tenfold - a constitutional article can be one sentence, a page window a thousand tokens - so a fixed count of passages gives wildly different amounts of evidence from one question to the next. **202-204** record the measurement: filling a token budget with the best-ranked chunks puts the answering passage in front of the model far more often than taking a fixed six.

```
206          cap = top_k or self.config.top_k
207          hits = self.search(question, top_k=cap, chunk_filter=chunk_filter, rerank=rerank)
208          budget = self.config.context_tokens
209          chosen: list[RetrievedChunk] = []
210          used = 0
211          for hit in hits:
212              if chosen and used + hit.chunk.token_count > budget:
213                  break
214              chosen.append(hit)
215              used += hit.chunk.token_count
216          return chosen
217
```

The budget loop. **212** carries the guarantee stated in the docstring: the `chosen and` test means the top-ranked hit is always included even if it alone exceeds the budget - the best passage is never dropped for being large. Everything after it must fit.

```
218      def ask(
219          self,
220          question: str,
221          top_k: int | None = None,
222          chunk_filter: ChunkFilter | None = None,
223          history: Sequence[dict[str, Any]] | None = None,
224          rerank: bool | None = None,
225      ) -> Answer:
226          hits = self.context_for(question, top_k=top_k, chunk_filter=chunk_filter, rerank=rerank)
227          return generate_answer(question, hits, self.config, history=history)
228
```

Asking a question: select context, then generate. Two lines, because every decision was made elsewhere.

```
229      def ask_stream(
230          self,
231          question: str,
232          top_k: int | None = None,
233          chunk_filter: ChunkFilter | None = None,
234          history: Sequence[dict[str, Any]] | None = None,
235          rerank: bool | None = None,
236      ) -> tuple[list[RetrievedChunk], Iterator[str]]:
237          """Return the sources up front, then a generator of answer text.
238
239          Sources first so a UI can render citations while the answer streams.
240          """
241          hits = self.context_for(question, top_k=top_k, chunk_filter=chunk_filter, rerank=rerank)
242          return hits, stream_answer(question, hits, self.config, history=history)
243
```

The streaming variant, whose signature is the interesting part. It returns the sources and a generator, sources first, so a UI can render the citation list immediately and let the answer arrive into it - rather than showing nothing until the model finishes.

```
244      # -- introspection ---------------------------------------------------
245
246      def stats(self) -> dict[str, Any]:
247          from .llm import credentials_present, default_model, resolve_provider
248
249          stats = self.store.stats()
250          stats["warning"] = self.store.check_fingerprint()
251          keys = credentials_present()
252          stats["keys_found"] = sorted(name for name, ok in keys.items() if ok)
253          # Where the key came from, so "it is set but not picked up" is visible.
254          from .config import ENV_FILENAME, LOADED_ENV
255
256          stats["env_file_keys"] = sorted(LOADED_ENV)
257          stats["env_filename"] = ENV_FILENAME
```

Introspection, built to answer the question users actually have: why is it not working. **250** reports settings drift. **252** reports which providers have credentials. **253-257** report which keys came from the .env file and what that file is called - the comment names the exact confusion this resolves, a key that is set somewhere but not being picked up.

```
258          try:
259              provider = resolve_provider(self.config)
260              stats["provider"] = provider
261              stats["answer_model"] = self.config.model or default_model(provider)
262          except Exception:
263              # No credentials, or an unknown provider name: retrieval still works.
264              stats["provider"] = None
265              stats["answer_model"] = None
266          return stats
267
268      def chunk_by_id(self, chunk_id: str) -> Chunk | None:
269          return next((c for c in self.store.chunks if c.chunk_id == chunk_id), None)
```

258-265 resolve the provider and model inside a try, because having no credentials is a normal state for this command - retrieval works without them, and `stats` must still print rather than raise. **268** looks a chunk up by id, which is how the web UI expands a citation into its full text.

rag/cli.py

388 lines. Nine commands over the pipeline. Most of the length is presentation, and most of the decisions in it are about making failures legible: what went wrong, and what to do next.

```
"""Command-line interface.

    python -m rag ingest ./docs
    python -m rag search "deployment rollback"
    python -m rag ask "How do I roll back a deploy?"
    python -m rag chat
    python -m rag stats
"""

```

The docstring is the command list, so `python -m rag --help` and the module's own documentation cannot drift apart.

```
from __future__ import annotations

import json
import logging
import sys
from pathlib import Path
from typing import Optional

import typer
from rich.console import Console
from rich.markdown import Markdown
from rich.markup import escape
from rich.panel import Panel
from rich.table import Table

from .config import Config
from .pipeline import RAGPipeline
from .retriever import make_filter

```

Imports. Typer builds the argument parsing from type hints, and rich handles the terminal output. **21** imports the markup escaper, which **88** and others need: square brackets in document text are rich markup, so an unescaped Arabic title containing them would be swallowed or would corrupt the colouring.

```
app = typer.Typer(
    add_completion=False,
    no_args_is_help=True,
    help="Hybrid-retrieval RAG over your local documents, answered by Gemini or Claude.",
)
console = Console()

```

The application object. **31** prints help when called with no arguments, which is the right behaviour for a tool with nine subcommands.

```
36
37  def _pipeline(
38      index_dir: Optional[str] = None,
39      top_k: Optional[int] = None,
40      model: Optional[str] = None,
41      rerank: Optional[bool] = None,
42      chunk_tokens: Optional[int] = None,
43      provider: Optional[str] = None,
44  ) -> RAGPipeline:
45      cfg = Config()
46      if index_dir:
47          cfg.index_dir = Path(index_dir)
48      if chunk_tokens:
49          cfg.chunk_tokens = chunk_tokens
50      if top_k:
51          cfg.top_k = top_k
52      if model:
53          cfg.model = model
54      if provider:
55          cfg.llm_provider = provider
56      if rerank is not None:
57          cfg.rerank = rerank
58      return RAGPipeline(cfg, adopt_index_settings=chunk_tokens is None)
59
```

Building a pipeline from the common flags, used by every command. Each override is applied only when the flag was given, so an unset flag falls through to the environment and then to the default. **58** is the subtle one: index settings are adopted unless the user passed an explicit chunk size, so an explicit flag always wins over what the index remembers.

```
60
61  def _warn_if_stale(rag: RAGPipeline, querying: bool = True) -> None:
62      # A search only goes stale if the embedding model changed; chunk settings
63      # are an ingest-time concern.
64      warning = rag.store.check_fingerprint(("embed_model",) if querying else None)
65      if warning:
66          console.print(f"[yellow]![/yellow] {warning}")
67
68
```

The staleness warning, with the scope rule in its comment: a search only goes stale if the embedding model changed, while chunk settings are an ingest-time concern - so querying checks one key and ingesting checks them all.

```
69  @app.command()
70  def ingest(
71      paths: list[str] = typer.Argument(..., help="Files or directories to index."),
72      index_dir: Optional[str] = typer.Option(None, "--index", help="Index location."),
73      force: bool = typer.Option(False, "--force", help="Re-index even if unchanged."),
74      no_recursive: bool = typer.Option(False, "--no-recursive", help="Do not descend into subdirectories."),
75      chunk_tokens: Optional[int] = typer.Option(
76          None, "--chunk-tokens",
77          help="Chunk size. Remembered by the index; 1200 suits Arabic-only corpora.",
78      ),
79      quiet: bool = typer.Option(False, "--quiet", "-q"),
80  ) -> None:
```

The ingest command's arguments. **75-78** document the chunk size where the user will actually look for it, including the measured value that suits an Arabic corpus.

```
81      """Index documents. Unchanged files are skipped, so re-running is cheap."""
82      logging.basicConfig(level=logging.WARNING, format="%(message)s")
83      rag = _pipeline(index_dir, chunk_tokens=chunk_tokens)
84      _warn_if_stale(rag, querying=False)
85
86      def progress(source: str, status: str) -> None:
87          if not quiet and status != "unchanged":
88              console.print(f"  [dim]{escape(Path(source).name)}[/dim] -> {escape(status)}")
89
90      with console.status("Indexing..."):
91          report = rag.ingest(paths, recursive=not no_recursive, force=force, progress=progress)
92
```

86-88 print per-file progress, skipping unchanged files so re-running on a large corpus prints only what changed. **88** escapes both the filename and the status, because both can contain brackets. **90** shows a spinner during the slow part.

```
    console.print(f"[green]OK[/green] {report.summary()}")
    for gone in report.pruned:
        console.print(f"  [yellow]removed[/yellow] {escape(Path(gone).name)} (file no longer exists)")
    for skipped, kept in report.duplicates:
        console.print(f"  [yellow]duplicate[/yellow] {escape(Path(skipped).name)} is identical to {escape(P
            ath(kept).name)}")
    for source, error in report.failures:
        console.print(f"  [red]failed[/red] {escape(source)}: {escape(str(error))}")

```

The summary, then three lists that each name a specific outcome: files removed because they no longer exist, files skipped as duplicates - naming the file kept, so the decision can be checked - and files that failed, with the error. A silent skip is a bug report waiting to happen.

```
    stats = rag.stats()
    mode = f"dense+bm25 ({stats['embedder']})" if stats["dense"] else "bm25 only"
    console.print(
        f"[dim]Index: {stats['chunks']} chunks from {stats['sources']} sources, {mode}, "
        f"{rag.config.chunk_tokens}-token chunks[/dim]"
    )

```

A closing line stating what the index now is: how many chunks, from how many sources, whether dense retrieval is active, and at what chunk size. That one line answers most questions a user has after an ingest.

```

@app.command()
def search(
    query: str = typer.Argument(..., help="Search query."),
    top_k: int = typer.Option(5, "--top-k", "-k"),
    index_dir: Optional[str] = typer.Option(None, "--index"),
    source: Optional[str] = typer.Option(None, "--source", help="Only chunks whose path contains this."),
    fmt: Optional[str] = typer.Option(None, "--format", help="Only this document format, e.g. pdf."),
    rerank: Optional[bool] = typer.Option(None, "--rerank/--no-rerank", help="Reorder results with the rera
        nker (default: on)."),
    full: bool = typer.Option(False, "--full", help="Print whole chunks."),
    as_json: bool = typer.Option(False, "--json"),
) -> None:
    """Retrieve chunks without generating an answer. Needs no API key."""
```

The search command. Its docstring carries the fact worth advertising: retrieval needs no API key, so the system is useful before any credentials exist.

```
    rag = _pipeline(index_dir, top_k=top_k)
    _warn_if_stale(rag)
    hits = rag.search(query, top_k=top_k, chunk_filter=make_filter(source, fmt), rerank=rerank)

    if as_json:
        # Plain stdout, not console.print_json: rich wraps long values to the
        # terminal width, which corrupts the JSON as soon as it is piped.
        print(json.dumps([h.to_dict() for h in hits], ensure_ascii=False))
        return
    if not hits:
        console.print("[yellow]No matches.[/yellow] Is anything indexed? Try `python -m rag stats`.")
        raise typer.Exit(1)

```

125-129 print JSON with the standard library rather than rich's JSON printer, and the comment records why: rich wraps long values to the terminal width, which corrupts the output the moment it is piped into anything. **130-132** exit non-zero on no matches, so a script can tell, and suggest the command that explains why.

```
    for hit in hits:
        body = hit.text if full else (hit.text[:400] + ("..." if len(hit.text) > 400 else ""))
        console.print(
            Panel(
                escape(body),
                title=f"[bold]{hit.rank + 1}. {escape(hit.location)}[/bold]",
                subtitle=f"[dim]{hit.why()}[/dim]",
                border_style="blue",
            )
        )


```

Result panels. **135** truncates to 400 characters unless `--full` was passed. **140** puts the retrieval explanation in the subtitle - which retriever found this, at what rank, with what score - so tuning is a matter of reading the output.

```
146 @app.command()
147 def ask(
148     question: str = typer.Argument(..., help="Question to answer from the indexed documents."),
149     top_k: Optional[int] = typer.Option(None, "--top-k", "-k", help="Max passages (default: fill the token
        budget)."),
150     index_dir: Optional[str] = typer.Option(None, "--index"),
151     model: Optional[str] = typer.Option(None, "--model", help="Model id, e.g. gemini-3.8-flash."),
152     provider: Optional[str] = typer.Option(None, "--provider", help="gemini | claude (default: whichever ha
        s a key)."),
153     source: Optional[str] = typer.Option(None, "--source"),
154     fmt: Optional[str] = typer.Option(None, "--format"),
155     rerank: Optional[bool] = typer.Option(None, "--rerank/--no-rerank", help="Reorder results with the rera
        nker (default: on)."),
156     show_sources: bool = typer.Option(True, "--sources/--no-sources"),
157     as_json: bool = typer.Option(False, "--json"),
158 ) -> None:
159     """Answer a question using the indexed documents, with citations."""
160     rag = _pipeline(index_dir, top_k=top_k, model=model, rerank=rerank, provider=provider)
161     _warn_if_stale(rag)
162     chunk_filter = make_filter(source, fmt)
163
```

The ask command, which carries the fullest set of options: model, provider, filters, reranking, and whether to show sources.

```
164     if as_json:
165         try:
166             answer = rag.ask(question, top_k=top_k, chunk_filter=chunk_filter)
167         except RuntimeError as exc:
168             console.print(f"[red]{exc}[/red]")
169             raise typer.Exit(1)
170         print(json.dumps(answer.to_dict(), ensure_ascii=False))
171         return
172
173     hits, stream = rag.ask_stream(question, top_k=top_k, chunk_filter=chunk_filter)
174     if not hits:
175         console.print("[yellow]Nothing indexed matches that question.[/yellow]")
176         raise typer.Exit(1)
177
```

164-171 are the JSON path, which cannot stream because the whole answer is needed to serialise it. **173** is the normal path, taking sources and a stream together. **174-176** stop before calling the model when retrieval found nothing - there is no point paying for a request that can only be answered with "not in the documents".

```
178     try:
179         parts: list[str] = []
180         with console.status("Thinking..."):
181             first = next(stream, "")
182         parts.append(first)
183         console.print(first, end="")
184         for delta in stream:
185             parts.append(delta)
186             console.print(delta, end="")
187         console.print()
188     except RuntimeError as exc:
189         console.print(f"[red]{exc}[/red]")
190         raise typer.Exit(1)
191
```

Streaming to the terminal. **180-181** show a spinner until the first token arrives and then drop it, which is exactly as long as the user is waiting with nothing to read. **179** and **182** accumulate the text as it prints, because the citations have to be parsed from the complete answer afterwards.

```
192      if show_sources:
193          from .generate import parse_citations
194
195          cited = parse_citations("".join(parts), len(hits))
196          table = Table(show_header=True, header_style="dim", box=None, padding=(0, 2, 0, 0))
197          table.add_column("#")
198          table.add_column("Source")
199          table.add_column("Location")
200          for i, hit in enumerate(hits, start=1):
201              marker = "[green]*[/green]" if i in cited else " "
202              table.add_row(f"{marker}{i}", escape(Path(hit.chunk.source).name), escape(hit.location))
203          console.print()
204          console.print(table)
205          console.print("[dim]* = cited in the answer[/dim]")
206
```

The source table. **195** parses the citations out of the streamed text, and **201** marks the ones the answer actually used - so a reader sees both what was retrieved and what was relied on, which is the honest presentation of a RAG result.

```
207
208  @app.command()
209  def chat(
210      index_dir: Optional[str] = typer.Option(None, "--index"),
211      top_k: Optional[int] = typer.Option(None, "--top-k", "-k", help="Max passages (default: fill the token
         budget)."),
212      model: Optional[str] = typer.Option(None, "--model", help="Model id, e.g. gemini-3.8-flash."),
213      provider: Optional[str] = typer.Option(None, "--provider", help="gemini | claude (default: whichever ha
         s a key)."),
214  ) -> None:
215      """Interactive multi-turn session. Each turn retrieves afresh."""
216      rag = _pipeline(index_dir, top_k=top_k, model=model, provider=provider)
217      _warn_if_stale(rag)
218      stats = rag.stats()
219      console.print(
220          Panel(
221              f"{stats['chunks']} chunks from {stats['sources']} sources. "
222              "Ask a question, or /exit to quit.",
223              title="RAG chat",
224              border_style="green",
225          )
226      )
227
```

The chat command. Its docstring states the design in four words: each turn retrieves afresh. A follow-up question gets its own retrieval rather than relying on passages fetched for an earlier one.

```
228      history: list[dict] = []
229      while True:
230          try:
231              question = console.input("[bold cyan]>[/bold cyan] ").strip()
232          except (EOFError, KeyboardInterrupt):
233              console.print()
234              break
235          if not question:
236              continue
237          if question in {"/exit", "/quit", "/q"}:
238              break
239          if question == "/clear":
240              history.clear()
241              console.print("[dim]History cleared.[/dim]")
242              continue
243
```

The input loop. **232-234** treat Ctrl-C and end-of-input as a clean exit rather than a traceback. **237-242** are the two commands: quit, in three spellings, and clear the history.

```
244         try:
245             answer = rag.ask(question, top_k=top_k, history=history)
246         except RuntimeError as exc:
247             console.print(f"[red]{exc}[/red]")
248             continue
249 
250         console.print(Markdown(answer.text))
251         if answer.cited_sources:
252             locations = ", ".join(s.location for s in answer.cited_sources)
253             console.print(f"[dim]Sources: {escape(locations)}[/dim]")
254         console.print()
255         history.append({"role": "user", "content": question})
256         history.append({"role": "assistant", "content": answer.text})
257         # Keep the window short: retrieval, not history, carries the context.
258         history[:] = history[-8:]
259 
```

A turn. **246-248** keep the session alive after an error - a rate limit should not end a conversation. **250** renders markdown. **257-258** cap the history at four exchanges, and the comment gives the reasoning: in this system retrieval carries the context, so a long history adds cost without adding grounding.

```
260 
261 @app.command()
262 def stats(index_dir: Optional[str] = typer.Option(None, "--index")) -> None:
263     """Show what is in the index."""
264     rag = _pipeline(index_dir)
265     info = rag.stats()
266     table = Table(box=None, show_header=False, padding=(0, 2, 0, 0))
267     for key in ("index_dir", "sources", "chunks", "tokens", "avg_chunk_tokens", "embedder", "dim"):
268         table.add_row(key, str(info.get(key)))
269     table.add_row("formats", ", ".join(f"{k}:{v}" for k, v in sorted(info["by_format"].items())) or "-")
270     table.add_row("retrieval", "dense + bm25" if info["dense"] else "bm25 only")
271 
```

The stats command: index location, size, embedder and retrieval mode.

```
272     answers = (
273         f"{info['provider']} ({info['answer_model']})"
274         if info.get("provider")
275         else "[yellow]no API key - retrieval only[/yellow]"
276     )
277     table.add_row("answers", answers)
278     keys = ", ".join(info["keys_found"]) or "none"
279     if info.get("env_file_keys"):
280         keys += f"  (from {info['env_filename']}: {', '.join(info['env_file_keys'])})"
281     table.add_row("keys found", keys)
282     console.print(table)
283     if not info["keys_found"]:
284         console.print(
285             f"[dim]To answer questions, put a key in {info['env_filename']} "
286             "(GEMINI_API_KEY=...) -- free key: https://aistudio.google.com/apikey[/dim]"
287         )
288     if info.get("warning"):
289         console.print(f"[yellow]![/yellow] {info['warning']}")
290     if not info["chunks"]:
291         console.print("[dim]Nothing indexed yet: python -m rag ingest <path>[/dim]")
292 
```

The part of stats that exists to answer "why is it not answering". **272-277** name the provider and model, or say plainly that there is no key and retrieval still works. **278-281** show which keys were found and which of them came from the .env file, which is what distinguishes "no key" from "a key that is not being picked up". **283-291** each print a next step: where to put a key, that settings have drifted, or that nothing is indexed yet.

```
293
294 @app.command()
295 def sources(index_dir: Optional[str] = typer.Option(None, "--index")) -> None:
296     """List indexed files."""
297     rag = _pipeline(index_dir)
298     if not rag.store.sources:
299         console.print("[dim]Nothing indexed.[/dim]")
300         return
301     table = Table(show_header=True, header_style="dim", box=None, padding=(0, 2, 0, 0))
302     table.add_column("Source")
303     table.add_column("Format")
304     table.add_column("Chunks", justify="right")
305     for source, meta in sorted(rag.store.sources.items()):
306         table.add_row(source, str(meta.get("format", "-")), str(meta.get("chunks", 0)))
307     console.print(table)
308
```

Listing indexed files with their formats and chunk counts - the quickest way to check whether a document that should be searchable actually made it in.

```
309
310 @app.command()
311 def remove(
312     source: str = typer.Argument(..., help="Exact source path to drop from the index."),
313     index_dir: Optional[str] = typer.Option(None, "--index"),
314 ) -> None:
315     """Remove one source from the index."""
316     rag = _pipeline(index_dir)
317     removed = rag.remove(source)
318     if removed:
319         console.print(f"[green]OK[/green] removed {removed} chunks from {source}")
320     else:
321         console.print(f"[yellow]Not indexed:[/yellow] {source}")
322         raise typer.Exit(1)
323
```

Removing one source. **320-322** report a path that was not indexed and exit non-zero, rather than silently reporting success for a no-op.

```
324
325 @app.command()
326 def reset(
327     index_dir: Optional[str] = typer.Option(None, "--index"),
328     yes: bool = typer.Option(False, "--yes", "-y", help="Skip the confirmation prompt."),
329 ) -> None:
330     """Delete the whole index."""
331     rag = _pipeline(index_dir)
332     target = rag.store.dir
333     if not target.exists():
334         console.print("[dim]No index to delete.[/dim]")
335         return
336     if not yes and not typer.confirm(f"Delete the index at {target}?"):
337         raise typer.Abort()
338     rag.reset()
339     console.print(f"[green]OK[/green] deleted {target}")
340
```

Deleting the index. **333-335** handle there being nothing to delete. **336-337** confirm first unless the user passed the flag that says not to - this is the one destructive command, and it should be hard to run by accident.

```
341
342 @app.command()
343 def serve(
344     host: str = typer.Option("127.0.0.1", "--host"),
345     port: int = typer.Option(8000, "--port"),
346     index_dir: Optional[str] = typer.Option(None, "--index"),
347 ) -> None:
348     """Run the HTTP API (needs: pip install uvicorn)."""
349     try:
350         import uvicorn
351     except ImportError:
352         console.print("[red]Install the server extras: pip install uvicorn[/red]")
353         raise typer.Exit(1)
354     if index_dir:
355         import os
356
357         os.environ["RAG_INDEX_DIR"] = index_dir
358     console.print(f"Test page: [bold]http://{host}:{port}/[/bold]  (Ctrl+C to stop)")
359     uvicorn.run("rag.server:app", host=host, port=port, log_level="warning")
360
```

Running the HTTP API. **349-353** turn a missing dependency into the install command. **354-357** pass the index location through the environment, because the server is started by name in a separate process and cannot be handed an object. **358** prints the URL, since the test page is the point of this command.

```
361
362 def _force_utf8_output() -> None:
363     """Emit UTF-8 even when stdout is a pipe or a file.
364
365     An interactive Windows console already receives Unicode through the console
366     API, but a pipe or redirect falls back to the ANSI code page (cp1256 on
367     Arabic-locale Windows). Arabic then arrives as mojibake in anything reading
368     the output -- `search --json | ...` included -- or fails outright for
369     characters the code page lacks.
370     """
371     for stream in (sys.stdout, sys.stderr):
372         try:
373             stream.reconfigure(encoding="utf-8", errors="replace")
374         except (AttributeError, ValueError):  # replaced or detached streams
375             pass
376
```

A Windows-specific fix, documented because it is invisible until it bites. An interactive console receives Unicode through the console API, but a pipe or a redirect falls back to the ANSI code page - which on an Arabic-locale Windows is cp1256. Arabic output then arrives as mojibake in anything reading it, or fails outright on characters the code page lacks. **371-375** force UTF-8 on both streams, tolerating streams that have been replaced or detached.

```
377
378 def main() -> None:
379     _force_utf8_output()
380     try:
381         app()
382     except KeyboardInterrupt:
383         console.print("\n[dim]Interrupted.[/dim]")
384         sys.exit(130)
385
386
387 if __name__ == "__main__":
388     main()
```

The entry point. **382-384** turn Ctrl-C into a short message and the conventional exit code for an interrupt, rather than a traceback that looks like a crash.

rag/server.py

216 lines. The HTTP API and the test page it serves. Seven endpoints, one of them streaming, and one deliberately disabled by default.

```
"""HTTP API.

    python -m rag serve --port 8000

    GET  /                the test page
    GET  /health
    GET  /stats
    POST /search       {"query": "...", "top_k": 5}
    POST /ask          {"question": "...", "history": [...]}
    POST /ask/stream   same body, Server-Sent Events
    POST /ingest       {"paths": [...]}   (disabled unless RAG_ALLOW_INGEST=1)

The pipeline is loaded once at startup: embedding weights take seconds to load
and the whole index is read into memory, so per-request construction would
dominate latency.
"""
```

The docstring is the API reference. **11** flags the one endpoint that is off unless explicitly enabled. **13-15** explain the startup design: the pipeline is built once, because embedding weights take seconds to load and the index is read into memory - building either per request would dominate the latency of every call.

```

from __future__ import annotations

import json
import os
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Any, AsyncIterator, Iterator

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse, StreamingResponse
from pydantic import BaseModel, Field

from .config import Config
from .pipeline import RAGPipeline
from .retriever import make_filter

```

Imports. FastAPI for routing, pydantic for request validation, and the three pieces of the package the API actually needs.

```
_pipeline: RAGPipeline | None = None


def get_pipeline() -> RAGPipeline:
    global _pipeline
    if _pipeline is None:
        _pipeline = RAGPipeline(Config())
    return _pipeline

```

One shared pipeline for the process, created on first use. The global is deliberate: this object holds the index and the model weights, and a second copy would double the memory for no benefit.

```

@asynccontextmanager
async def lifespan(_: FastAPI) -> AsyncIterator[None]:
    # Load model weights at startup, not on the first user request: the
    # embedder and the reranker together take several seconds to load.
    rag = get_pipeline()
    _ = rag.embedder
    if rag.store.size:
        rag.search("warm-up", top_k=1)  # also builds BM25 and loads the reranker
    yield

```

Startup. **48-49** load the embedder before any request arrives. **50-51** run a throwaway search, which is doing more than it looks: it builds the keyword index and loads the reranker as well, so the first real question is answered at full speed instead of paying several seconds of model loading while a user waits.

```

app = FastAPI(
    title="RAG API",
    version="1.0.0",
    description="Hybrid retrieval over local documents, answered by Gemini or Claude.",
    lifespan=lifespan,
)

```

The application, with its lifespan hook attached.

```

class SearchRequest(BaseModel):
    query: str = Field(..., min_length=1)
    top_k: int = Field(5, ge=1, le=50)
    source: str | None = None
    format: str | None = None
    # None means "use the configured default" rather than "off".
    rerank: bool | None = None

```

The search request shape. Pydantic validates it before any handler runs, so a missing query or an out-of-range depth is rejected with a clear message rather than failing deeper in. **68-69** note the three-state rerank flag: null means "use the configured default", which is different from false.

```

class AskRequest(BaseModel):
    question: str = Field(..., min_length=1)
    # None lets context_tokens decide how many passages fit, which is the
    # measured-better behaviour; a number caps it.
    top_k: int | None = Field(None, ge=1, le=50)
    source: str | None = None
    format: str | None = None
    rerank: bool | None = None
    history: list[dict[str, Any]] = Field(default_factory=list)

```

The ask request. **74-76** record why the default depth is null rather than a number: letting the token budget decide how many passages fit measured better than any fixed count, so the API's default is the measured-best behaviour and a number is an override.

```

class IngestRequest(BaseModel):
    paths: list[str] = Field(..., min_length=1)
    force: bool = False


PAGE = Path(__file__).parent / "web" / "index.html"

```

The ingest request, and the path to the test page.

```

@app.get("/", include_in_schema=False)
def index() -> FileResponse:
    """The test page: ask questions, see which articles answered them."""
    return FileResponse(PAGE, media_type="text/html")

```

Serving the page itself at the root, which is what makes the serve command useful.

```

@app.get("/examples", include_in_schema=False)
def examples(limit: int = 4) -> list[str]:
    """A few questions to click on the test page.

    Taken from the evaluation set when there is one, so the page opens with
    questions that are known to have answers in the indexed documents.
    """
    dataset = Path(__file__).resolve().parent.parent / "eval" / "laws_questions.jsonl"
    if not dataset.exists():
        return []
```

Example questions for the page. They are read from the evaluation set, so the page opens with questions that are known to have answers in the indexed documents - a first impression that demonstrates the system rather than testing the user's luck. **105-106** return nothing when there is no evaluation set, and the page simply shows no examples.

```
    by_kind: dict[str, list[str]] = {}
    for line in dataset.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        try:
            row = json.loads(line)
        except json.JSONDecodeError:
            continue
        by_kind.setdefault(row.get("kind", "all"), []).append(row["question"])
    # One of each kind first, so the page shows the range (Arabic, English,
    # article lookup) rather than four variations of the same thing.
    picked: list[str] = []
    for kind in sorted(by_kind):
        picked.extend(by_kind[kind][:1])
    for kind in sorted(by_kind):
        picked.extend(q for q in by_kind[kind][1:3] if q not in picked)
    return picked[:limit]


```

Choosing which examples to show. **117-121** take one question of each kind first, so the page displays the range of what the system handles - Arabic, English, a direct article lookup - rather than four variations of one thing. **122-123** fill the remaining slots without repeating.

```


@app.get("/health")
def health() -> dict[str, Any]:
    rag = get_pipeline()
    return {"status": "ok", "chunks": rag.store.size, "dense": rag.store.embeddings is not None}


@app.get("/stats")
def stats() -> dict[str, Any]:
    return get_pipeline().stats()

```

Health and stats. Health answers the two questions a monitor cares about: is it up, and does it have anything indexed.

```

@app.post("/search")
def search(req: SearchRequest) -> dict[str, Any]:
    rag = get_pipeline()
    hits = rag.search(
        req.query,
        top_k=req.top_k,
        chunk_filter=make_filter(req.source, req.format),
        rerank=req.rerank,
    )
    return {"query": req.query, "results": [h.to_dict() for h in hits]}

```

Search, which is the pipeline call plus serialisation.

```

@app.post("/ask")
def ask(req: AskRequest) -> dict[str, Any]:
    rag = get_pipeline()
    try:
        answer = rag.ask(
            req.question,
            top_k=req.top_k,
            chunk_filter=make_filter(req.source, req.format),
            history=req.history,
            rerank=req.rerank,
        )
    except RuntimeError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc
    return answer.to_dict()

```

Ask. **161-162** translate a provider failure into a 502 - the upstream model failed, not this service - carrying the explanatory message from the backend so the caller sees "Gemini rejected the API key" rather than a bare status code.

```
165
166 @app.post("/ask/stream")
167 def ask_stream(req: AskRequest) -> StreamingResponse:
168     """SSE: a `sources` event first, then `delta` events, then `done`."""
169     rag = get_pipeline()
170     hits, stream = rag.ask_stream(
171         req.question,
172         top_k=req.top_k,
173         chunk_filter=make_filter(req.source, req.format),
174         history=req.history,
175         rerank=req.rerank,
176     )
177
```

The streaming endpoint. The sources are resolved before the generator is created, which is what allows them to be sent first.

```
178     def events() -> Iterator[str]:
179         payload = [h.to_dict() for h in hits]
180         yield f"event: sources\ndata: {json.dumps(payload, ensure_ascii=False)}\n\n"
181         try:
182             for delta in stream:
183                 yield f"event: delta\ndata: {json.dumps({'text': delta}, ensure_ascii=False)}\n\n"
184         except RuntimeError as exc:
185             yield f"event: error\ndata: {json.dumps({'detail': str(exc)})}\n\n"
186             return
187         yield "event: done\ndata: {}\n\n"
188
```

The event stream. **179-180** send the sources as the opening event, so the page can render the citation list while the answer is still being written. **182-183** send each text fragment as it arrives. **184-186** turn a mid-stream failure into an error event rather than a truncated response that would look like a complete answer. **187** closes the stream explicitly, so the client knows the difference between finished and disconnected.

```
189     return StreamingResponse(
190         events(),
191         media_type="text/event-stream",
192         headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
193     )
194
195
```

The streaming response headers. **192** disables proxy buffering, without which a reverse proxy would hold the whole answer and deliver it in one block - which defeats the entire point of streaming.

```
196 @app.post("/ingest")
197 def ingest(req: IngestRequest) -> dict[str, Any]:
198     """Index paths on the server's filesystem.
199
200     Off by default: it reads arbitrary server-side paths, which is a real
201     capability to hand to an HTTP caller. Enable deliberately with
202     ``RAG_ALLOW_INGEST=1`` behind whatever authentication you already run.
203     """
204     if os.environ.get("RAG_ALLOW_INGEST", "").lower() not in {"1", "true", "yes"}:
205         raise HTTPException(
206             status_code=403,
207             detail="Ingestion over HTTP is disabled. Set RAG_ALLOW_INGEST=1 to enable.",
208         )
209     rag = get_pipeline()
```

Ingestion over HTTP, off by default. The docstring is explicit about why: this endpoint reads arbitrary paths on the server's filesystem, which is a real capability to hand to an HTTP caller. **204-208** refuse unless the environment variable is set, and the message names the variable - so enabling it is a deliberate act, behind whatever authentication the deployment already has.

```
210     report = rag.ingest(req.paths, force=req.force)
211     return {
212         "summary": report.summary(),
213         "added_chunks": report.added_chunks,
214         "failures": report.failures,
215         "chunks": rag.store.size,
216     }
```

The ingest result, summarised the same way the CLI summarises it.

rag/__init__.py

38 lines. The package's front door: what you get when you write 'import rag', and a three-line example of the whole system.

```
"""A hybrid-retrieval RAG system: local embeddings + BM25, answers from Gemini or Claude.

Quick start::

    from rag import RAGPipeline

    rag = RAGPipeline()
    rag.ingest(["./docs"])
    print(rag.ask("What changed in the v2 release?").text)
"""
```

The docstring doubles as the quickest possible tutorial: construct a pipeline, point it at a folder, ask a question. Three lines of real, working code is a more honest description of an API than a paragraph about it - and because it is in the module docstring, `help(rag)` shows it.

```

from .config import Config, DEFAULT_CONFIG
from .chunking import Chunk, chunk_document, count_tokens
from .generate import Answer
from .loaders import Document, load_document, discover
from .pipeline import RAGPipeline
from .retriever import RetrievedChunk, Retriever, make_filter
from .store import VectorStore, IngestReport

```

The re-exports, which define the supported surface. Importing them here means a caller writes `from rag import RAGPipeline` rather than naming the module it happens to live in today - so the internal layout can change without breaking anyone. The imports are ordered by module name, not by importance, which keeps the list mechanical to maintain.

```
__version__ = "1.0.0"

```

The version. One string, read by the HTTP API's health endpoint and by the CLI's `--version` flag, so both report the same thing.

```
__all__ = [
    "RAGPipeline",
    "Config",
    "DEFAULT_CONFIG",
    "Answer",
    "Chunk",
    "Document",
    "RetrievedChunk",
    "Retriever",
    "VectorStore",
    "IngestReport",
    "chunk_document",
    "count_tokens",
    "load_document",
    "discover",
    "make_filter",
]
```

The explicit public list. It controls what a star-import brings in, but its real value is as documentation: fourteen names out of roughly a hundred defined across the package, which is the difference between what is supported and what merely exists. The pipeline comes first because it is the one thing most callers need.